



MONASH
University

Monash University

A Succulent Chinese Algorithm

Indra Kusumah-Kasim, Alex Yu, and Parsa Pordastan

2026-10-08

Contents

- 1 Contest
- 2 Mathematics
- 3 Data structures
- 4 Numerical
- 5 Number theory
- 6 Combinatorial
- 7 Graph
- 8 Strings
- 9 Geometry
- 10 Various

Alphabetical index

Contest (1)

template.cpp

```
#include <bits/stdc++.h>
using namespace std;
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define pb push_back
#define nl '\n'
#define fr first
#define sc second
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

int main() {
    cin.tie(0)->sync_with_stdio(0);
}
```

.bashrc

```
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++20 \
    -fsanitize=undefined,address'
```

.vimrc

```
set smartindent
set splitright
syntax on
set tabstop=2
set shiftwidth=2
set number
set smartcase
set incsearch
set hlsearch
set mouse=a
set clipboard=unnamed #plus

" Select region and then type :Hash to hash your selection.
" Useful for verifying that there aren't mistypes.
ca Hash w !cpp -dD -P -fpreprocessed \ | tr -d '[:space:]' \
\ | sh -c 'if command -v md5sum >/dev/null; then md5sum; else md5 -q; fi'
\
\ | cut -c-6
```

hash.sh

```
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]' | {
    if command -v md5sum >/dev/null 2>&1; then md5sum
    else md5 -q
    fi
} | cut -c-6
```

troubleshoot.txt

```
Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.
```

Runtime error:

Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?

Random.h

Description: Seeded RNG and drawing random integers. randll(l,r) draws from [l,r]: both endpoints included. For string hashing, take a random odd 61-bit base.

```
mt19937_64 rng((uint64_t)chrono::steady_clock::now()
    .time_since_epoch().count());

ll randll(ll l, ll r) { // inclusive
    return uniform_int_distribution<ll>(l, r)(rng);
}

// shuffle(all(v), rng);
// Odd 61-bit hash base:
// ll base = randll(256, (1LL << 61) - 2) | 1;
```

1.1 I/O

Input.h

Description: scanf reference. Arguments are pointers to the exact types below. The return value is the number of assignments, or EOF if input ends before the first conversion. Check it before use.

```
// int: %d (decimal), unsigned: %u, long long: %lld,
// unsigned long long: %llu. %i auto-detects octal/hex.
int i; unsigned u; ll n;
if (scanf("%d %u %lld", &i, &u, &n) != 3) return 0;

// Unlike printf: %f needs float*, %lf needs double*.
float f; double x; long double y;
if (scanf("%f %lf %Lf", &f, &x, &y) != 3) return 0;

// %s reads a token; bound width to buffer size minus 1.
char s[100];
if (scanf("%99s", s) != 1) return 0; // no &s
// It appends '\0'; don't pass std::string to scanf.
char c;
if (scanf("%c", &c) != 1) return 0;
// %c alone reads whitespace too; leading space skips it.
// Format whitespace consumes any amount of whitespace;
// avoid trailing spaces or '\n' (can wait for more input).
// With sync_with_stdio(false), don't mix cin and scanf.
```

Output.h

Description: Output formatting with streams, C++20 std::format, and printf. Field width is a minimum.

d41d8c, 38 lines

```
#include <format>

// Binary: format needs C++20 library support.
cout << format("{:b}", 5); // 101
cout << format("{:08b}", 5); // 00000101
cout << bitset<8>(5); // 00000101, low 8 bits
// bitset<N> prints exactly N bits; N is compile-time.

// Streams: precision/fill/alignment persist; setw is once.
cout << fixed << setprecision(2) << 1.5; // 1.50
cout << defaultfloat << setprecision(3) << 1.2345;
// Above: 1.23 (defaultfloat uses significant digits).
cout << setfill('0') << setw(5) << 42; // 00042
cout << setfill(' ') << left << setw(5) << "hi";
// Above: "hi  "; restore right for right alignment.
cout << right;

// format: padding/alignment, decimal places, runtime width.
cout << format("{:05}", 42); // 00042
cout << format("{:<5}", "hi"); // "hi  "
cout << format("{:8.2f}", 1.5); // "      1.50"
cout << format("{:03f}", 1.5, 3); // 1.500
cout << format("{:03b}", 5, 8); // 00000101

// printf: int %d, unsigned %u, long long %lld,
// unsigned long long %llu, C string %s, char %c.
// float promotes to double: %f; long double: %Lf.
int i = 42; unsigned u = 7; ll n = 12345678901LL;
string s = "hi"; double x = 1.5; long double y = 2.5L;
printf("%d %u %lld\n", i, u, n);
printf("%s %c\n", s.c_str(), 'A');
printf("%.2f %.2Lf\n", x, y); // 1.50 2.50
printf("%05d\n", i); // 00042 (sign before zeros)
printf("%-5s\n", s.c_str()); // "hi  !"
printf("%8.2f\n", x); // "      1.50"
printf("%*.f\n", 8, 2, x); // runtime width/precision
// %e: scientific; %3g: 3 significant digits; %l: literal %.
// With sync_with_stdio(false), don't mix cout and printf.
```

Mathematics (2)

2.1 Mathematical constants

Constant	Approximate value	C++20 name
π	3.14159265358979323846	pi
e	2.71828182845904523536	e
$\tau = 2\pi$	6.28318530717958647693	2*pi
$\sqrt{2}$	1.41421356237309504880	sqrt2
$\sqrt{3}$	1.73205080756887729353	sqrt3
$\ln 2$	0.69314718055994530942	ln2
$\ln 10$	2.30258509299404568402	ln10
$\phi = \frac{1+\sqrt{5}}{2}$	1.61803398874989484820	phi
γ	0.57721566490153286061	egamma

With #include <numbers>, names above are in std::numbers; e.g. std::numbers::pi is a constexpr double. For long double, use std::numbers::pi_v<long double> (likewise e_v, ln2_v, etc.). For τ , multiply the chosen π constant by 2. Before C++20, use acos(-1.0) and exp(1.0) from <cmath>; use acos(-1.0L) and exp(1.0L) for long double.

Angles: $180^\circ = \pi$ radians; multiply degrees by $\pi/180$ or radians by $180/\pi$. Log bases: $\log_2 x = \ln x / \ln 2$, $\log_{10} x = \ln x / \ln 10$. Golden ratio: $\phi^2 = \phi + 1$, $1/\phi = \phi - 1$ (Fibonacci growth and golden-section search). Euler–Mascheroni constant: $\gamma = \lim_{n \rightarrow \infty} (H_n - \ln n)$, where $H_n = \sum_{k=1}^n 1/k$; for large n ,

$$H_n = \ln n + \gamma + \frac{1}{2n} - \frac{1}{12n^2} + O(n^{-4}).$$

2.2 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by &= e & x &= \frac{ed - bf}{ad - bc} \\ cx + dy &= f & y &= \frac{af - ec}{ad - bc} \end{aligned} \Rightarrow$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A'_i}{\det A}$$

where A'_i is A with the i 'th column replaced by b .

2.3 Recurrences

If $a_n = c_1 a_{n-1} + \dots + c_k a_{n-k}$, and $r_1, \dots, r_k$ are distinct roots of $x^k - c_1 x^{k-1} - \dots - c_k$, there are $d_1, \dots, d_k$ s.t.

$$a_n = d_1 r_1^n + \dots + d_k r_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1 n + d_2) r^n$.

2.4 Trigonometry

$$\begin{aligned} \sin(v + w) &= \sin v \cos w + \cos v \sin w \\ \cos(v + w) &= \cos v \cos w - \sin v \sin w \end{aligned}$$

$$\begin{aligned} \tan(v + w) &= \frac{\tan v + \tan w}{1 - \tan v \tan w} \\ \sin v + \sin w &= 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2} \\ \cos v + \cos w &= 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2} \end{aligned}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w .

$$\begin{aligned} a \cos x + b \sin x &= r \cos(x - \phi) \\ a \sin x + b \cos x &= r \sin(x + \phi) \end{aligned}$$

where $r = \sqrt{a^2 + b^2}$, $\phi = \operatorname{atan2}(b, a)$.

2.5 Geometry

2.5.1 Triangles

Side lengths: a, b, c

$$\text{Semiperimeter: } p = \frac{a + b + c}{2}$$

$$\text{Area: } A = \sqrt{p(p - a)(p - b)(p - c)}$$

$$\text{Circumradius: } R = \frac{abc}{4A}$$

$$\text{Inradius: } r = \frac{A}{p}$$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2} \sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc \left[1 - \left(\frac{a}{b + c} \right)^2 \right]}$$

$$\text{Law of sines: } \frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$$

$$\text{Law of cosines: } a^2 = b^2 + c^2 - 2bc \cos \alpha$$

$$\text{Law of tangents: } \frac{a + b}{a - b} = \frac{\tan \frac{\alpha + \beta}{2}}{\tan \frac{\alpha - \beta}{2}}$$

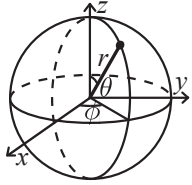
2.5.2 Quadrilaterals

With side lengths a, b, c, d , diagonals e, f , diagonals angle θ , area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2 f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is 180° , $ef = ac + bd$, and $A = \sqrt{(p - a)(p - b)(p - c)(p - d)}$.

2.5.3 Spherical coordinates



$$\begin{aligned}x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\y &= r \sin \theta \sin \phi & \theta &= \arccos(z / \sqrt{x^2 + y^2 + z^2}) \\z &= r \cos \theta & \phi &= \operatorname{atan2}(y, x)\end{aligned}$$

2.6 Derivatives/Integrals

$$\begin{aligned}\frac{d}{dx} \arcsin x &= \frac{1}{\sqrt{1-x^2}} & \frac{d}{dx} \arccos x &= -\frac{1}{\sqrt{1-x^2}} \\ \frac{d}{dx} \tan x &= 1 + \tan^2 x & \frac{d}{dx} \arctan x &= \frac{1}{1+x^2} \\ \int \tan ax &= -\frac{\ln |\cos ax|}{a} & \int x \sin ax &= \frac{\sin ax - ax \cos ax}{a^2} \\ \int e^{-x^2} &= \frac{\sqrt{\pi}}{2} \operatorname{erf}(x) & \int x e^{ax} dx &= \frac{e^{ax}}{a^2} (ax - 1)\end{aligned}$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.7 Sums

$$c^a + c^{a+1} + \dots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

$$1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.8 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots, (-1 < x \leq 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \dots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots, (-\infty < x < \infty)$$

2.9 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x . It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y ,

$$V(aX + bY) = a^2 V(X) + b^2 V(Y).$$

2.9.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\operatorname{Bin}(n, p)$, $n = 1, 2, \dots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1-p)^{n-k}$$

$$\mu = np, \sigma^2 = np(1-p)$$

$\operatorname{Bin}(n, p)$ is approximately $\operatorname{Po}(np)$ for small p .

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\operatorname{Fs}(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1-p)^{k-1}, k = 1, 2, \dots$$

$$\mu = \frac{1}{p}, \sigma^2 = \frac{1-p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\operatorname{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, k = 0, 1, 2, \dots$$

$$\mu = \lambda, \sigma^2 = \lambda$$

2.9.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $\operatorname{U}(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2}, \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\operatorname{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda}, \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.10 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \dots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

π is a stationary distribution if $\pi = \pi \mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i . π_j / π_i is the expected number of visits in state j between two visits in state i .

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i 's degree.

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k \rightarrow \infty} \mathbf{P}^k = \mathbf{1}\pi$.

A Markov chain is an A-chain if the states can be partitioned into two sets $\mathbf{A}$ and $\mathbf{G}$, such that all states in $\mathbf{A}$ are absorbing ($p_{ii} = 1$), and all states in $\mathbf{G}$ leads to an absorbing state in $\mathbf{A}$. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j , is $a_{ij} = p_{ij} + \sum_{k \in \mathbf{G}} a_{ik} p_{kj}$. The expected time until absorption, when the initial state is i , is $t_i = 1 + \sum_{k \in \mathbf{G}} p_{ki} t_k$.

Data structures (3)

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n 'th element, and finding the index of an element. To get a map, change `null_type`. `order_of_key(x)` counts keys strictly below x (x excluded). `find_by_order(k)` is 0-indexed; returns `end()` if $k \geq \text{size}()$.
Time: $\mathcal{O}(\log N)$

```
#include <bits/extc++.h>
using namespace __gnu_pbds;

template<class T>
using Tree = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
// MEGA WARNING: use A.swap(B) instead of swap(A, B)!

void example() {
    Tree<int> t, t2; t.insert(8);
    auto it = t.insert(10).first;
    assert(it == t.lower_bound(9));
    assert(t.order_of_key(10) == 1);
    assert(t.order_of_key(11) == 2);
    assert(*t.find_by_order(0) == 8);
```

d41d8c, 17 lines

```
t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

Rope.h

Description: GNU extension for editable sequences. Copies share storage; later edits leave copies unchanged. Indices are 0-based; erase/substr take (position, length). They use `[pos, pos + len)`: pos included, pos+len excluded. No range aggregates or lazy reversal. Requires `libstdc++`.
Time: Index $\mathcal{O}(\log N)$, copy $\mathcal{O}(1)$. Edits typically $\mathcal{O}(\log N)$ plus new data, but can take $\mathcal{O}(N)$.

d41d8c, 18 lines

```
#include <ext/rope>
using __gnu_cxx::rope;

void ropeExample() {
    int a[] = {0, 1, 2, 3, 4};
    rope<int> r(a, a + 5); // or rope<int>(n, value)
    rope<int> saved = r; // cheap snapshot
    r.insert(2, rope<int>(3, 10)); // insert before index 2
    r.erase(1, 3); // remove [1, 4): {0, 10, 2, 3, 4}
    r.replace(0, 9); // set index 0 (also r[0] = 9)
    rope<int> sub = r.substr(1, 2); // {10, 2}
    r += sub; // concatenate; r.push_back(x) also works
    vector<int> v(all(r)); // materialize in O(N)
    assert(v == (vi{9, 10, 2, 3, 4, 10, 2}));
    assert(saved[0] == 0 && saved.size() == 5);
    rope<char> s("abc"); // string variant
    s.insert(1, "XY"); // aXYbc
}
```

WaveletTree.h

Description: Static range order statistics on an int array. Compresses values; preserves input. All queries use indices $[l, r)$: l included, r excluded. `kth(l, r, k)` is the 0-indexed k th smallest in $a[l, r)$, including duplicates; requires $0 \leq k < r - l$. `countLess(l, r, x)` counts values $< x$; `count(l, r, lo, hi)` counts $lo \leq a_i < hi$. Value range $[lo, hi)$: lo included, hi excluded. Counts allow empty ranges. Thresholds are 1l so $x + 1LL$ is safe for inclusive bounds, even at `INT_MAX`.

Usage: `WaveletTree w({5, 1, 5, -2, 7});`
`w.kth(1, 5, 1);` // 1: sorted subarray is -2,1,5,7
`w.count(0, 5, 5, 6);` // 2 copies of 5
`w.kth(l, r, (r-l-1)/2);` // lower median ($l < r$)
`int k = w.countLess(l, r, x);`
`if (k < r-l) w.kth(l, r, k);` // smallest $\geq x$
Memory: $\mathcal{O}(N \log(\sigma + 1))$, σ distinct values.
Time: $\mathcal{O}(N \log N)$ build, $\mathcal{O}(\log(\sigma + 1))$ query.

d41d8c, 58 lines

```
struct WaveletTree {
    int n;
    vi vals;
    vector<vi> b;
    WaveletTree(const vi& a) : n(sz(a)), vals(a) {
        sort(all(vals));
        vals.erase(unique(all(vals)), vals.end());
        b.resize(4 * sz(vals));
        if (n) build(a, 1, 0, sz(vals));
    }
    void build(const vi& a, int v, int lo, int hi) {
        if (hi-lo <= 1) return;
        int m = lo + (hi-lo)/2;
        vi c[2];
        b[v].resize(sz(a) + 1);
        rep(i, 0, sz(a)) {
            int left = a[i] < vals[m];
```

```
b[v][i+1] = b[v][i] + left;
c[!left].push_back(a[i]);
}
build(c[0], 2*v, lo, m);
build(c[1], 2*v+1, m, hi);
}
int kth(int l, int r, int k) const {
    assert(0 <= l && l <= r && r <= n);
    assert(0 <= k && k < r-l);
    int v = 1, lo = 0, hi = sz(vals);
    while (hi-lo > 1) {
        int m = lo + (hi-lo)/2, lb = b[v][l], rb = b[v][r];
        if (k < rb-lb) {
            l = lb; r = rb; hi = m; v *= 2;
        } else {
            k -= rb-lb; l -= lb; r -= rb;
            lo = m; v = 2*v+1;
        }
    }
    return vals[lo];
}
int countLess(int l, int r, ll x) const {
    assert(0 <= l && l <= r && r <= n);
    if (l == r) return 0;
    int v = 1, lo = 0, hi = sz(vals), ans = 0;
    while (hi-lo > 1) {
        int m = lo + (hi-lo)/2, lb = b[v][l], rb = b[v][r];
        if (x <= vals[m]) {
            l = lb; r = rb; hi = m; v *= 2;
        } else {
            ans += rb-lb; l -= lb; r -= rb;
            lo = m; v = 2*v+1;
        }
    }
    return ans + (vals[lo] < x ? r-l : 0);
}
int count(int l, int r, ll lo, ll hi) const {
    assert(lo <= hi);
    return countLess(l, r, hi) - countLess(l, r, lo);
}
};
```

HashMap.h

Description: Hash map with mostly the same API as `unordered_map`, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided). For a standard map, use the commented alternative below. Randomization is optional; set `RANDOM` to 0 to disable it.

d41d8c, 12 lines

```
#include <bits/extc++.h>
const uint64_t RANDOM = chrono::steady_clock::now()
    .time_since_epoch().count();
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
    const uint64_t C = 1LL(4e18 * acos(0)) | 71;
    size_t operator()(uint64_t x) const {
        return __builtin_bswap64((x ^ RANDOM) * C);
    }
};
__gnu_pbds::gp_hash_table<ll, int, chash> h{ {}, {}, {}, {}, {1 < 16} };
// unordered_map<ll, int, chash> h; // alternative
```

SegmentTree.h

Description: Zero-indexed max-tree. `query(b, e)` uses $[b, e)$: b included, e excluded. Can be changed by modifying `T`, `f` and `unit`.

Time: $\mathcal{O}(\log N)$

d41d8c, 19 lines

```

struct Tree {
    typedef int T;
    static constexpr T unit = INT_MIN;
    T f(T a, T b) { return max(a, b); } // (any associative fn)
    vector<T> s; int n;
    Tree(int n = 0, T def = unit) : s(2*n, def), n(n) {}
    void update(int pos, T val) {
        for (s[pos += n] = val; pos /= 2;)
            s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
    }
    T query(int b, int e) { // query [b, e)
        T ra = unit, rb = unit;
        for (b += n, e += n; b < e; b /= 2, e /= 2) {
            if (b % 2) ra = f(ra, s[b++]);
            if (e % 2) rb = f(s[--e], rb);
        }
        return f(ra, rb);
    }
};

```

LazySegmentTree.h

Description: Recursive segment tree with range updates, point set, and range queries. Update/query bounds $[ql, qr]$ include both endpoints; full range $[0, n - 1]$. Change $V, U, id, def, idU, binop, applyUpdate, mergeUpdate$. Default is range add and range max. Used by HLD.

Time: $\mathcal{O}(\log N)$

d41d8c, 60 lines

```

struct LazyUpdateTree {
    using V = int;
    using U = int;
    static constexpr V id = INT_MIN;
    static constexpr V def = 0;
    static constexpr U idU = 0;
    V binop(V a, V b) { return max(a, b); }
    V applyUpdate(U u, V v) { return v + u; }
    U mergeUpdate(U oldU, U nw) { return oldU + nw; }
    vector<V> arr;
    vector<U> lazy;
    int size;
    LazyUpdateTree(int n) :
        arr(4 * n + 2, def), lazy(4 * n + 2, idU), size(n) {}
    void updateNode(int cur, int, int, U u) {
        lazy[cur] = mergeUpdate(lazy[cur], u);
        arr[cur] = applyUpdate(u, arr[cur]);
    }
    void push(int cur, int l, int r) {
        int mid = l + (r - l) / 2;
        updateNode(2 * cur, l, mid, lazy[cur]);
        updateNode(2 * cur + 1, mid + 1, r, lazy[cur]);
        lazy[cur] = idU;
    }
    void update(int cur, int l, int r, int ql, int qr, U u) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) {
            updateNode(cur, l, r, u); return;
        }
        push(cur, l, r);
        int mid = l + (r - l) / 2;
        update(2 * cur, l, mid, ql, qr, u);
        update(2 * cur + 1, mid + 1, r, ql, qr, u);
        arr[cur] = binop(arr[2 * cur], arr[2 * cur + 1]);
    }
    void update(int ql, int qr, U u) {

```

```

        update(1, 0, size - 1, ql, qr, u);
    }
    void set(int cur, int l, int r, int i, V v) {
        if (i < l || r < i) return;
        if (l == r) { arr[cur] = v; lazy[cur] = idU; return; }
        push(cur, l, r);
        int mid = l + (r - l) / 2;
        if (i <= mid) set(2 * cur, l, mid, i, v);
        else set(2 * cur + 1, mid + 1, r, i, v);
        arr[cur] = binop(arr[2 * cur], arr[2 * cur + 1]);
    }
    void set(int i, V v) { set(1, 0, size - 1, i, v); }
    V query(int cur, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return id;
        if (ql <= l && r <= qr) return arr[cur];
        push(cur, l, r);
        int mid = l + (r - l) / 2;
        return binop(query(2 * cur, l, mid, ql, qr),
            query(2 * cur + 1, mid + 1, r, ql, qr));
    }
    V query(int ql, int qr) {
        return query(1, 0, size - 1, ql, qr);
    }
};

```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip `st`, `time()` and `rollback()`.

Usage: `int t = uf.time(); ...; uf.rollback(t);`**Time:** $\mathcal{O}(\log(N))$

d41d8c, 21 lines

```

struct RollbackUF {
    vi e; vector<pii> st;
    RollbackUF(int n) : e(n, -1) {}
    int size(int x) { return -e[find(x)]; }
    int find(int x) { return e[x] < 0 ? x : find(e[x]); }
    int time() { return sz(st); }
    void rollback(int t) {
        for (int i = time(); i --> t;)
            e[st[i].first] = st[i].second;
        st.resize(t);
    }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
        st.push_back({a, e[a]});
        st.push_back({b, e[b]});
        e[a] += e[b]; e[b] = a;
        return true;
    }
};

```

OfflineDynamicConnectivity.h

Description: Offline dynamic connectivity. Times are operation indices. `toggle` adds or deletes an undirected edge. `query` records a component-count query. `ans` returns answers in order. `add(l, r, e)` makes e active at times $[l, r)$: l included, r excluded. An edge is active from its insertion `toggle` up to, excluding, its deletion `toggle`. q must be at least the number of `toggle`/ `query` calls.

Time: $\mathcal{O}(Q \log Q \log N)$

"UnionFindRollback.h"

d41d8c, 44 lines

```

struct DynCon {
    RollbackUF uf;

```

```

    vector<vector<pii>> st;
    map<pii, int> eds;
    int n, q, t = 0;
    DynCon(int n, int q) : uf(n), n(n), q(1) {
        while (this->q < max(q, 1)) this->q *= 2;
        st.resize(2 * this->q);
    }
    void add(int l, int r, pii e) {
        for (l += q, r += q; l < r; l /= 2, r /= 2) {
            if (l & 1) st[l++].push_back(e);
            if (r & 1) st[--r].push_back(e);
        }
    }
    void toggle(int u, int v) {
        assert(t < q);
        if (u > v) swap(u, v);
        pii e{u, v};
        if (eds.count(e)) {
            add(eds[e], t, e);
            eds.erase(e);
        } else eds[e] = t;
        ++t;
    }
    void query() {
        assert(t < q);
        st[q + t++].push_back({-1, -1});
    }
    void rec(int i, vi& res) {
        int t0 = uf.time();
        for (auto [a, b] : st[i]) if (a >= 0) uf.join(a, b);
        for (auto [a, b] : st[i])
            if (a < 0) res.push_back(n - uf.time() / 2);
        if (i < q) rec(2 * i, res), rec(2 * i + 1, res);
        uf.rollback(t0);
    }
    vi ans() {
        for (auto [e, s] : eds) add(s, t, e);
        vi res;
        rec(1, res);
        return res;
    }
};

```

SubMatrix.h

Description: Calculate submatrix sums quickly. `sum(u, l, d, r)` sums rows $[u, d)$ and columns $[l, r)$: upper/left included, lower/right excluded.

Usage: `SubMatrix<int> m(matrix);``m.sum(0, 0, 2, 2);` // top left 4 elements**Time:** $\mathcal{O}(N^2 + Q)$

d41d8c, 13 lines

```

template<class T>
struct SubMatrix {
    vector<vector<T>>> p;
    SubMatrix(vector<vector<T>>> v) {
        int R = sz(v), C = sz(v[0]);
        p.assign(R+1, vector<T>(C+1));
        rep(r, 0, R) rep(c, 0, C)
            p[r+1][c+1] = v[r][c] + p[r][c+1] + p[r+1][c] - p[r][c];
    }
    T sum(int u, int l, int d, int r) {
        return p[d][r] - p[d][l] - p[u][r] + p[u][l];
    }
};

```

PrefixSum.h

Description: Static sums in any number of dimensions. Construct with each dimension's width and flat row-major values (last coordinate varies fastest). T is the sum type. $\text{sum}(l, r)$ sums cells in $[l_0, r_0) \times [l_1, r_1) \times \dots \times [l_{d-1}, r_{d-1})$. Each lower bound is included and each upper bound is excluded. Both vectors have d coordinates, with $0 \leq l_i \leq r_i \leq w_i$. Empty boxes return zero. Requires at least one dimension, nonnegative widths, and exactly $\prod_i w_i$ values. All suffix products must fit in int; T must hold all intermediate sums.

Usage: // Matrix $\{\{1, 2, 3\}, \{4, 5, 6\}\}$, flattened by row:

PrefixSum<ll> p({2, 3}, {1, 2, 3, 4, 5, 6});

p.sum({0, 1}, {2, 3}); // 2 + 3 + 5 + 6 = 16

Memory: $\mathcal{O}(N)$ prefix table.

Time: $\mathcal{O}(dN)$ build, $\mathcal{O}(2^d)$ query, where $N = \prod_i w_i$ and d is the number of dimensions.

d41d8c, 25 lines

```
template<class T>
struct PrefixSum {
    vi n, stride;
    vector<T> p;
    PrefixSum(vi widths, vector<T> values)
        : n(widths), stride(sz(n) + 1, 1), p(move(values)) {
        for (int d = sz(n); d--;)
            stride[d] = stride[d+1] * n[d];
        rep(d, 0, sz(n)) rep(i, 0, sz(p))
            if (i / stride[d+1] % n[d])
                p[i] += p[i - stride[d+1]];
    }
    T sum(const vi& l, const vi& r) const {
        rep(d, 0, sz(n)) if (l[d] == r[d]) return T{};
        return query(l, r, 0, 0);
    }
    T query(const vi& l, const vi& r, int d, int i) const {
        if (d == sz(n)) return p[i];
        int s = stride[d+1];
        T ans = query(l, r, d+1, i + (r[d]-l)*s);
        if (l[d])
            ans -= query(l, r, d+1, i + (l[d]-1)*s);
        return ans;
    }
};
```

Matrix.h

Description: Basic operations on square matrices.

Usage: Matrix<int, 3> A;

A.d = {{{{1,2,3}}, {{4,5,6}}, {{7,8,9}}}};

array<int, 3> vec = {1,2,3};

vec = (A^N) * vec;

d41d8c, 26 lines

```
template<class T, int N> struct Matrix {
    typedef Matrix M;
    array<array<T, N>, N> d{};
    M operator*(const M& m) const {
        M a;
        rep(i, 0, N) rep(j, 0, N)
            rep(k, 0, N) a.d[i][k] += d[i][j] * m.d[j][k];
        return a;
    }
    array<T, N> operator*(const array<T, N>& vec) const {
        array<T, N> ret{};
        rep(i, 0, N) rep(j, 0, N) ret[i] += d[i][j] * vec[j];
        return ret;
    }
    M operator^(ll p) const {
```

```
    assert(p >= 0);
    M a, b(*this);
    rep(i, 0, N) a.d[i][i] = 1;
    while (p) {
        if (p&1) a = a*b;
        b = b*b;
        p >>= 1;
    }
    return a;
}
```

LineContainer.h

Description: Container where you can add lines of the form $kx+m$, and query maximum values at points x . Useful for dynamic programming (“convex hull trick”).

Time: $\mathcal{O}(\log N)$

d41d8c, 30 lines

```
struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}); y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};
```

MonotonicCHT.h

Description: Add lines $kx + m$ in nondecreasing slope order and query maximum values at nondecreasing x . Additions and queries may be interleaved; query requires a line. For minimum, negate slopes, intercepts and answers (original slopes must then be nonincreasing). Answers must fit in ll; products of coefficient differences must fit in signed $_int128$. LineContainer handles arbitrary slopes and queries.

Memory: $\mathcal{O}(N)$

Time: $\mathcal{O}(1)$ amortized per operation, $\mathcal{O}(N + Q)$ total.

d41d8c, 28 lines

```
struct MonotonicCHT {
    using T = __int128;
    struct Line {
        ll k, m;
        T eval(ll x) const { return (T)k * x + m; }
```

```
};
vector<Line> h;
int p = 0;
bool bad(Line a, Line b, Line c) {
    return (T(b.k)-a.k) * (T(c.m)-b.m)
        >= (T(b.m)-a.m) * (T(c.k)-b.k);
}
void add(ll k, ll m) {
    Line l{k, m};
    if (sz(h) && h.back().k == k && h.back().m >= m)
        return;
    while (sz(h) && (h.back().k == k || (sz(h) > 1
        && bad(h[sz(h)-2], h.back(), l))))
        h.pop_back();
    h.push_back(l), p = min(p, sz(h)-1);
}
ll query(ll x) {
    assert(sz(h));
    while (p+1 < sz(h) && h[p+1].eval(x) > h[p].eval(x))
        p++;
    return (ll)h[p].eval(x);
}
};
```

LiChao.h

Description: Li Chao tree for lines $ax+b$ on an integer domain $[L, R)$: L included, R excluded. Node bounds $[l, r)$ use the same convention. Queries minimum. Negate a, b and the result for maximum. Unlike LineContainer, lines can be restricted to a subrange by inserting on a node range.

Time: $\mathcal{O}(\log(R-L))$

d41d8c, 36 lines

```
struct LiChao {
    struct Fn {
        ll a, b;
        Fn(ll a = 0, ll b = 0) : a(a), b(b) {}
        ll operator()(ll x) const { return a * x + b; }
    };
    static const ll INF = LLONG_MAX / 4;
    struct Node {
        Fn f{0, INF};
        Node *l = 0, *r = 0;
    };
    Node *root = new Node();
    int L, R;
    LiChao(int L, int R) : L(L), R(R) {}
    void insert(Fn nw, Node *n, int l, int r) {
        if (!n) return;
        int m = l + (r - l) / 2;
        bool lef = nw(l) < n->f(l);
        bool mid = nw(m) < n->f(m);
        if (mid) swap(nw, n->f);
        if (l == r - 1) return;
        Node *lch = lef != mid ? n->l : n->r;
        if (!ch) ch = new Node();
        insert(nw, ch, lef != mid ? l : m,
            lef != mid ? m : r);
    }
    void insert(Fn nw) { insert(nw, root, L, R); }
    ll query(int x, Node *n, int l, int r) {
        if (!n || l > x || r <= x) return INF;
        int m = l + (r - l) / 2;
        ll res = n->f(x);
        return x < m ? min(res, query(x, n->l, l, m))
            : min(res, query(x, n->r, m, r));
    }
};
```

```
ll query(int x) { return query(x, root, L, R); }
};
```

Treap.h

Description: Implicit treap on a sequence. Split/merge by index. split(t , k) puts the first k elements on the left: indices $[0, k)$, k excluded. Lazy range updates and queries. Index ranges $[l, r)$ include l and exclude r , including move. Change V , U , id , def , idU , $binop$, $applyUpdate$, $mergeUpdate$. Default is range add/sum.

Time: $\mathcal{O}(\log N)$

d41d8c, 82 lines

```
struct Node {
    using V = ll;
    using U = ll;
    static constexpr V id = 0;
    static constexpr V def = 0;
    static constexpr U idU = 0;
    V binop(V a, V b) { return a + b; }
    V applyUpdate(U u, V v, int len) { return v + u * len; }
    U mergeUpdate(U oldU, U nw) { return oldU + nw; }
    Node *l = 0, *r = 0;
    V val = def, agg = def;
    int y, c = 1;
    U lazy = idU;
    Node(V val = def) : val(val), agg(val), y(rand()) {}
    void apply(U u) {
        val = applyUpdate(u, val, 1);
        agg = applyUpdate(u, agg, c);
        lazy = mergeUpdate(lazy, u);
    }
    void push() {
        if (lazy == idU) return;
        if (l) l->apply(lazy);
        if (r) r->apply(lazy);
        lazy = idU;
    }
    void pull();
};

int cnt(Node* n) { return n ? n->c : 0; }
Node::V query(Node* n) { return n ? n->agg : Node::id; }
void Node::pull() {
    c = cnt(l) + cnt(r) + 1;
    agg = binop(binop(query(l), val), query(r));
}

template<class F> void each(Node* n, F f) {
    if (!n) return;
    n->push(); each(n->l, f);
    f(n->val); each(n->r, f);
}

pair<Node*, Node*> split(Node* n, int k) { // left gets k nodes
    if (!n) return {};
    n->push();
    if (cnt(n->l) >= k) { // "n->val >= k" for lower_bound(k)
        auto [L, R] = split(n->l, k);
        n->l = R;
        n->pull();
        return {L, n};
    } else {
        auto [L, R] = split(n->r, k - cnt(n->l) - 1); // and just "k"
        n->r = L;
        n->pull();
        return {n, R};
    }
}
```

```
}

Node* merge(Node* l, Node* r) {
    if (!l) return r;
    if (!r) return l;
    if (l->y > r->y) {
        l->push();
        l->r = merge(l->r, r);
        return l->pull(), l;
    } else {
        r->push();
        r->l = merge(l, r->l);
        return r->pull(), r;
    }
}

Node* ins(Node* t, Node* n, int pos) {
    auto [l, r] = split(t, pos);
    return merge(merge(l, n), r);
}

// Example application: move the range [l, r) to index k
void move(Node*& t, int l, int r, int k) {
    Node *a, *b, *c;
    tie(a, b) = split(t, l); tie(b, c) = split(b, r - l);
    if (k <= l) t = merge(ins(a, b, k), c);
    else t = merge(a, ins(c, b, k - r));
}

CartesianTree.h
```

CartesianTree.h

Description: Min Cartesian tree with array indices in inorder. Returns parent, left and right child indices; -1 means no parent/child. Equal values favor the leftmost index. Empty input is OK. Remove p , l or r and its assignments if not needed.

Usage: auto $[p, l, r] = \text{cartesianTree}(a)$;

Time: $\mathcal{O}(n)$

d41d8c, 18 lines

```
array<vi, 3> cartesianTree(const vi& a) {
    vi p(sz(a), -1), l = p, r = p, s;
    rep(i, 0, sz(a)) {
        int last = -1;
        while (!s.empty() && a[s.back()] > a[i]) {
            last = s.back();
            s.pop_back();
        }
        if (!s.empty()) {
            p[i] = s.back();
            r[s.back()] = i;
        }
        if (last != -1) p[last] = i;
        l[i] = last;
        s.push_back(i);
    }
    return {p, l, r};
}
```

FenwickTree.h

Description: Point updates and prefix sums. query(pos) sums $[0, pos)$: 0 included, pos excluded. Updates add the difference between the old and new value. With nonnegative values, lower_bound(sum) finds the first index pos whose inclusive prefix $[0, pos]$ sums to at least sum; returns n if absent, or -1 if sum ≤ 0 .

Time: Both operations are $\mathcal{O}(\log N)$.

d41d8c, 22 lines

```
struct FT {
    vector<ll> s;
    FT(int n) : s(n) {}
    void update(int pos, ll dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    ll query(int pos) { // sum of values in [0, pos)
        ll res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
    int lower_bound(ll sum) { // min pos st sum of [0, pos] >= sum
        // Returns n if no sum is >= sum, or -1 if empty sum is.
        if (sum <= 0) return -1;
        int pos = 0;
        for (int pw = 1 << 25; pw; pw >>= 1) {
            if (pos + pw <= sz(s) && s[pos + pw-1] < sum)
                pos += pw, sum -= s[pos-1];
        }
        return pos;
    }
};
```

FenwickTree2d.h

Description: Point increments and 2D prefix sums. query(x, y) sums points with $0 \leq i < x$ and $j < y$: both upper bounds excluded, $i = 0$ included. The y coordinates may be negative. Requires that the elements to be updated are known in advance (call fakeUpdate() before init()).

Time: $\mathcal{O}(\log^2 N)$. (Use persistent segment trees for $\mathcal{O}(\log N)$.)

"FenwickTree.h"

d41d8c, 22 lines

```
struct FT2 {
    vector<vi> ys; vector<FT> ft;
    FT2(int limx) : ys(limx) {}
    void fakeUpdate(int x, int y) {
        for (; x < sz(ys); x |= x + 1) ys[x].push_back(y);
    }
    void init() {
        for (vi& v : ys) sort(all(v)), ft.emplace_back(sz(v));
    }
    int ind(int x, int y) {
        return (int)(lower_bound(all(ys[x]), y) - ys[x].begin());
    }
    void update(int x, int y, ll dif) {
        for (; x < sz(ys); x |= x + 1)
            ft[x].update(ind(x, y), dif);
    }
    ll query(int x, int y) {
        ll sum = 0;
        for (; x; x &= x - 1)
            sum += ft[x-1].query(ind(x-1, y));
        return sum;
    }
};
```

RMQ.h

Description: Range Minimum Queries on an array. Returns $\min(V[a], V[a+1], \dots, V[b-1])$ in constant time. query(a, b) uses nonempty $[a, b)$: a included, b excluded.

Usage: RMQ rmq(values);

rmq.query(inclusive, exclusive);

Time: $\mathcal{O}(|V| \log |V| + Q)$

d41d8c, 16 lines

```
template<class T>
struct RMQ {
    vector<vector<T>>> jmp;
```

```

RMQ(const vector<T>& V) : jmp(1, V) {
    for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
        jmp.emplace_back(sz(V) - pw * 2 + 1);
        rep(j, 0, sz(jmp[k]))
            jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
    }
}
T query(int a, int b) {
    assert(a < b); // or return inf if a == b
    int dep = 31 - __builtin_clz(b - a);
    return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
}
};

```

StaticRangeQuery.h

Description: Static range queries for any associative operation. Change T , unit and f . Default is range sum. $query(l, r)$ uses nonempty $[l, r)$: l included, r excluded.

Time: $\mathcal{O}(N \log N)$ build, $\mathcal{O}(1)$ query

d41d8c, 36 lines

```

struct RangeQuery {
    typedef int T;
    static constexpr T unit = 0;
    T f(T a, T b) { return a + b; }
    vector<vector<T>> stor;
    vector<T> a;
    int n = 0;
    RangeQuery() {}
    RangeQuery(const vector<T>& v) { build(v); }
    void fill(int l, int r, int ind) {
        if (ind < 0) return;
        int m = (l + r) / 2;
        T prod = unit;
        rep(i, m, r) stor[i][ind] = prod = f(prod, a[i]);
        prod = unit;
        for (int i = m; i-- > l;)
            stor[i][ind] = prod = f(a[i], prod);
        fill(l, m, ind - 1);
        fill(m, r, ind - 1);
    }
    void build(const vector<T>& v) {
        int lg = 0, m = max(sz(v), 1);
        while ((1 << lg) < m) lg++;
        a.assign(1 << lg, unit);
        rep(i, 0, sz(v)) a[i] = v[i];
        n = 1 << lg;
        stor.assign(n, vector<T>(max(lg, 1)));
        fill(0, n, lg - 1);
    }
    T query(int l, int r) { // [l, r)
        assert(l < r);
        if (l == -r) return a[l];
        int t = 31 - __builtin_clz(l ^ r);
        return f(stor[l][t], stor[r][t]);
    }
};

```

MonotonicMap.h

Description: Prefix/suffix min/max queries with insertions. Useful replacement for sparse segtrees. $query(k)$ includes key k : keys $\geq k$ for $dir=less<>$, keys $\leq k$ for $dir=greater<>$. Insert a sentinel so every query has a matching key.

Time: $\mathcal{O}(\log N)$

d41d8c, 18 lines

```

// dir = less<> for suffix queries, greater<> for prefix
// cmp = less_equal<> for min, greater_equal<> for max
template<class dir, class cmp>
struct RangeQuery {
    map<int, ll, dir> data;
    void ins(int k, ll v) {
        if (auto it = data.lower_bound(k);
            it != data.end() && cmp<it->second, v))
            return;
        auto it = data.insert_or_assign(k, v).first;
        while (it != data.begin()
            && cmp<v, prev(it)->second))
            data.erase(prev(it));
    }
    ll query(int k) { // inclusive: careful UB, insert infinity
        return data.lower_bound(k)->second;
    }
};

```

MoQueries.h

Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. mo queries are $[l, r)$: l included, r excluded. $moTree$ queries are vertex pairs (u, v) ; the path includes both endpoint vertices. If values are on tree edges, change step to add/remove the edge (a, c) and remove the initial add call (but keep in).

Time: $\mathcal{O}(N\sqrt{Q})$

d41d8c, 49 lines

```

void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer

vi mo(vector<pii> Q) {
    int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
    vi s(sz(Q)), res = s;
    #define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
    iota(all(s), 0);
    sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
    for (int qi : s) {
        pii q = Q[qi];
        while (L > q.first) add(--L, 0);
        while (R < q.second) add(R++, 1);
        while (L < q.first) del(L++, 0);
        while (R > q.second) del(--R, 1);
        res[qi] = calc();
    }
    return res;
}

vi moTree(vector<array<int, 2>> Q, vector<vi>& ed, int root=0){
    int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
    vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
    add(0, 0), in[0] = 1;
    auto dfs = [&](int x, int p, int dep, auto& f) -> void {
        par[x] = p;
        L[x] = N;
        if (dep) I[x] = N++;
        for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
        if (!dep) I[x] = N++;
        R[x] = N;
    };
    dfs(root, -1, 0, dfs);
    #define K(x) pii(I[x[0]] / blk, I[x[1]] ^ ~(I[x[0]] / blk & 1))
    iota(all(s), 0);
    sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
};

```

```

for (int qi : s) rep(end, 0, 2) {
    int &a = pos[end], b = Q[qi][end], i = 0;
    #define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
        else { add(c, end); in[c] = 1; } a = c; }
    while (!L[b] <= L[a] && R[a] <= R[b])
        I[i++] = b, b = par[b];
    while (a != b) step(par[a]);
    while (i--) step(I[i]);
    if (end) res[qi] = calc();
}
return res;
}

```

Numerical (4)

4.1 Polynomials and recurrences

QuadRoots.h

Description: Real roots of $ax^2 + bx + c = 0$ with better precision than the naive formula. Assumes $a \neq 0$. Returns the number of roots (0, 1, or 2); writes them to out (unordered; repeated root when one).

Time: $\mathcal{O}(1)$

d41d8c, 9 lines

```

int quadRoots(double a, double b, double c,
    pair<double, double>& out) {
    assert(a != 0);
    double disc = b*b - 4*a*c;
    if (disc < 0) return 0;
    double sum = (b >= 0) ? -b - sqrt(disc) : -b + sqrt(disc);
    out = {sum/(2*a), sum == 0 ? 0 : (2*c)/sum};
    return 1 + (disc > 0);
}

```

Polynomial.h

d41d8c, 17 lines

```

struct Poly {
    vector<double> a;
    double operator()(double x) const {
        double val = 0;
        for (int i = sz(a); i--;) (val *= x) += a[i];
        return val;
    }
    void diff() {
        rep(i, 1, sz(a)) a[i-1] = i*a[i];
        a.pop_back();
    }
    void divroot(double x0) {
        double b = a.back(), c; a.back() = 0;
        for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
        a.pop_back();
    }
};

```

PolyRoots.h

Description: Finds the real roots to a polynomial. $xmin/xmax$ should enclose all roots. Search sentinels are $xmin-1$ and $xmax+1$; returned roots are not clipped to $[xmin, xmax]$. A linear polynomial ignores bounds.

Usage: `polyRoots({{2,-3,1}}, -1e9, 1e9)` // solve $x^2-3x+2 = 0$

Time: $\mathcal{O}(n^2 \log(1/\epsilon))$

"Polynomial.h"

d41d8c, 23 lines

```

vector<double> polyRoots(Poly p, double xmin, double xmax) {
    if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
};

```

```

vector<double> ret;
Poly der = p;
der.diff();
auto dr = polyRoots(der, xmin, xmax);
dr.push_back(xmin-1);
dr.push_back(xmax+1);
sort(all(dr));
rep(i,0,sz(dr)-1) {
    double l = dr[i], h = dr[i+1];
    bool sign = p(l) > 0;
    if (sign ^ (p(h) > 0)) {
        rep(it,0,60) { // while (h - l > 1e-8)
            double m = (l + h) / 2, f = p(m);
            if ((f <= 0) ^ sign) l = m;
            else h = m;
        }
        ret.push_back((l + h) / 2);
    }
}
return ret;
}

```

PolyInterpolate.h

Description: Given n points $(x[i], y[i])$, computes an $n-1$ -degree polynomial p that passes through them: $p(x) = a[0]x^0 + \dots + a[n-1]x^{n-1}$. For numerical precision, pick $x[k] = c \cos(k/(n-1)\pi)$, $k = 0 \dots n-1$.
Time: $\mathcal{O}(n^2)$

d41d8c, 13 lines

```

typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
    vd res(n), temp(n);
    rep(k,0,n-1) rep(i,k+1,n)
        y[i] = (y[i] - y[k]) / (x[i] - x[k]);
    double last = 0; temp[0] = 1;
    rep(k,0,n) rep(i,0,n) {
        res[i] += y[k] * temp[i];
        swap(last, temp[i]);
        temp[i] -= last * x[k];
    }
    return res;
}

```

Lagrange.h

Description: Evaluates $P(x)$ modulo a prime from samples $y[i] = P(i)$, $i = 0, \dots, n-1$, where $\deg P < n \leq \text{mod}$ and $n > 0$. Requires $0 \leq y[i] < \text{mod}$ and products to fit in ll. Handles any signed x . Template modulus defaults to $10^9 + 7$. Useful for polynomial counting and sums of powers: $\sum_{i=1}^x i^k$ has degree $k+1$ if $k+1 < \text{mod}$, so use $k+2$ samples. Prove a degree bound first.

Usage: lagrange({0, 1, 4}, 10) // 100

lagrange<998244353>(y, x)

Memory: $\mathcal{O}(n)$

Time: $\mathcal{O}(n + \log \text{mod})$

"./number-theory/ModPow.h"

d41d8c, 26 lines

```

template<ll mod = 1000000007>
ll lagrange(const vector<ll>& y, ll x) {
    int n = sz(y);
    assert(0 < n && n <= mod);
    x %= mod;
    if (x < 0) x += mod;
    if (x < n) return y[x];
    vector<ll> pre(n+1, 1), suf(pre), ifac(n);
    ll fac = 1, ans = 0;
    rep(i,0,n) {

```

```

        pre[i+1] = pre[i] * (x-i) % mod;
        if (i) fac = fac * i % mod;
    }
    ifac[n-1] = modpow<mod>(fac, mod-2);
    for (int i = n-1; i > 0; --i)
        ifac[i-1] = ifac[i] * i % mod;
    for (int i = n-1; i >= 0; --i)
        suf[i] = suf[i+1] * (x-i) % mod;
    rep(i,0,n) {
        ll t = y[i] * pre[i] % mod * suf[i+1] % mod;
        t = t * ifac[i] % mod * ifac[n-1-i] % mod;
        if ((n-1-i) & 1) t = (mod-t) % mod;
        ans = (ans+t) % mod;
    }
    return ans;
}

```

BerlekampMassey.h

Description: Recovers any n -order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$. Template modulus defaults to $10^9 + 7$.

Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}

Time: $\mathcal{O}(N^2)$

"./number-theory/ModPow.h"

d41d8c, 21 lines

```

template<ll mod = 1000000007>
vector<ll> berlekampMassey(vector<ll> s) {
    int n = sz(s), L = 0, m = 0;
    vector<ll> C(n), B(n), T;
    C[0] = B[0] = 1;

    ll b = 1;
    rep(i,0,n) { ++m;
        ll d = s[i] % mod;
        rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
        if (!d) continue;
        T = C; ll coef = d * modpow<mod>(b, mod-2) % mod;
        rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
        if (2 * L > i) continue;
        L = i + 1 - L; B = T; b = d; m = 0;
    }

    C.resize(L + 1); C.erase(C.begin());
    for (ll& x : C) x = (mod - x) % mod;
    return C;
}

```

LinearRecurrence.h

Description: Generates the k 'th term of an n -order linear recurrence $S[i] = \sum_j S[i-j-1] \text{tr}[j]$, given $S[0] \dots \geq n-1$ and $\text{tr}[0] \dots n-1$. Faster than matrix multiplication. Useful together with Berlekamp-Massey.

Usage: linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number

Time: $\mathcal{O}(n^2 \log k)$

d41d8c, 26 lines

```

typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
    int n = sz(tr);

    auto combine = [&](Poly a, Poly b) {
        Poly res(n * 2 + 1);
        rep(i,0,n+1) rep(j,0,n+1)
            res[i + j] = (res[i + j] + a[i] * b[j]) % mod;

```

```

        for (int i = 2 * n; i > n; --i) rep(j,0,n)
            res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
        res.resize(n + 1);
        return res;
    };

    Poly pol(n + 1), e(pol);
    pol[0] = e[1] = 1;

    for (++k; k; k /= 2) {
        if (k % 2) pol = combine(pol, e);
        e = combine(e, e);
    }

    ll res = 0;
    rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
    return res;
}

```

4.2 Optimization

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is eps . Works equally well for maximization with a small change in the code. See TernarySearch.h in the Various chapter for a discrete version. Both a and b are included in the search domain; for $a < b$, f is sampled only at interior points.

Usage: double func(double x) { return 4*x+.3*x*x; }

double xmin = gss(-1000,1000,func);

Time: $\mathcal{O}(\log((b-a)/\epsilon))$

d41d8c, 14 lines

```

double gss(double a, double b, double (*f)(double)) {
    double r = (sqrt(5)-1)/2, eps = 1e-7;
    double x1 = b - r*(b-a), x2 = a + r*(b-a);
    double f1 = f(x1), f2 = f(x2);
    while (b-a > eps)
        if (f1 < f2) { //change to > to find maximum
            b = x2; x2 = x1; f2 = f1;
            x1 = b - r*(b-a); f1 = f(x1);
        } else {
            a = x1; x1 = x2; f1 = f2;
            x2 = a + r*(b-a); f2 = f(x2);
        }
    return a;
}

```

HillClimbing.h

Description: Poor man's optimization for unimodal functions.

d41d8c, 14 lines

typedef array<double, 2> P;

```

template<class F> pair<double, P> hillClimb(P start, F f) {
    pair<double, P> cur(f(start), start);
    for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
        rep(j,0,100) rep(dx,-1,2) rep(dy,-1,2) {
            P p = cur.second;
            p[0] += dx*jmp;
            p[1] += dy*jmp;
            cur = min(cur, make_pair(f(p), p));
        }
    }
    return cur;
}

```

Integrate.h

Description: Simple integration of a function over an interval using Simpson's rule. The error should be proportional to h^4 , although in practice you will want to verify that the result is stable to desired precision when epsilon changes. `quad(a,b,f)` integrates over $[a,b]$; `f` must be defined at both endpoints (both sampled).

d41d8c, 7 lines

```
template<class F>
double quad(double a, double b, F f, const int n = 1000) {
    double h = (b - a) / 2 / n, v = f(a) + f(b);
    rep(i,1,n*2)
        v += f(a + i*h) * (i&1 ? 4 : 2);
    return v * h / 3;
}
```

IntegrateAdaptive.h

Description: Fast integration using an adaptive Simpson's rule. `quad(a,b,f)` integrates over $[a,b]$; `f` must be defined at both endpoints (both sampled).

Usage: `double sphereVolume = quad(-1, 1, [](double x) { return quad(-1, 1, [&](double y) { return quad(-1, 1, [&](double z) { return x*x + y*y + z*z < 1; }}});});`

d41d8c, 15 lines

```
typedef double d;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6

template <class F>
d rec(F& f, d a, d b, d eps, d S) {
    d c = (a + b) / 2;
    d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
    if (abs(T - S) <= 15 * eps || b - a < 1e-10)
        return T + (T - S) / 15;
    return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2, S2);
}

template<class F>
d quad(d a, d b, F f, d eps = 1e-8) {
    return rec(f, a, b, eps, S(a, b));
}
```

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \leq b$, $x \geq 0$. Returns `-inf` if there is no solution, `inf` if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.

Usage: `vvd A = {{1,-1}, {-1,1}, {-1,-2}};`

`vd b = {1,1,-4}, c = {-1,-1}, x;`

`T val = LPSolver(A, b, c).solve(x);`

Time: $\mathcal{O}(NM * \#pivots)$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}(2^n)$ in the general case.

d41d8c, 68 lines

```
typedef double T; // long double, Rational, double + mod<P>...
typedef vector<T> vd;
typedef vector<vd> vvd;
```

```
const T eps = 1e-8, inf = 1/.0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s])) s=j
```

```
struct LPSolver {
    int m, n;
```

```
vi N, B;
vvd D;

LPSolver(const vvd& A, const vd& b, const vd& c) :
    m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
    rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
    rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] = b[i]; }
    rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
    N[n] = -1; D[m+1][n] = 1;
}
```

```
void pivot(int r, int s) {
    T *a = D[r].data(), inv = 1 / a[s];
    rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
        T *b = D[i].data(), inv2 = b[s] * inv;
        rep(j,0,n+2) b[j] -= a[j] * inv2;
        b[s] = a[s] * inv2;
    }
    rep(j,0,n+2) if (j != s) D[r][j] *= inv;
    rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
}
```

```
bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
        int s = -1;
        rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
        if (D[x][s] >= -eps) return true;
        int r = -1;
        rep(i,0,m) {
            if (D[i][s] <= eps) continue;
            if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
                < MP(D[r][n+1] / D[r][s], B[r])) r = i;
        }
        if (r == -1) return false;
        pivot(r, s);
    }
}
```

```
T solve(vd &x) {
    int r = 0;
    rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
    if (D[r][n+1] < -eps) {
        pivot(r, n);
        if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
        rep(i,0,m) if (B[i] == -1) {
            int s = 0;
            rep(j,1,n+1) ltj(D[i]);
            pivot(i, s);
        }
    }
    bool ok = simplex(1); x = vd(n);
    rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
    return ok ? D[m][n+1] : inf;
}
```

};

4.3 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix. **Time:** $\mathcal{O}(N^3)$

d41d8c, 15 lines

```
double det(vector<vector<double>>& a) {
    int n = sz(a); double res = 1;
    rep(i,0,n) {
        int b = i;
        rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
        if (i != b) swap(a[i], a[b]), res *= -1;
        res *= a[i][i];
        if (res == 0) return 0;
        rep(j,i+1,n) {
            double v = a[j][i] / a[i][i];
            if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
        }
    }
    return res;
}
```

IntDeterminant.h

Description: Calculates determinant using modular arithmetics. Modular can also be removed to get a pure-integer version.

Time: $\mathcal{O}(N^3)$

d41d8c, 18 lines

```
const ll mod = 12345;
ll det(vector<vector<ll>>& a) {
    int n = sz(a); ll ans = 1;
    rep(i,0,n) {
        rep(j,i+1,n) {
            while (a[j][i] != 0) { // gcd step
                ll t = a[i][i] / a[j][i];
                if (t) rep(k,i,n)
                    a[i][k] = (a[i][k] - a[j][k] * t) % mod;
                swap(a[i], a[j]);
                ans *= -1;
            }
        }
        ans = ans * a[i][i] % mod;
        if (!ans) return 0;
    }
    return (ans + mod) % mod;
}
```

SolveLinear.h

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or `-1` if no solutions. Data in A and b is lost.

Time: $\mathcal{O}(n^2 m)$

d41d8c, 38 lines

```
typedef vector<double> vd;
const double eps = 1e-12;

int solveLinear(vector<vd>& A, vd& b, vd& x) {
    int n = sz(A), m = sz(x), rank = 0, br, bc;
    if (n) assert(sz(A[0]) == m);
    vi col(m); iota(all(col), 0);
```

```
    rep(i,0,n) {
        double v, bv = 0;
        rep(r,i,n) rep(c,i,m)
            if ((v = fabs(A[r][c])) > bv)
                br = r, bc = c, bv = v;
        if (bv <= eps) {
            rep(j,i,n) if (fabs(b[j]) > eps) return -1;
            break;
        }
        swap(A[i], A[br]);
```

```

    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) swap(A[j][i], A[j][bc]);
    bv = 1/A[i][i];
    rep(j,i+1,n) {
        double fac = A[j][i] * bv;
        b[j] -= fac * b[i];
        rep(k,i+1,m) A[j][k] -= fac*A[i][k];
    }
    rank++;
}

x.assign(m, 0);
for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    rep(j,0,i) b[j] -= A[j][i] * b[i];
}
return rank; // (multiple solutions if rank < m)
}

```

SolveLinear2.h

Description: To get all uniquely determined values of x back from SolveLinear, make the following changes:

```

"SolveLinear.h"
rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
// ... then at the end:
x.assign(m, undefined);
rep(i,0,rank) {
    rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
    x[col[i]] = b[i] / A[i][i];
fail:; }

```

SolveLinearBinary.h

Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b .

Time: $\mathcal{O}(n^2m)$

```

typedef bitset<1000> bs;

int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
    int n = sz(A), rank = 0, br;
    assert(m <= sz(x));
    vi col(m); iota(all(col), 0);
    rep(i,0,n) {
        for (br=i; br<n; ++br) if (A[br].any()) break;
        if (br == n) {
            rep(j,i,n) if(b[j]) return -1;
            break;
        }
        int bc = (int)A[br]._Find_next(i-1);
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        rep(j,0,n) if (A[j][i] != A[j][bc]) {
            A[j].flip(i); A[j].flip(bc);
        }
        rep(j,i+1,n) if (A[j][i]) {
            b[j] ^= b[i];
            A[j] ^= A[i];
        }
        rank++;
    }
}

```

```

x = bs();
for (int i = rank; i--;) {
    if (!b[i]) continue;
    x[col[i]] = 1;
    rep(j,0,i) b[j] ^= A[j][i];
}
return rank; // (multiple solutions if rank < m)
}

```

RREF.h

Description: Partial-pivot reduced row echelon form of a (possibly rectangular) matrix. Returns (rank, det). det is meaningful for square matrices. Use this when you need the RREF itself (rank, nullspace, row space). For $Ax = b$ or inversion, see SolveLinear / MatrixInverse.

Time: $\mathcal{O}(n^2m)$

```

d41d8c, 26 lines

typedef vector<double> vd;
typedef vector<vd> vvd;
const double RREF_EPS = 1e-10;

pair<int, double> rref(vvd& a) {
    int n = sz(a), m = sz(a[0]), r = 0;
    double det = 1;
    for (int c = 0; c < m && r < n; c++) {
        int j = r;
        rep(i,r+1,n)
            if (fabs(a[i][c]) > fabs(a[j][c])) j = i;
        if (fabs(a[j][c]) < RREF_EPS) continue;
        swap(a[j], a[r]);
        if (j != r) det *= -1;
        det *= a[r][c];
        double s = 1 / a[r][c];
        rep(k,0,m) a[r][k] *= s;
        rep(i,0,n) if (i != r) {
            double t = a[i][c];
            rep(k,0,m) a[i][k] -= t * a[r][k];
        }
        r++;
    }
    if (n == m && r < n) det = 0;
    return {r, det};
}

```

XORBasis.h

Description: Incremental XOR basis (span of vectors over $\mathbb{F}_2$). Insert online; x is in the span iff it reduces to 0. Span size is $2^{|basis|}$. XorBasis is for unsigned; BigBasis for bitblocks. For solving $Ax = b$, see SolveLinearBinary.

Time: $\mathcal{O}(B \cdot |basis|)$ per insert

```

d41d8c, 41 lines

struct XorBasis {
    vector<unsigned> basis;
    void add(unsigned x) {
        rep(i,0,sz(basis)) x = min(x, x ^ basis[i]);
        if (x) basis.push_back(x);
    }
    bool inSpan(unsigned x) {
        rep(i,0,sz(basis)) x = min(x, x ^ basis[i]);
        return x == 0;
    }
};

bool nonzero(const vector<uint64_t>& x) {

```

```

    for (auto a : x) if (a) return true;
    return false;
}

struct BigBasis {
    vector<vector<uint64_t>> basis;
    vector<uint64_t> reduce(vector<uint64_t> x) {
        rep(i,0,sz(basis)) {
            int state = 0;
            rep(j,0,sz(x)) {
                uint64_t cur = basis[i][j] ^ x[j];
                if (state == 0 && cur < x[j]) state = -1;
                if (state == 0 && cur > x[j]) state = 1;
                if (state <= 0) x[j] = cur;
            }
        }
        return x;
    }
    void add(vector<uint64_t> x) {
        x = reduce(x);
        if (nonzero(x)) basis.push_back(x);
    }
    bool equal(const BigBasis& o) {
        if (sz(o.basis) != sz(basis)) return false;
        for (auto& v : o.basis)
            if (nonzero(reduce(v))) return false;
        return true;
    }
};

```

MatrixInverse.h

Description: Invert matrix A . Returns rank; result is stored in A unless singular ($\text{rank} < n$). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of $A \pmod{p}$, and k is doubled in each step.

Time: $\mathcal{O}(n^3)$

```

d41d8c, 35 lines

int matInv(vector<vector<double>>& A) {
    int n = sz(A); vi col(n);
    vector<vector<double>> tmp(n, vector<double>(n));
    rep(i,0,n) tmp[i][i] = 1, col[i] = i;

    rep(i,0,n) {
        int r = i, c = i;
        rep(j,i,n) rep(k,i,n)
            if (fabs(A[j][k]) > fabs(A[r][c]))
                r = j, c = k;
        if (fabs(A[r][c]) < 1e-12) return i;
        A[i].swap(A[r]); tmp[i].swap(tmp[r]);
        rep(j,0,n)
            swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
        swap(col[i], col[c]);
        double v = A[i][i];
        rep(j,i+1,n) {
            double f = A[j][i] / v;
            A[j][i] = 0;
            rep(k,i+1,n) A[j][k] -= f*A[i][k];
            rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
        }
        rep(j,i+1,n) A[i][j] /= v;
        rep(j,0,n) tmp[i][j] /= v;
        A[i][i] = 1;
    }

    for (int i = n-1; i > 0; --i) rep(j,0,i) {
        double v = A[j][i];

```

```

    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
}

rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
return n;
}

```

MatrixInverse-mod.h

Description: Invert matrix A modulo a prime. Template modulus defaults to $10^9 + 7$. Returns rank; result is stored in A unless singular (rank $< n$). For prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of $A \pmod p$, and k is doubled in each step. **Time:** $\mathcal{O}(n^3)$

"/number-theory/ModPow.h" d41d8c, 38 lines

```

template<ll mod = 1000000007>
int matInv(vector<vector<ll>>& A) {
    int n = sz(A); vi col(n);
    vector<vector<ll>> tmp(n, vector<ll>(n));
    rep(i,0,n) tmp[i][i] = 1, col[i] = i;

    rep(i,0,n) {
        int r = i, c = i;
        rep(j,i,n) rep(k,i,n) if (A[j][k]) {
            r = j; c = k; goto found;
        }
        return i;
    found:
        A[i].swap(A[r]); tmp[i].swap(tmp[r]);
        rep(j,0,n)
            swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
        swap(col[i], col[c]);
        ll v = modpow<mod>(A[i][i], mod - 2);
        rep(j,i+1,n) {
            ll f = A[j][i] * v % mod;
            A[j][i] = 0;
            rep(k,i+1,n) A[j][k] = (A[j][k] - f*A[i][k]) % mod;
            rep(k,0,n) tmp[j][k] = (tmp[j][k] - f*tmp[i][k]) % mod;
        }
        rep(j,i+1,n) A[i][j] = A[i][j] * v % mod;
        rep(j,0,n) tmp[i][j] = tmp[i][j] * v % mod;
        A[i][i] = 1;
    }

    for (int i = n-1; i > 0; --i) rep(j,0,i) {
        ll v = A[j][i];
        rep(k,0,n) tmp[j][k] = (tmp[j][k] - v*tmp[i][k]) % mod;
    }

    rep(i,0,n) rep(j,0,n)
        A[col[i]][col[j]] = tmp[i][j] % mod + (tmp[i][j] < 0)*mod;
    return n;
}

```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i, \quad 1 \leq i \leq n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, \dots, -1, 1\}, \{0, c_1, c_2, \dots, c_n\}, \{b_1, b_2, \dots, b_n, 0\}, \{a_0, d_1, d_2, \dots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.

If $|d_i| > |p_i| + |q_{i-1}|$ for all i , or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.

Time: $\mathcal{O}(N)$

d41d8c, 26 lines

```

typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>& super,
    const vector<T>& sub, vector<T> b) {
    int n = sz(b); vi tr(n);
    rep(i,0,n-1) {
        if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
            b[i+1] -= b[i] * diag[i+1] / super[i];
            if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
            diag[i+1] = sub[i]; tr[i+1] = 1;
        } else {
            diag[i+1] -= super[i]*sub[i]/diag[i];
            b[i+1] -= b[i]*sub[i]/diag[i];
        }
    }
    for (int i = n; i--;) {
        if (tr[i]) {
            swap(b[i], b[i-1]);
            diag[i-1] = diag[i];
            b[i] /= super[i-1];
        } else {
            b[i] /= diag[i];
            if (i) b[i-1] -= b[i]*super[i-1];
        }
    }
    return b;
}

```

4.4 Convolutions

FastFourierTransform.h

Description: `fft(a)` computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. Useful for convolution: `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n , reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16} ; higher for random inputs). Otherwise, use NTT/FFTMod.

Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

d41d8c, 35 lines

```

typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1); // ^ 10% faster if double)
    for (static int k = 2; k < n; k *= 2) {
        R.resize(n); rt.resize(n);
        auto x = polar(1.0/L, acos(-1.0/L) / k);
        rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
}

```

```

for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
        C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
        a[i + j + k] = a[i + j] - z;
        a[i + j] += z;
    }
}
vd conv(const vd& a, const vd& b) {
    if (a.empty() || b.empty()) return {};
    vd res(sz(a) + sz(b) - 1);
    int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
    vector<C> in(n), out(n);
    copy(all(a), begin(in));
    rep(i,0,sz(b)) in[i].imag(b[i]);
    fft(in);
    for (C& x : in) x *= x;
    rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
    return res;
}

```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$. 0 included, mod excluded.

Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

"FastFourierTransform.h"

d41d8c, 22 lines

```

typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    vl res(sz(a) + sz(b) - 1);
    int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
    vector<C> L(n), R(n), outs(n), outl(n);
    rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
    rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
    fft(L), fft(R);
    rep(i,0,n) {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
    }
    fft(outl), fft(outs);
    rep(i,0,sz(res)) {
        ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
        ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
        res[i] = ((av * M * cut + bv) % M * cut + cv) % M;
    }
    return res;
}

```

NumberTheoreticTransform.h

Description: `ntt(a)` computes $\hat{f}(k) = \sum_x a[x]g^{xk}$ for all k , where $g = \text{root}^{(\text{mod}-1)/N}$. N must be a power of 2 dividing $\text{mod} - 1$. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a . For arbitrary modulo, see FFT-Mod. `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n , reverse(start+1, end), NTT back. Inputs must be in $[0, \text{mod})$. 0 included, mod excluded.

Time: $\mathcal{O}(N \log N)$

"/number-theory/ModPow.h"

d41d8c, 35 lines

```

const ll mod = (119 << 23) + 1, root = 62; // = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.

```

```
typedef vector<ll> vl;
void ntt(vl &a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vl rt(2, 1);
    for (static int k = 2, s = 2; k < n; k *= 2, s++) {
        rt.resize(n);
        ll z[] = {1, modpow<mod>(root, mod >> s)};
        rep(i, k, 2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
    }
    vi rev(n);
    rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j, 0, k) {
            ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
            a[i + j + k] = ai - z + (z > ai ? mod : 0);
            ai += (ai + z >= mod ? z - mod : z);
        }
}
vl conv(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, n = 1;
    while (n < s) n *= 2;
    int inv = modpow<mod>(n, mod - 2);
    vl L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    rep(i, 0, n)
        out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
    ntt(out);
    return {out.begin(), out.begin() + s};
}
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.
Time: $\mathcal{O}(N \log N)$

```
void FST(vi& a, bool inv) {
    for (int n = sz(a), step = 1; step < n; step *= 2) {
        for (int i = 0; i < n; i += 2 * step) rep(j, i, i+step) {
            int &u = a[j], &v = a[j + step]; tie(u, v) =
                inv ? pii(v - u, u) : pii(v, u + v); // AND
            inv ? pii(v, u - v) : pii(u + v, u); // OR
            pii(u + v, u - v); // XOR
        }
    }
    if (inv) for (int& x : a) x /= sz(a); // XOR only
}
vi conv(vi a, vi b) {
    FST(a, 0); FST(b, 0);
    rep(i, 0, sz(a)) a[i] *= b[i];
    FST(a, 1); return a;
}
```

Number theory (5)

5.1 Modular arithmetic

ModularArithmetic.h

Description: Operators for modular arithmetic. You need to set mod to some number first and then you can use the structure.

```
"euclid.h"
const ll mod = 17; // change to something else
struct Mod {
    ll x;
    Mod(ll y) : Mod(y%mod+mod,0){}
    Mod(ll y,int) : x(y%mod?y:mod){}
    Mod operator+(Mod b) { return {x + b.x,0}; }
    Mod operator-(Mod b) { return {x - b.x + mod,0}; }
    Mod operator*(Mod b) { return {x * b.x % mod,0}; }
    Mod operator/(Mod b) { return *this * invert(b); }
    Mod invert(Mod a) {
        ll x, y, g = euclid(a.x, mod, x, y);
        assert(g == 1); return x;
    }
    Mod operator^(ll e) {
        if (!e) return 1;
        Mod r = *this ^ (e / 2); r = r * r;
        return e&1 ? *this * r : r;
    }
};
```

ModInverse.h

Description: Pre-computation of modular inverses. Assumes $LIM \leq \text{mod}$ and that mod is a prime. Fills inv for $[1, LIM]$: 1 included, LIM excluded.

```
const ll mod = 1000000007, LIM = 200000;
ll* inv = new ll[LIM] - 1; inv[1] = 1;
rep(i, 2, LIM) inv[i] = mod - (mod / i) * inv[mod % i] % mod;
```

ModPow.h

Description: Computes $b^e \bmod m$ for $0 \leq b < m$ and $e \geq 0$. Requires $m > 1$ and products to fit in ll. Compile-time modulus defaults to 10^9+7 .
Usage: modpow(b, e) // default modulus
 modpow<998244353>(b, e) // compile-time modulus
 modpow(b, e, p) // runtime modulus
Time: $\mathcal{O}(\log e)$

```
ll modpow(ll b, ll e, ll mod) {
    ll ans = 1;
    for (; e; b = b * b % mod, e /= 2)
        if (e & 1) ans = ans * b % mod;
    return ans;
}
```

```
template<ll mod = 1000000007>
ll modpow(ll b, ll e) { return modpow(b, e, mod); }
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a .
Time: $\mathcal{O}(\sqrt{m})$

```
ll modLog(ll a, ll b, ll m) {
    ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<ll, ll> A;
    while (j <= n && (e = f * e % m) != b % m)
        A[e * b % m] = j++;
    if (e == b % m) return j;
    if (__gcd(m, e) == __gcd(m, b))
        rep(i, 2, n+2) if (A.count(e = e * f % m))
            return n * i - A[e];
}
```

```
return -1;
}
```

ModSum.h

Description: Sums of mod'ed arithmetic progressions.
 $\text{modsum}(to, c, k, m) = \sum_{i=0}^{to-1} (ki + c) \% m$. divsum is similar but for floored division. Both sum over $[0, to)$: 0 included, to excluded.
Time: $\log(m)$, with a large constant.

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }

ull divsum(ull to, ull c, ull k, ull m) {
    ull res = k / m * sumsq(to) + c / m * to;
    k %= m; c %= m;
    if (!k) return res;
    ull to2 = (to * k + c) / m;
    return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}
```

```
ll modsum(ull to, ll c, ll k, ll m) {
    c = ((c % m) + m) % m;
    k = ((k % m) + m) % m;
    return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.
Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
    ll ret = a * b - M * ull(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {
    ull ans = 1;
    for (; e; b = modmul(b, b, mod), e /= 2)
        if (e & 1) ans = modmul(ans, b, mod);
    return ans;
}
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).
Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

```
"ModPow.h"
ll sqrt(ll a, ll p) {
    a %= p; if (a < 0) a += p;
    if (a == 0) return 0;
    assert(modpow(a, (p-1)/2, p) == 1); // else no solution
    if (p % 4 == 3) return modpow(a, (p+1)/4, p);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
    ll s = p - 1, n = 2;
    int r = 0, m;
    while (s % 2 == 0)
        ++r, s /= 2;
    while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
    ll x = modpow(a, (s + 1) / 2, p);
    ll b = modpow(a, s, p), g = modpow(n, s, p);
    for (; r = m) {
        ll t = b;
```

```

    for (m = 0; m < r && t != 1; ++m)
        t = t * t % p;
    if (m == 0) return x;
    ll gs = modpow(g, 1LL << (r - m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
}
}

```

5.2 Primality

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM. Primes lie in $[2, \text{LIM})$: 2 included, LIM excluded.

Time: $\text{LIM} = 1\text{e}9 \approx 1.5\text{s}$

d41d8c, 20 lines

```

const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
    const int S = (int)round(sqrt(LIM)), R = LIM / 2;
    vi pr = {2}, sieve(S+1); pr.reserve(int(LIM/log(LIM)*1.1));
    vector<pii> cp;
    for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
        cp.push_back({i, i * i / 2});
        for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
    }
    for (int L = 1; L <= R; L += S) {
        array<bool, S> block{};
        for (auto &[p, idx] : cp)
            for (int i = idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
        rep(i, 0, min(S, R - L))
            if (!block[i]) pr.push_back((L + i) * 2 + 1);
    }
    for (int i : pr) isPrime[i] = 1;
    return pr;
}

```

LinearSieve.h

Description: Linear sieve. $\text{lp}[i]$ is the least prime factor of i ; pr lists primes $\leq N$. Factor i by repeatedly dividing out $\text{lp}[i]$. Sieves $[2, \text{SIEVE_N}]$, both endpoints included; lp has indices $[0, \text{SIEVE_N}]$.

Time: $\mathcal{O}(N)$

d41d8c, 13 lines

```

const int SIEVE_N = 10000000;
vi lp, pr;
void linearSieve() {
    lp.assign(SIEVE_N + 1, 0);
    pr.clear();
    for (int i = 2; i <= SIEVE_N; ++i) {
        if (lp[i] == 0) { lp[i] = i; pr.push_back(i); }
        for (int j = 0; pr[j] <= SIEVE_N / i; ++j) {
            lp[i * pr[j]] = pr[j];
            if (pr[j] == lp[i]) break;
        }
    }
}

```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.

Time: 7 times the complexity of $a^b \bmod c$.

"ModMuLLL.h"

d41d8c, 12 lines

```

bool isPrime(ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
    s = __builtin_ctzll(n-1), d = n >> s;
    for (ull a : A) { // ^ count trailing zeroes
        ull p = modpow(a%n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
            p = modmul(p, p, n);
        if (p != n-1 && i != s) return 0;
    }
    return 1;
}

```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. $2299 \rightarrow \{11, 19, 11\}$).

Time: $\mathcal{O}(n^{1/4})$, less for numbers with small factors.

"ModMuLLL.h", "MillerRabin.h"

d41d8c, 18 lines

```

ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return modmul(x, x, n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}
vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}

```

5.3 Divisibility

euclid.h

Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in `__gcd` instead. If a and b are coprime, then x is the inverse of $a \pmod{b}$.

d41d8c, 5 lines

```

ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a/b * x, d;
}

```

CRT.h

Description: Chinese Remainder Theorem.

$\text{crt}(a, m, b, n)$ computes x such that $x \equiv a \pmod{m}$, $x \equiv b \pmod{n}$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.

Time: $\log(n)$

"euclid.h"

d41d8c, 7 lines

```

ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m*n/g : x;
}

```

```

}

```

5.3.1 Bézout's identity

For $a \neq 0$, $b \neq 0$, then $d = \gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a, b)}, y - \frac{ka}{\gcd(a, b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: Euler's ϕ function is the number of integers in $[1, n]$ coprime to n . $\phi(1) = 1$.

p prime $\Rightarrow \phi(p^k) = (p-1)p^{k-1}$. m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$.

If $n = \prod_i p_i^{k_i}$ then $\phi(n) = \prod_i (p_i - 1)p_i^{k_i-1} = n \prod_{p|n} (1 - 1/p)$.

$\sum_{d|n} \phi(d) = n$. For $n > 1$, $\sum_{k \perp n} k = n\phi(n)/2$.

Euler's thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod{n}$.

Fermat's little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod{p} \forall a$.

d41d8c, 8 lines

```

const int LIM = 5000000;
int phi[LIM];

```

```

void calculatePhi() {
    rep(i, 0, LIM) phi[i] = i&1 ? i : i/2;
    for (int i = 3; i < LIM; i += 2) if (phi[i] == i)
        for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}

```

5.4 Fractions

ContinuedFractions.h

Description: Given N and a real number $x \geq 0$, finds the closest rational approximation p/q with $p, q \leq N$. It will obey $|p/q - x| \leq 1/qN$.

For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. (p_k/q_k alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞ ; if x is the root of a degree 2 polynomial the a 's eventually become cyclic.

Time: $\mathcal{O}(\log N)$

d41d8c, 21 lines

```

typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximate(d x, ll N) {
    ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
    for (;;) {
        ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf),
            a = (ll)floor(y), b = min(a, lim),
            NP = b*P + LP, NQ = b*Q + LQ;
        if (a > b) {
            // If b > a/2, we have a semi-convergent that gives us a
            // better approximation; if b = a/2, we *may* have one.
            // Return {P, Q} here for a more canonical approximation.
            return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
                make_pair(NP, NQ) : make_pair(P, Q);
        }
        if (abs(y = 1/(y - (d)a)) > 3*N) {
            return {NP, NQ};
        }
        LP = P; P = NP;
        LQ = Q; Q = NQ;
    }
}

```

FracBinarySearch.h

Description: Given f and N , finds the smallest fraction $p/q \in [0, 1]$ such that $f(p/q)$ is true, and $p, q \leq N$. Both 0 and 1 are included in the search domain. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.

Usage: `fracBS([])(Frac f) { return f.p>=3*f.q; }, 10); // {1,3}`

Time: $\mathcal{O}(\log(N))$

d41d8c, 25 lines

```
struct Frac { ll p, q; };

template<class F>
Frac fracBS(F f, ll N) {
    bool dir = 1, A = 1, B = 1;
    Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
    if (f(lo)) return lo;
    assert(f(hi));
    while (A || B) {
        ll adv = 0, step = 1; // move hi if dir, else lo
        for (int si = 0; step; (step *= 2) >>= si) {
            adv += step;
            Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
            if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
                adv -= step; si = 2;
            }
        }
        hi.p += lo.p * adv;
        hi.q += lo.q * adv;
        dir = !dir;
        swap(lo, hi);
        A = B; B = !adv;
    }
    return dir ? hi : lo;
}
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Common moduli: $10^9 + 7$, $10^9 + 9$, and $998244353 = 119 \cdot 2^{23} + 1$ (NTT). Switch if one may collide.

Primitive roots exist modulo n iff $n = 2, 4, p^k$ or $2p^k$ for odd prime p , and there are $\phi(\phi(n))$ of them. For $p = 2$, $a > 2$, $\mathbb{Z}_{2^a}^\times \cong \mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$. If g is a primitive root mod odd prime p , then g is one mod all p^k iff $g^{p-1} \not\equiv 1 \pmod{p^2}$; otherwise $g + p$ works. If g is a primitive root mod p^k , the odd one of g and $g + p^k$ is a primitive root mod $2p^k$.

PrimitiveRoot.h

Description: Multiplicative order of a modulo prime p (HAC 4.79: smallest $k > 0$ with $a^k \equiv 1 \pmod{p}$). Assumes $p \nmid a$. `primitiveRoot(p)` is the smallest generator of $\mathbb{Z}_p^\times$ (ACL). If calling many times for the same p , precompute `pfactors(p-1)`.

Time: factoring $p - 1$, then $\mathcal{O}(\log^2 p)$ per candidate

"Factor.h"

d41d8c, 27 lines

```
vector<ull> pfactors(ull n) {
    auto f = factor(n);
    sort(all(f));
    f.erase(unique(all(f)), f.end());
    return f;
}
ull ord(ull a, ull p) {
    a %= p;
    ull t = p - 1;
    for (ull q : pfactors(t))
        while (t % q == 0 && modpow(a, t / q, p) == 1)
            t /= q;
    return t;
}
ull primitiveRoot(ull p) {
    if (p == 2) return 1;
    auto divs = pfactors(p - 1);
    for (ull g = 2; g < p; g++) {
        bool ok = 1;
        for (ull q : divs)
            if (modpow(g, (p - 1) / q, p) == 1) {
                ok = 0; break;
            }
        if (ok) return g;
    }
    return 0;
}
```

5.7 Estimates

$\sum_{d|n} d = \mathcal{O}(n \log \log n)$. Max $d(n)$ (highly composite): 128 at 83160 = $2^3 \cdot 3^3 \cdot 5 \cdot 7 \cdot 11$ ($n \leq 10^5$); 1344 at 735134400 = $2^6 \cdot 3^3 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17$ ($n \leq 10^9$); 103680 at 897612484786617600 = $2^8 \cdot 3^4 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31 \cdot 37$ ($n \leq 10^{18}$).

5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

Other useful formulas/forms:

$$\sum_{d|n} \mu(d) = [n = 1] \text{ (very useful)}$$

$$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$$

$$g(n) = \sum_{1 \leq m \leq n} f\left(\left\lfloor \frac{n}{m} \right\rfloor\right) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m) g\left(\left\lfloor \frac{n}{m} \right\rfloor\right)$$

$\lfloor n/i \rfloor$ is constant on $\mathcal{O}(\sqrt{n})$ blocks:

FloorBlocks.h

Description: Enumerates the $\mathcal{O}(\sqrt{n})$ blocks where $\lfloor n/i \rfloor$ is constant. Calls $f(l, r, q)$ for each $1 \leq l \leq r \leq n$ with $\lfloor n/i \rfloor = q$ for all $i \in [l, r]$. Both callback endpoints l and r are included.

Time: $\mathcal{O}(\sqrt{n})$

d41d8c, 8 lines

```
template<class F>
void floorBlocks(ll n, F f) {
    for (ll l = 1, r; l <= n; l = r + 1) {
        ll q = n / l;
        r = n / q;
        f(l, r, q);
    }
}
```

Mobius.h

Description: Fills $\mu(n)$ for $n \in [0, sz)$. Indices $[0, sz(\text{mobius}))$: 0 included, vector size excluded. See chapter text for inversion formulas.

Time: $\mathcal{O}(N \log N)$

d41d8c, 10 lines

```
void calcMobius(vi& mobius) {
    int n = sz(mobius);
    fill(all(mobius), 0);
    if (n > 1) mobius[1] = -1;
    rep(i, 1, n) if (mobius[i]) {
        mobius[i] = -mobius[i];
        for (int j = 2 * i; j < n; j += i)
            mobius[j] += mobius[i];
    }
}
```

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h

Description: Permutation -> integer conversion. (Not order preserving.) Integer -> permutation can use a lookup table.

Time: $\mathcal{O}(n)$

d41d8c, 6 lines

```
int permToInt(vi& v) {
    int use = 0, i = 0, r = 0;
    for(int x:v) r = r * ++i + __builtin_popcount(use & ~(1<<x)),
        use |= 1 << x; // (note: minus, not ~!)
    return r;
}
```

6.1.2 Cycles

Let $g_S(n)$ be the number of n -permutations whose cycle lengths all belong to the set S . Then

$$\sum_{n=0}^\infty g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{n \in S} \frac{x^n}{n}\right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1)+D(n-2)) = nD(n-1)+(-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

6.1.4 Burnside’s lemma

Given a group G of symmetries and a set X , the number of elements of X up to symmetry equals

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts “configurations” (of some sort) of length n , we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n} \sum_{k|n} f(k) \phi(n/k).$$

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \quad p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	$\sim 2e5$	$\sim 2e8$

6.2.2 Lucas’ Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \dots + n_1 p + n_0$ and $m = m_k p^k + \dots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod{p}$.

multinomial

6.2.3 Binomials

multinomial.h

Description: Computes $\binom{k_1 + \dots + k_n}{k_1, k_2, \dots, k_n} = \frac{(\sum k_i)!}{k_1! k_2! \dots k_n!}$. d41d8c, 5 lines

```
ll multinomial(vi& v) {
    ll c = 1, m = v.empty() ? 1 : v[0];
    rep(i,1,sz(v)) rep(j,0,v[i]) c = c * ++m / (j+1);
    return c;
}
```

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able).
 $B[0, \dots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \dots]$

Sums of powers:

$$\sum_{i=1}^n i^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\begin{aligned} \sum_{i=m}^\infty f(i) &= \int_m^\infty f(x) dx - \sum_{k=1}^\infty \frac{B_k}{k!} f^{(k-1)}(m) \\ &\approx \int_m^\infty f(x) dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m)) \end{aligned}$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n, k) = c(n-1, k-1) + (n-1)c(n-1, k), \quad c(0, 0) = 1$$
$$\sum_{k=0}^n c(n, k) x^k = x(x+1) \dots (x+n-1)$$

$$c(8, k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$$
$$c(n, 2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$$

6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j :s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j :s s.t. $\pi(j) \geq j$, k j :s s.t. $\pi(j) > j$.

$$E(n, k) = (n-k)E(n-1, k-1) + (k+1)E(n-1, k)$$
$$E(n, 0) = E(n, n-1) = 1$$
$$E(n, k) = \sum_{j=0}^k (-1)^j \binom{n+1}{j} (k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n, k) = S(n-1, k-1) + kS(n-1, k)$$
$$S(n, 1) = S(n, n) = 1$$
$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

6.3.5 Bell numbers

Total number of partitions of n distinct elements.
 $B(n) = \sum_k S(n, k) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$
For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod{p}$$

6.3.6 12-fold way

n balls into k bins (functions $[n] \rightarrow [k]$). D/I = distinguishable/indistinguishable. Columns: unrestricted, injective ($\leq 1/\text{bin}$), surjective ($\geq 1/\text{bin}$).

$n \ k$	any	≤ 1	≥ 1
D D	k^n	$k^{\underline{n}}$	$k! S(n, k)$
D I	$B(n, k)$	$[n \leq k]$	$S(n, k)$
I D	$\binom{n+k-1}{n}$	$\binom{k}{n}$	$\binom{n-1}{k-1}$
I I	$p_k(n+k)$	$[n \leq k]$	$p_k(n)$

$k^{\underline{n}} = k(k-1) \dots (k-n+1)$. $B(n, k) = \sum_{i=0}^k S(n, i)$. $p_k(n)$: partitions of n into exactly k parts ($p_k(n+k)$: at most k parts). $p_k(n) = p_{k-1}(n-1) + p_k(n-k)$, $p_0(0) = 1$.

6.3.7 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i : $n_1 n_2 \dots n_k n^{k-2}$
with degrees d_i : $(n-2)! / ((d_1-1)! \dots (d_n-1)!)$

6.3.8 Catalan numbers

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$
$$C_0 = 1, \quad C_{n+1} = \frac{2(2n+1)}{n+2} C_n, \quad C_{n+1} = \sum C_i C_{n-i}$$
$$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.

- permutations of $[n]$ with no 3-term increasing subseq.

Graph (7)

7.1 Fundamentals

7.1.1 Johnson's algorithm

All-pairs shortest paths with negative weights and no negative cycles. Run Bellman-Ford with every vertex at distance 0 (or from a virtual source); check for negative cycles. Reweight $w'(u, v) = w(u, v) + h(u) - h(v)$, then Dijkstra from each source.

TopoSort.h

Description: Topological sorting. Given is an oriented graph. Output is an ordering of vertices, such that there are edges only from left to right. If there are cycles, the returned list will have size smaller than n – nodes reachable from cycles will not be returned.

Time: $\mathcal{O}(|V| + |E|)$

d41d8c, 8 lines

```
vi topoSort(const vector<vi>& gr) {
    vi indeg(sz(gr)), q;
    for (auto& li : gr) for (int x : li) indeg[x]++;
    rep(i, 0, sz(gr)) if (indeg[i] == 0) q.push_back(i);
    rep(j, 0, sz(q)) for (int x : gr[q[j]])
        if (--indeg[x] == 0) q.push_back(x);
    return q;
}
```

7.2 Network flow

PushRelabel.h

Description: Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.

Time: $\mathcal{O}(V^2\sqrt{E})$

d41d8c, 48 lines

```
struct PushRelabel {
    struct Edge {
        int dest, back;
        ll f, c;
    };
    vector<vector<Edge>> g;
    vector<ll> ec;
    vector<Edge*> cur;
    vector<vi> hs; vi H;
    PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}

    void addEdge(int s, int t, ll cap, ll rcap=0) {
        if (s == t) return;
        g[s].push_back({t, sz(g[t]), 0, cap});
        g[t].push_back({s, sz(g[s])-1, 0, rcap});
    }

    void addFlow(Edge& e, ll f) {
        Edge &back = g[e.dest][e.back];
        if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
        e.f += f; e.c -= f; ec[e.dest] += f;
        back.f -= f; back.c += f; ec[back.dest] -= f;
    }

    ll calc(int s, int t) {
```

```
int v = sz(g); H[s] = v; ec[t] = 1;
vi co(2*v); co[0] = v-1;
rep(i, 0, v) cur[i] = g[i].data();
for (Edge& e : g[s]) addFlow(e, e.c);

for (int hi = 0;;) {
    while (hs[hi].empty()) if (!hi--) return -ec[s];
    int u = hs[hi].back(); hs[hi].pop_back();
    while (ec[u] > 0) // discharge u
        if (cur[u] == g[u].data() + sz(g[u])) {
            H[u] = 1e9;
            for (Edge& e : g[u]) if (e.c && H[u] > H[e.dest]+1)
                H[u] = H[e.dest]+1, cur[u] = &e;
            if (++co[H[u]], !--co[hi] && hi < v)
                rep(i, 0, v) if (hi < H[i] && H[i] < v)
                    --co[H[i]], H[i] = v + 1;
            hi = H[u];
        } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
            addFlow(*cur[u], min(ec[u], cur[u]->c));
        else ++cur[u];
    }
}

bool leftOfMinCut(int a) { return H[a] >= sz(g); }
```

MinCostMaxFlow.h

Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.

Time: $\mathcal{O}(FE \log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

d41d8c, 79 lines

```
#include <bits/extc++.h>

const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>> ed;
    vi seen;
    vector<ll> dist, pi;
    vector<edge*> par;

    MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

    void addEdge(int from, int to, ll cap, ll cost) {
        if (from == to) return;
        ed[from].push_back(edge{from, to, sz(ed[to]), cap, cost, 0});
        ed[to].push_back(edge{to, from, sz(ed[from])-1, 0, -cost, 0});
    }

    void path(int s) {
        fill(all(seen), 0);
        fill(all(dist), INF);
        dist[s] = 0; ll di;

        __gnu_pbds::priority_queue<pair<ll, int>> q;
        vector<decltype(q)::point_iterator> its(N);
        q.push({0, s});

        while (!q.empty()) {
            s = q.top().second; q.pop();
```

```
seen[s] = 1; di = dist[s] + pi[s];
for (edge& e : ed[s]) if (!seen[e.to]) {
    ll val = di - pi[e.to] + e.cost;
    if (e.cap - e.flow > 0 && val < dist[e.to]) {
        dist[e.to] = val;
        par[e.to] = &e;
        if (its[e.to] == q.end())
            its[e.to] = q.push({-dist[e.to], e.to});
        else
            q.modify(its[e.to], {-dist[e.to], e.to});
    }
}
rep(i, 0, N) pi[i] = min(pi[i] + dist[i], INF);
}

pair<ll, ll> maxflow(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
        ll fl = INF;
        for (edge* x = par[t]; x; x = par[x->from])
            fl = min(fl, x->cap - x->flow);

        totflow += fl;
        for (edge* x = par[t]; x; x = par[x->from]) {
            x->flow += fl;
            ed[x->to][x->rev].flow -= fl;
        }
    }
    rep(i, 0, N) for (edge& e : ed[i]) totcost += e.cost * e.flow;
    return {totflow, totcost/2};
}

// If some costs can be negative, call this before maxflow:
void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF); pi[s] = 0;
    int it = N, ch = 1; ll v;
    while (ch-- && it--)
        rep(i, 0, N) if (pi[i] != INF)
            for (edge& e : ed[i]) if (e.cap)
                if ((v = pi[i] + e.cost) < pi[e.to])
                    pi[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
}
};
```

EdmondsKarp.h

Description: Flow algorithm with guaranteed complexity $\mathcal{O}(VE^2)$. To get edge flow values, compare capacities before and after, and take the positive values only.

d41d8c, 36 lines

```
template<class T> T edmondsKarp(vector<unordered_map<int, T>>&
    graph, int source, int sink) {
    assert(source != sink);
    T flow = 0;
    vi par(sz(graph)), q = par;

    for (;) {
        fill(all(par), -1);
        par[source] = 0;
        int ptr = 1;
        q[0] = source;

        rep(i, 0, ptr) {
            int x = q[i];
```

```

    for (auto e : graph[x]) {
        if (par[e.first] == -1 && e.second > 0) {
            par[e.first] = x;
            q[ptr++] = e.first;
            if (e.first == sink) goto out;
        }
    }
    return flow;
out:
T inc = numeric_limits<T>::max();
for (int y = sink; y != source; y = par[y])
    inc = min(inc, graph[par[y]][y]);

flow += inc;
for (int y = sink; y != source; y = par[y]) {
    int p = par[y];
    if ((graph[p][y] -= inc) <= 0) graph[p].erase(y);
    graph[y][p] += inc;
}
}
}
}

```

Dinic.h

Description: Flow algorithm with complexity $O(VE \log U)$ where $U = \max |cap|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching.

d41d8c, 42 lines

```

struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need flows
    };
    vi lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, sz(adj[b]), c, c});
        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int& i = ptr[v]; i < sz(adj[v]); i++) {
            Edge& e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }
    ll calc(int s, int t) {
        ll flow = 0; q[0] = s;
        rep(L, 0, 31) do { // 'int L=30' maybe faster for random data
            lvl = ptr = vi(sz(q));
            int qi = 0, qe = lvl[s] = 1;
            while (qi < qe && !lvl[t]) {
                int v = q[qi++];
                for (Edge e : adj[v])
                    if (!lvl[e.to] && e.c >> (30 - L))
                        q[qi++] = e.to, lvl[e.to] = lvl[v] + 1;
            }
            while ((ll p = dfs(s, t, LLONG_MAX)) flow += p;

```

```

    } while (lvl[t]);
    return flow;
}
bool leftOfMinCut(int a) { return lvl[a] != 0; }
};

```

Dinic2.h

Description: Dinic without capacity scaling: smaller edges, one blocking-flow DFS per phase, and dead-end pruning. Often faster than Dinic.h; scaling can win on some graphs. Supports self-loops and parallel edges. DFS is recursive. Requires nonnegative capacities, $s \neq t$, $lim \geq 0$. Residual sums and total flow must fit in ll. `calc(s,t,lim)` returns at most `lim` additional units; repeated calls continue from the current residual graph. `leftOfMinCut` requires exhausted augmenting paths. If stopped at `lim`, finish with `calc(s,t)`. No original caps are stored. For edge `e`, signed net flow is `adj[e.to][e.rev].c - original_rcap`. Usually `rcap = 0`. See chapter notes for capacity edits between calls.

Time: $O(V^2E)$; $O(\min(E^{1/2}, V^{2/3})E)$ for unit caps; $O(\sqrt{V}E)$ for bipartite matching.

d41d8c, 40 lines

```

struct Dinic2 {
    struct Edge { int to, rev; ll c; };
    vector<vector<Edge>> adj;
    vi lvl, ptr, q;
    Dinic2(int n) : adj(n), lvl(n), ptr(n), q(n) {}
    void addEdge(int a, int b, ll c, ll rcap = 0) {
        int i = sz(adj[a]), j = sz(adj[b]) + (a == b);
        adj[a].push_back({b, j, c});
        adj[b].push_back({a, i, rcap});
    }
    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        ll g = 0;
        for (int& i = ptr[v]; i < sz(adj[v]); i++) {
            auto& [to, rev, c] = adj[v][i];
            if (lvl[to] != lvl[v] + 1 || !c) continue;
            ll w = min(f - g, c), p = dfs(to, t, w);
            if (p < w) lvl[to] = -1; // dead end this phase
            c -= p, adj[to][rev].c += p, g += p;
            if (g == f) break; // edge may still have room
        }
        return g;
    }
    ll calc(int s, int t, ll lim = LLONG_MAX) {
        assert(s != t && lim >= 0);
        ll flow = 0; q[0] = s;
        do {
            fill(all(lvl), 0), fill(all(ptr), 0);
            int qi = 0, qe = lvl[s] = 1;
            while (qi < qe && !lvl[t]) {
                int v = q[qi++];
                for (Edge e : adj[v]) if (!lvl[e.to] && e.c)
                    q[qi++] = e.to, lvl[e.to] = lvl[v] + 1;
            }
            ll p = dfs(s, t, lim); flow += p, lim -= p;
        } while (lvl[t] && lim);
        return flow;
    }
    bool leftOfMinCut(int a) { return lvl[a] != 0; }
};

```

MinCut.h

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s , only traversing edges with positive residual capacity.

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.

Time: $O(V^3)$

d41d8c, 21 lines

```

pair<int, vi> globalMinCut(vector<vi> mat) {
    pair<int, vi> best = {INT_MAX, {}};
    int n = sz(mat);
    vector<vi> co(n);
    rep(i, 0, n) co[i] = {i};
    rep(ph, 1, n) {
        vi w = mat[0];
        size_t s = 0, t = 0;
        rep(it, 0, n-ph) { // O(V^2) -> O(E log V) with prio. queue
            w[it] = INT_MIN;
            s = t, t = max_element(all(w)) - w.begin();
            rep(i, 0, n) w[i] += mat[t][i];
        }
        best = min(best, {w[t] - mat[t][t], co[t]});
        co[s].insert(co[s].end(), all(co[t]));
        rep(i, 0, n) mat[s][i] += mat[t][i];
        rep(i, 0, n) mat[i][s] = mat[s][i];
        mat[0][t] = INT_MIN;
    }
    return best;
}

```

GomoryHu.h

Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.

Time: $O(V)$ Flow Computations

"PushRelabel.h"

d41d8c, 13 lines

```

typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
    vector<Edge> tree;
    vi par(N);
    rep(i, 1, N) {
        PushRelabel D(N); // Dinic also works
        for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
        tree.push_back({i, par[i], D.calc(i, par[i])});
        rep(j, i+1, N)
            if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
    }
    return tree;
}

```

7.2.1 Dinic variants and capacity edits

Dinic uses capacity scaling; Dinic2 uses smaller edges and a blocking-flow DFS without scaling. Choose by workload:

Dinic2 is often faster, while scaling gives the stronger general worst-case bound. `Dinic2(n)` starts a fresh graph; to reset, assign `D = Dinic2(n)`. Save edge indices and original capacities yourself if needed.

Incremental flow (Dinic2). Track the total separately: $F += D.\text{calc}(s, t)$. Between calls the residuals must stay nonnegative, each pair must sum to its current cap plus reverse cap, and flow must be conserved away from s, t . For edge $e = \text{adj}[a][i]$ to b , let $r = \text{adj}[b][e.\text{rev}]$ and $f = r.c - \text{rcap}$. Raise its cap by d : $e.c += d$, then augment. Lower its cap by d without going below f : $e.c -= d$, then augment.

Lower below flow. For new cap $c' < f$, put $x = f - c'$, $e.c = 0$; $r.c = x$. Reroute with $z = x - D.\text{calc}(a, b, x)$; cancel the rest with $D.\text{calc}(a, s, z)$ if $a \neq s$ and $D.\text{calc}(t, b, z)$ if $b \neq t$. Then $F -= z$; $F += D.\text{calc}(s, t)$. Self-loops carry zero flow and need no rerouting. Min-cut labels refer to the last exhausted $s-t$ call; a call stopped by a flow limit does not certify a cut.

7.2.2 Demands and lower bounds

Lower bounds. Edge $u \rightarrow v$ with capacity in $[L, R]$: force L units via $\text{demand}[v] += L$, $\text{demand}[u] -= L$, then replace the edge by capacity $R - L$. ($L = 0$ is an ordinary capacity- R edge.) Original flow is L plus flow on the rewritten cap- $(R - L)$ edge.

Circulation. $d(v)$ is the forced net inflow (positive = excess). Residual flow must send net outflow $d(v)$ from each excess vertex. Add super-source S and super-sink T : $S \rightarrow v$ of cap $d(v)$ if $d(v) > 0$, $v \rightarrow T$ of cap $-d(v)$ if $d(v) < 0$. Feasible iff $\sum d = 0$ and max flow $S \rightarrow T$ equals $\sum_{d>0} d(v)$. A circulation is this check after the lower-bound rewrite.

Max $s-t$ flow. Add $t \rightarrow s$ cap ∞ , find a feasible circulation. Let F be the flow on $t \rightarrow s$; delete S, T , and $t \rightarrow s$, then max-augment $s \rightarrow t$. Value = F plus the extra max flow.

Min $s-t$ flow. Binary search the capacity C of $t \rightarrow s$; a feasible circulation exists iff a flow of value $\leq C$ exists. Minimize C ; the $s-t$ value is C (flow on $t \rightarrow s$).

7.2.3 König, path cover, Dilworth

König. A vertex cover meets every edge. On a bipartite graph, max matching = min vertex cover; the complement of a min cover is a max independent set. Hall: a matching saturates L iff $|N(S)| \geq |S|$ for every $S \subseteq L$. cover (Matching): from unmatched left vertices follow unmatched $L \rightarrow R$ and matched $R \rightarrow L$; the cover is unvisited left plus visited right. NP-hard on general graphs.

Path cover. Min vertex-disjoint path cover of a DAG: split each v into v_L (left) and v_R (right); each edge $u \rightarrow v$ becomes $u_L \rightarrow v_R$. Size = n minus max matching. A match $u \mapsto v$ means u is followed by v on a path. NP-hard on general digraphs.

Dilworth. In a poset, min chain cover = max antichain. Build the comparability DAG ($u \rightarrow v$ iff $u < v$) and take a min path cover. Max antichain = $\{x : \text{neither copy of } x \text{ is in the König cover}\}$. Mirsky (dual): min antichain cover = max chain; partition by longest-path height. On a sequence, min decreasing-subsequence cover = LIS.

7.3 Matching

HopcroftKarp.h

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and r should be a vector full of -1 's of the same size as the right partition. Returns the size of the matching. $r[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.

Time: $\mathcal{O}(E\sqrt{V})$

d41d8c, 20 lines

```
int hopcroftKarp(vector<vi>& g, vi& r) {
    int n = sz(g), res = 0;
    vi l(n, -1), q(n), d(n);
    auto dfs = [&](auto f, int u) -> bool {
        int t = exchange(d[u], 0) + 1;
        for (int v : g[u])
            if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
                return l[u] = v, r[v] = u, 1;
        return 0;
    };
    for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
        rep(i, 0, n) if (l[i] == -1) q[t++] = i, d[i] = 1;
        rep(i, 0, t) for (int v : g[q[i]]) {
            if (r[v] == -1) f = 1;
            else if (!d[r[v]]) d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
        }
        if (!f) return res;
        rep(i, 0, n) if (l[i] == -1) res += dfs(dfs, i);
    }
}
```

DFSMatching.h

Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1 's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.

Usage: $vi\ btoa(m, -1); \text{dfsMatching}(g, btoa);$

Time: $\mathcal{O}(VE)$

d41d8c, 22 lines

```
bool find(int j, vector<vi>& g, vi& btoa, vi& vis) {
    if (btoa[j] == -1) return 1;
    vis[j] = 1; int di = btoa[j];
    for (int e : g[di])
        if (!vis[e] && find(e, g, btoa, vis)) {
            btoa[e] = di;
            return 1;
        }
}
```

```
return 0;
}
int dfsMatching(vector<vi>& g, vi& btoa) {
    vi vis;
    rep(i, 0, sz(g)) {
        vis.assign(sz(btoa), 0);
        for (int j : g[i])
            if (find(j, g, btoa, vis)) {
                btoa[j] = i;
                break;
            }
    }
    return sz(btoa) - (int)count(all(btoa), -1);
}
```

MinimumVertexCover.h

Description: Vertex cover: a set of vertices incident to every edge. Min cover in a bipartite graph (König: size equals max matching). Complement is a max independent set.

d41d8c, 20 lines

```
vi cover(vector<vi>& g, int n, int m) {
    vi match(m, -1);
    int res = dfsMatching(g, match);
    vector<bool> lfound(n, true), seen(m);
    for (int it : match) if (it != -1) lfound[it] = false;
    vi q, cover;
    rep(i, 0, n) if (lfound[i]) q.push_back(i);
    while (!q.empty()) {
        int i = q.back(); q.pop_back();
        lfound[i] = 1;
        for (int e : g[i]) if (!seen[e] && match[e] != -1) {
            seen[e] = true;
            q.push_back(match[e]);
        }
    }
    rep(i, 0, n) if (!lfound[i]) cover.push_back(i);
    rep(i, 0, m) if (seen[i]) cover.push_back(n+i);
    assert(sz(cover) == res);
    return cover;
}
```

WeightedMatching.h

Description: Assignment (min-cost bipartite matching). Matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes $\text{cost}[N][M]$, where $\text{cost}[i][j] = \text{cost for } L[i] \text{ to be matched with } R[j]$, and returns (min cost, match), where $L[i]$ is matched with $R[\text{match}[i]]$. Negate costs for max cost. Requires $N \leq M$.

Time: $\mathcal{O}(N^2M)$

d41d8c, 24 lines

```
template<class T>
pair<T, vi> weightedMatching(vector<vector<T>> &C) {
    int i = sz(C), m = i ? sz(C[0]) : 0, c, s, r;
    vector<T> dist(m), pot(m);
    vi match(i, rev(m, -1), cols(m), prev(m));
    T d = 0, nd, cost = 0;
    while (i--) {
        rep(c, 0, m) dist[c] = C[i][c], cols[c] = c, prev[c] = i;
        for (s = 0;;) {
            for (s = 0;;) {
                rep(j, s, m) {
                    c = cols[j], nd = dist[c] - pot[c];
                    if (j == s || d > nd) d = nd, swap(cols[s], cols[j]);
                }
                if ((r = rev[c = cols[s++]]) == -1) break;
            }
        }
    }
```

```

    rep(j,0,m) if (dist[j] > (nd = C[r][j]-C[r][c]+dist[c]))
        dist[j] = nd, prev[j] = r;
}
cost += dist[c];
while (s--) pot[cols[s]] = dist[cols[s]] - d;
for (; r != i; swap(c, match[r]))
    r = rev[c] = prev[c];
}
return {cost, match};
}

```

Blossom.h

Description: Maximum cardinality matching in general graphs (Gabow–Edmonds). 0-indexed. ae adds an undirected edge. After solve(), mate[u] is the partner of *u*, or -1 if unmatched. Returns matching size.

Time: $\mathcal{O}(NM)$

d41d8c, 86 lines

```

struct Blossom {
    struct Edge { int to, idx; };
    int n;
    vector<vector<Edge>> g;
    vector<pii> edges;
    vi mt, label, first, q, mate;
    Blossom(int n) : n(n), g(n+1), mt(n+1),
        label(n+1, -1), first(n+1) {}
    void ae(int u, int v) {
        if (u == v) return;
        ++u, ++v;
        g[u].push_back({v, sz(edges) + n + 1});
        g[v].push_back({u, sz(edges) + n + 1});
        edges.emplace_back(u, v);
    }
    int find(int x) {
        return label[first[x]] < 0 ? first[x]
            : first[x] = find(first[x]);
    }
    void rematch(int v, int w) {
        int t = mt[v];
        mt[v] = w;
        if (mt[t] != v) return;
        if (label[v] <= n) {
            mt[t] = label[v];
            rematch(label[v], t);
        } else {
            auto [x, y] = edges[label[v] - n - 1];
            rematch(x, y);
            rematch(y, x);
        }
    }
    void assign_label(int x, int y, int num) {
        int r = find(x), s = find(y), join = 0;
        if (r == s) return;
        label[r] = label[s] = -num;
        while (1) {
            if (s) swap(r, s);
            r = find(label[mt[r]]);
            if (label[r] == -num) { join = r; break; }
            label[r] = -num;
        }
        for (int v : {first[x], first[y]}) {
            while (v != join) {
                q.push_back(v);
                label[v] = num;
                first[v] = join;
                v = first[label[mt[v]]];
            }
        }
    }
}

```

```

}
}
}
bool augment(int u) {
    q = {u};
    first[u] = 0;
    label[u] = 0;
    for (int qi = 0; qi < sz(q); ++qi) {
        int x = q[qi];
        for (auto e : g[x]) {
            int y = e.to;
            if (!mt[y] && y != u) {
                mt[y] = x;
                rematch(x, y);
                return 1;
            } else if (label[y] >= 0)
                assign_label(x, y, e.idx);
            else if (label[mt[y]] < 0) {
                label[mt[y]] = x;
                first[mt[y]] = y;
                q.push_back(mt[y]);
            }
        }
    }
    return 0;
}
int solve() {
    rep(i,1,n+1) if (!mt[i] && augment(i))
        label.assign(n+1, -1);
    int ans = 0;
    mate.assign(n, -1);
    rep(i,1,n+1) if (mt[i]) {
        mate[i-1] = mt[i] - 1;
        if (i < mt[i]) ++ans;
    }
    return ans;
}
};

```

7.4 DFS algorithms

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices *u*, *v* belong to the same component, we can reach *u* from *v* and vice versa.

Usage: scc(graph, [&](vi& v) { ... }) visits all components in reverse topological order. comp[i] holds the component index of a node (a component only has edges to components with lower index). ncomps will contain the number of components.

Time: $\mathcal{O}(E + V)$

d41d8c, 24 lines

```

vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F& f) {
    int low = val[j] = ++Time, x; z.push_back(j);
    for (auto e : g[j]) if (comp[e] < 0)
        low = min(low, val[e] ? dfs(e, g, f));

    if (low == val[j]) {
        do {
            x = z.back(); z.pop_back();
            comp[x] = ncomps;
            cont.push_back(x);
        } while (x != j);
        f(cont); cont.clear();
    }
}

```

```

    ncomps++;
}
return val[j] = low;
}
template<class G, class F> void scc(G& g, F f) {
    int n = sz(g);
    val.assign(n, 0); comp.assign(n, -1);
    Time = ncomps = 0;
    rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
}

```

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two internally disjoint paths between any two nodes (a cycle exists through them). Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.

Usage: int eid = 0; ed.resize(N); for each edge (a,b) { ed[a].emplace_back(b, eid); ed[b].emplace_back(a, eid++); } bicomps([&](const vi& edgelist) {...});

Time: $\mathcal{O}(E + V)$

d41d8c, 33 lines

```

// consider cut vertices, and cut edges
vi num, st;
vector<vector<pii>> ed;
int Time;
template<class F>
int dfs(int at, int par, F& f) {
    int me = num[at] = ++Time, top = me;
    for (auto [y, e] : ed[at]) if (e != par) {
        if (num[y]) {
            top = min(top, num[y]);
            if (num[y] < me)
                st.push_back(e);
        } else {
            int si = sz(st);
            int up = dfs(y, e, f);
            top = min(top, up);
            if (up == me) {
                st.push_back(e);
                f(vi(st.begin() + si, st.end()));
                st.resize(si);
            }
            else if (up < me) st.push_back(e);
            else { /* e is a bridge */ }
        }
    }
    return top;
}
}

```

```

template<class F>
void bicomps(F f) {
    num.assign(sz(ed), 0);
    rep(i,0,sz(ed)) if (!num[i]) dfs(i, -1, f);
}

```

2sat.h

Description: Calculates a valid assignment to boolean variables *a*, *b*, *c*,... to a 2-SAT problem, so that an expression of the type (a||b)&&(!a||c)&&(d||!b)&&... becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).

Usage: TwoSat ts(number of boolean variables);
 ts.either(0, ~3); // Var 0 is true or var 3 is false
 ts.setValue(2); // Var 2 is true
 ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true
 ts.solve(); // Returns true iff it is solvable
 ts.values[0..N-1] holds the assigned values to the vars
Time: $\mathcal{O}(N + E)$, where N is the number of boolean variables, and E is the number of clauses.

d41d8c, 56 lines

```
struct TwoSat {
    int N;
    vector<vi> gr;
    vi values; // 0 = false, 1 = true

    TwoSat(int n = 0) : N(n), gr(2*n) {}

    int addVar() { // (optional)
        gr.emplace_back();
        gr.emplace_back();
        return N++;
    }

    void either(int f, int j) {
        f = max(2*f, -1-2*f);
        j = max(2*j, -1-2*j);
        gr[f].push_back(j^1);
        gr[j].push_back(f^1);
    }

    void setValue(int x) { either(x, x); }

    void atMostOne(const vi& li) { // (optional)
        if (sz(li) <= 1) return;
        int cur = ~li[0];
        rep(i,2,sz(li)) {
            int next = addVar();
            either(cur, ~li[i]);
            either(cur, next);
            either(~li[i], next);
            cur = ~next;
        }
        either(cur, ~li[1]);
    }

    vi val, comp, z; int time = 0;
    int dfs(int i) {
        int low = val[i] = ++time, x; z.push_back(i);
        for(int e : gr[i]) if (!comp[e])
            low = min(low, val[e] ? dfs(e));
        if (low == val[i]) do {
            x = z.back(); z.pop_back();
            comp[x] = low;
            if (values[x>>1] == -1)
                values[x>>1] = x&1;
        } while (x != i);
        return val[i] = low;
    }

    bool solve() {
        values.assign(N, -1);
        val.assign(2*N, 0); comp = val;
        rep(i,0,2*N) if (!comp[i]) dfs(i);
        rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
        return 1;
    }
};
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle starting at src (and ending at src if it is a cycle), or empty list if no path/cycle exists. To get edge indices back, add .second to s and ret.
Time: $\mathcal{O}(V + E)$

d41d8c, 15 lines

```
vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src=0) {
    int n = sz(gr);
    vi D(n), its(n), eu(nedges), ret, s = {src};
    D[src]++; // to allow Euler paths, not just cycles
    while (!s.empty()) {
        int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
        if (it == end){ ret.push_back(x); s.pop_back(); continue; }
        tie(y, e) = gr[x][it++];
        if (!eu[e]) {
            D[x]--, D[y]++;
            eu[e] = 1; s.push_back(y);
        }
    }
    for (int x : D) if (x < 0 || sz(ret) != nedges+1) return {};
    return {ret.rbegin(), ret.rend()};
}
```

DominatorTree.h

Description: Dominator tree of a directed graph. Assumes every node is reachable from *root*. *a* dominates *b* iff every path from *root* to *b* passes through *a*. Construct with the adjacency list and root; then ans[u] holds the children of *u* in the dominator tree.
Time: $\mathcal{O}(M \log N)$

d41d8c, 44 lines

```
struct Dominator {
    int n, co = 0;
    vector<vi> adj, ans, radj, child, sdomChild;
    vi label, rlabel, sdom, dom, par, bes;
    int get(int x) { // DSU with path compression
        // vertex with smallest sdom on path to root
        if (par[x] != x) {
            int t = get(par[x]); par[x] = par[par[x]];
            if (sdom[t] < sdom[bes[x]]) bes[x] = t;
        }
        return bes[x];
    }
    void dfs(int x) { // create DFS tree; labels 0..n-1
        label[x] = co; rlabel[co] = x;
        sdom[co] = par[co] = bes[co] = co;
        ++co;
        for (int y : adj[x]) {
            if (label[y] < 0) {
                dfs(y); child[label[x]].pb(label[y]);
                radj[label[y]].pb(label[x]);
            }
        }
    }
    Dominator(const vector<vi>& g, int root) : n(sz(g)),
        adj(g), ans(n), radj(n), child(n), sdomChild(n),
        label(n, -1), rlabel(n), sdom(n),
        dom(n), par(n), bes(n) {
        dfs(root);
        for (int i = co - 1; i >= 0; --i) {
            for (int j : radj[i])
                sdom[i] = min(sdom[i], sdom[get(j)]);
            if (i) sdomChild[sdom[i]].pb(i);
            for (int j : sdomChild[i]) {
                int k = get(j);
```

```
                if (sdom[j] == sdom[k]) dom[j] = sdom[j];
                else dom[j] = k;
            }
            for (int j : child[i]) par[j] = i;
        }
        rep(i,1,co) {
            if (dom[i] != sdom[i]) dom[i] = dom[dom[i]];
            ans[rlabel[dom[i]]].pb(rlabel[i]);
        }
    }
};
```

7.5 Coloring

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D , computes a $(D+1)$ -coloring of the edges such that no neighboring edges share a color. (D -coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)
Time: $\mathcal{O}(NM)$

d41d8c, 31 lines

```
vi edgeColoring(int N, vector<pii> eds) {
    vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
    for (pii e : eds) ++cc[e.first], ++cc[e.second];
    int u, v, ncols = *max_element(all(cc)) + 1;
    vector<vi> adj(N, vi(ncols, -1));
    for (pii e : eds) {
        tie(u, v) = e;
        fan[0] = v;
        loc.assign(ncols, 0);
        int at = u, end = u, d, c = free[u], ind = 0, i = 0;
        while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
            loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
        cc[loc[d]] = c;
        for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
            swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
        while (adj[fan[i]][d] != -1) {
            int left = fan[i], right = fan[++i], e = cc[i];
            adj[u][e] = left;
            adj[left][e] = u;
            adj[right][e] = -1;
            free[right] = e;
        }
        adj[u][d] = fan[i];
        adj[fan[i]][d] = u;
        for (int y : {fan[0], u, end})
            for (int& z = free[y] = 0; adj[y][z] != -1; z++);
    }
    rep(i,0,sz(eds))
        for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
    return ret;
}
```

7.6 Heuristics

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

d41d8c, 12 lines

```
typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}) {
```

```

if (!P.any()) { if (!X.any()) f(R); return; }
auto q = (P | X)._Find_first();
auto cands = P & ~eds[q];
rep(i,0,sz(eds)) if (cands[i]) {
    R[i] = 1;
    cliques(eds, f, P & eds[i], X & eds[i], R);
    R[i] = P[i] = 0; X[i] = 1;
}
}
}

```

MaximumClique.h

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.

Time: Runs in about 1s for $n=155$ and worst case random graphs ($p=.90$). Runs faster for sparse graphs.

d41d8c, 49 lines

```

typedef vector<bitset<200>> vb;
struct Maxclique {
    double limit=0.025, pk=0;
    struct Vertex { int i, d=0; };
    typedef vector<Vertex> vv;
    vb e;
    vv V;
    vector<vi> C;
    vi qmax, q, S, old;
    void init(vv& r) {
        for (auto& v : r) v.d = 0;
        for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
        sort(all(r), [](auto a, auto b) { return a.d > b.d; });
        int mxD = r[0].d;
        rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
    }
    void expand(vv& R, int lev = 1) {
        S[lev] += S[lev - 1] - old[lev];
        old[lev] = S[lev - 1];
        while (sz(R)) {
            if (sz(q) + R.back().d <= sz(qmax)) return;
            q.push_back(R.back().i);
            vv T;
            for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
            if (sz(T)) {
                if (S[lev]++ / ++pk < limit) init(T);
                int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
                C[1].clear(), C[2].clear();
                for (auto v : T) {
                    int k = 1;
                    auto f = [&](int i) { return e[v.i][i]; };
                    while (any_of(all(C[k]), f)) k++;
                    if (k > mxk) mxk = k, C[mxk + 1].clear();
                    if (k < mnk) T[j++].i = v.i;
                    C[k].push_back(v.i);
                }
                if (j > 0) T[j - 1].d = 0;
                rep(k,mnk,mxk + 1) for (int i : C[k])
                    T[j].i = i, T[j++].d = k;
                expand(T, lev + 1);
            } else if (sz(q) > sz(qmax)) qmax = q;
            q.pop_back(), R.pop_back();
        }
    }
    vi maxClique() { init(V), expand(V); return qmax; }
    Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
        rep(i,0,sz(e)) V.push_back({i});
    }
}

```

};

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertexCover.

7.7 Trees

BinaryLifting.h

Description: Calculate power of two jumps in a tree, to support fast upward jumps and LCAs. Assumes the root node points to itself.

Time: construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$

d41d8c, 25 lines

```

vector<vi> treeJump(vi& P){
    int on = 1, d = 1;
    while(on < sz(P)) on *= 2, d++;
    vector<vi> jmp(d, P);
    rep(i,1,d) rep(j,0,sz(P))
        jmp[i][j] = jmp[i-1][jmp[i-1][j]];
    return jmp;
}

```

```

int jmp(vector<vi>& tbl, int nod, int steps){
    rep(i,0,sz(tbl))
        if(steps&(1<<i)) nod = tbl[i][nod];
    return nod;
}

```

```

int lca(vector<vi>& tbl, vi& depth, int a, int b) {
    if (depth[a] < depth[b]) swap(a, b);
    a = jmp(tbl, a, depth[a] - depth[b]);
    if (a == b) return a;
    for (int i = sz(tbl); i--;) {
        int c = tbl[i][a], d = tbl[i][b];
        if (c != d) a = c, b = d;
    }
    return tbl[0][a];
}

```

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.

Time: $\mathcal{O}(N \log N + Q)$

"../data-structures/RMQ.h"

d41d8c, 21 lines

```

struct LCA {
    int T = 0;
    vi time, path, ret;
    RMQ<int> rmq;

    LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C,0,-1), ret)) {}
    void dfs(vector<vi>& C, int v, int par) {
        time[v] = T++;
        for (int y : C[v]) if (y != par) {
            path.push_back(v), ret.push_back(time[v]);
            dfs(C, y, v);
        }
    }

    int lca(int a, int b) {
        if (a == b) return a;
    }
}

```

```

    tie(a, b) = minmax(time[a], time[b]);
    return path[rmq.query(a, b)];
}
//dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];}
};

```

VirtualTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S| - 1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.

Time: $\mathcal{O}(|S| \log |S|)$

"LCA.h"

d41d8c, 21 lines

```

typedef vector<pair<int, int>> vpi;
vpi virtualTree(LCA& lca, const vi& subset) {
    static vi rev; rev.resize(sz(lca.time));
    vi li = subset, &T = lca.time;
    auto cmp = [&](int a, int b) { return T[a] < T[b]; };
    sort(all(li), cmp);
    int m = sz(li)-1;
    rep(i,0,m) {
        int a = li[i], b = li[i+1];
        li.push_back(lca.lca(a, b));
    }
    sort(all(li), cmp);
    li.erase(unique(all(li)), li.end());
    rep(i,0,sz(li)) rev[li[i]] = i;
    vpi ret = {pii(0, li[0])};
    rep(i,0,sz(li)-1) {
        int a = li[i], b = li[i+1];
        ret.emplace_back(rev[lca.lca(a, b)], b);
    }
    return ret;
}

```

Centroid.h

Description: Centroid decomposition. Call decomp(0, f). When f(c) runs, c is the centroid of its current component and ancestor centroids are in banned. DFS the original tree skipping banned to see c's component. Each node is passed to f once.

Time: $\mathcal{O}(N \log N)$

d41d8c, 28 lines

```

struct Centroid {
    vector<vi> g;
    vector<bool> banned;
    vi siz;
    Centroid(int n) : g(n), banned(n), siz(n) {}
    void ae(int a, int b) {
        g[a].push_back(b);
        g[b].push_back(a);
    }
    int dfsSz(int u, int p) {
        siz[u] = 1;
        for (int v : g[u]) if (v != p && !banned[v])
            siz[u] += dfsSz(v, u);
        return siz[u];
    }
    int dfsCen(int u, int p, int n) {
        for (int v : g[u]) if (v != p && !banned[v])
            if (siz[v] > n / 2) return dfsCen(v, u, n);
        return u;
    }
    template<class F>
    void decomp(int u, F f) {
    }
}

```

```

    int c = dfsCen(u, -1, dfsSz(u, -1));
    f(c);
    banned[c] = true;
    for (int v : g[c]) if (!banned[v]) decomp(v, f);
}
};

```

Reroot.h

Description: DP for every root of an undirected tree on vertices 0 through $N-1$. Modify T and the hooks. merge is associative with identity id; preserves adjacency order. vertex(a,v) includes v as the root above the merged branches a. edge(a,from,to) turns the component rooted at from into a branch of to, accounting for the edge but excluding to itself. Prefix/suffix merges exclude each child's branch.

Usage: Reroot dp(g); dp.run(root); root may be any vertex.

As written, ans[v].sum is the sum of distances from v. T = (count, distance sum). vertex adds v at distance zero; edge measures the same vertices from to, so every distance increases by one: sum += cnt.

For leaf u next to v: vertex(id,u) = (1,0), edge((1,0),u,v) = (1,1), vertex((1,1),v) = (2,1).

Memory: $\mathcal{O}(N)$; recursion depth can be N .

Time: $\mathcal{O}(N)$ with constant-size T and $\mathcal{O}(1)$ hooks.

d41d8c, 53 lines

```

struct Reroot {
    struct T { ll cnt, sum; };
    T id{0, 0};
    vector<vi> g;
    vector<T> down, ans;
    Reroot(vector<vi> g)
        : g(g), down(sz(g)), ans(sz(g)) {}

    // Modify these hooks for the problem:
    T merge(T a, T b) {
        return {a.cnt + b.cnt, a.sum + b.sum};
    }
    // Include v above the branches a, at distance zero.
    T vertex(T a, int v) { ++a.cnt; return a; }
    // Measure from's component from to; to stays excluded.
    T edge(T a, int from, int to) {
        a.sum += a.cnt;
        return a;
    }
}

```

```

void dfs1(int v, int p) {
    T cur = id;
    for (int u : g[v]) if (u != p) {
        dfs1(u, v);
        cur = merge(cur, edge(down[u], u, v));
    }
    down[v] = vertex(cur, v);
}

void dfs2(int v, int p, T up) {
    int m = sz(g[v]);
    vector<T> val(m), pre(m + 1, id);
    rep(i, 0, m) {
        int u = g[v][i];
        val[i] = u == p ? up : edge(down[u], u, v);
        pre[i + 1] = merge(pre[i], val[i]);
    }
    ans[v] = vertex(pre[m], v);
    T suf = id;
    for (int i = m; i--;) {
        int u = g[v][i];
        if (u != p) {

```

```

            T cur = vertex(merge(pre[i], suf), v);
            dfs2(u, v, edge(cur, v, u));
        }
        suf = merge(val[i], suf);
    }
}

void run(int root = 0) {
    if (g.empty()) return;
    dfs1(root, -1);
    dfs2(root, -1, id);
}
};

```

HLD.h

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most $\log(n)$ light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0. Path $u-v$ includes both endpoints (node values). In edge mode, includes all path edges, excluding the LCA's stored value. Subtrees include v in node mode, exclude its parent edge in edge mode. process callbacks use $[l, r]$: l included, r excluded. Subtree positions are $[pos[v] + VALS_EDGES, pos[v] + siz[v]]$. The underlying lazy tree uses inclusive $[l, r - 1]$.

Time: $\mathcal{O}((\log N)^2)$

"/data-structures/LazySegmentTree.h" d41d8c, 49 lines

```

template <bool VALS_EDGES> struct HLD {
    int N, tim = 0;
    vector<vi> adj;
    vi par, siz, rt, pos;
    LazyUpdateTree tree;
    HLD(vector<vi> adj_)
        : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
          rt(N), pos(N), tree(N) { dfsSz(0); dfsHld(0); }
    void dfsSz(int v) {
        for (int& u : adj[v]) {
            adj[u].erase(find(all(adj[u]), v));
            par[u] = v;
            dfsSz(u);
            siz[v] += siz[u];
            if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
        }
    }
    void dfsHld(int v) {
        pos[v] = tim++;
        for (int u : adj[v]) {
            rt[u] = (u == adj[v][0] ? rt[v] : u);
            dfsHld(u);
        }
    }
    template <class B> void process(int u, int v, B op) {
        for (; v = par[rt[v]];) {
            if (pos[u] > pos[v]) swap(u, v);
            if (rt[u] == rt[v]) break;
            op(pos[rt[v]], pos[v] + 1);
        }
        op(pos[u] + VALS_EDGES, pos[v] + 1);
    }
    void modifyPath(int u, int v, int val) {
        process(u, v, [&](int l, int r) {
            tree.update(l, r - 1, val);
        });
    }
}

```

```

int queryPath(int u, int v) { // Modify depending on problem
    int res = tree.id;
    process(u, v, [&](int l, int r) {
        res = tree.binop(res, tree.query(l, r - 1));
    });
    return res;
}

int querySubtree(int v) { // modifySubtree is similar
    return tree.query(pos[v] + VALS_EDGES,
        pos[v] + siz[v] - 1);
}
};

```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.

Time: $\mathcal{O}(E \log V)$

"/data-structures/UnionFindRollback.h" d41d8c, 60 lines

```

struct Edge { int a, b; ll w; };
struct Node {
    Edge key;
    Node *l, *r;
    ll delta;
    void prop() {
        key.w += delta;
        if (l) l->delta += delta;
        if (r) r->delta += delta;
        delta = 0;
    }
    Edge top() { prop(); return key; }
};

Node *merge(Node *a, Node *b) {
    if (!a || !b) return a ? b : a->prop(), b->prop();
    if (a->key.w > b->key.w) swap(a, b);
    swap(a->l, (a->r = merge(b, a->r)));
    return a;
}

void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

```

```

pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
    RollbackUF uf(n);
    vector<Node*> heap(n);
    ll res = 0;
    vi seen(n, -1), path(n), par(n);
    seen[r] = r;
    vector<Edge> Q(n), in(n, {-1,-1}), comp;
    deque<tuple<int, int, vector<Edge>>> cys;
    rep(s, 0, n) {
        int u = s, qi = 0, w;
        while (seen[u] < 0) {
            if (!heap[u]) return {-1, {}};
            Edge e = heap[u]->top();
            heap[u]->delta -= e.w, pop(heap[u]);
            Q[qi] = e, path[qi++] = u, seen[u] = s;
            res += e.w, u = uf.find(e.a);
            if (seen[u] == s) {
                Node* cyc = 0;
                int end = qi, time = uf.time();
                do cyc = merge(cyc, heap[w = path[--qi]]);
                while (uf.join(u, w));
                u = uf.find(u), heap[u] = cyc, seen[u] = -1;
                cys.push_front({u, time, {&Q[qi], &Q[end]}});
            }

```

```

    }
    rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
}

for (auto& [u,t,comp] : cycs) { // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
}
rep(i,0,n) par[i] = in[i].a;
return {res, par};
}
}

```

SteinerTree.h

Description: Minimum Steiner tree in an undirected graph with nonnegative weights. e is $\{u,v,w\}$ edges; s is the terminals. Returns (min cost, indices into e). Tree reconstruction is optional. Returns $\{-1, \{\}\}$ if the terminals cannot be connected. $k = 0$ costs 0. If k is large and n is small, enumerate extra vertices and MST instead.

Memory: $\mathcal{O}(2^k n)$

Time: $\mathcal{O}(3^k n + 2^k (n \log n + m))$

d41d8c, 56 lines

```

const ll inf = 1LL << 60;
pair<ll, vi> steinerTree(int n, vi s, vector<array<ll, 3>> e) {
    int k = sz(s), N = 1 << k;
    if (!k) return {0, {}};
    struct E { int to, id; ll w; };
    vector<vector<E>> g(n);
    rep(i,0,sz(e)) {
        int u = (int)e[i][0], v = (int)e[i][1];
        g[u].pb({v, i, e[i][2]});
        g[v].pb({u, i, e[i][2]});
    }
    vector<ll> dp(N * n, inf);
    vector<pii> pre(N * n, {-1,-1}); // (optional)
    rep(i,0,k) dp[(1 << i) * n + s[i]] = 0;
    typedef pair<ll, int> pli;
    rep(x,1,N) {
        int xn = x * n;
        for (int y = (x - 1) & x; y; y = (y - 1) & x) {
            int yn = y * n, zn = (x ^ y) * n;
            rep(r,0,n) {
                ll tmp = dp[yn + r] + dp[zn + r];
                if (tmp < dp[xn + r]) {
                    dp[xn + r] = tmp;
                    pre[xn + r] = {y, -1}; // (optional)
                }
            }
        }
    }
    priority_queue<pli, vector<pli>, greater<pli>> pq;
    rep(r,0,n) if (dp[xn + r] < inf)
        pq.push({dp[xn + r], r});
    while (!pq.empty()) {
        auto [d, v] = pq.top(); pq.pop();
        if (d != dp[xn + v]) continue;
        for (auto [to, id, w] : g[v])
            if (d + w < dp[xn + to]) {
                dp[xn + to] = d + w;
                pre[xn + to] = {v, id}; // (optional)
                pq.push({d + w, to});
            }
    }
}
}

```

```

int fn = (N - 1) * n, best = 0;
rep(r,0,n) if (dp[fn + r] < dp[fn + best]) best = r;
if (dp[fn + best] >= inf) return {-1, {}};
vi t; // reconstruct (optional)
vi use(sz(e));
auto rec = [&](auto rec, int x, int r) -> void {
    auto [a, b] = pre[x * n + r];
    if (a < 0) return;
    if (b < 0) rec(rec, a, r), rec(rec, x ^ a, r);
    else use[b] = 1, rec(rec, x, a);
};
rec(rec, N - 1, best);
rep(i,0,sz(e)) if (use[i]) t.pb(i);
return {dp[fn + best], t};
}

```

7.8 Math

7.8.1 Number of Spanning Trees

Create an $N \times N$ matrix mat , and for each edge $a \rightarrow b \in G$, do $mat[a][b]--$, $mat[b][b]++$ (and $mat[b][a]--$, $mat[a][a]++$ if G is undirected). Delete row and column i and take the determinant to get the number of directed spanning trees rooted at i (if undirected, any row/column).

7.8.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Strings (8)

KMP.h

Description: $pi[x]$ computes the length of the longest prefix of s that ends at x , other than $s[0\dots x]$ itself (abacaba -> 0010123). That prefix is $[0, pi[x])$ and matches $[x - pi[x] + 1, x + 1]$: starts included, ends excluded. Can be used to find all occurrences of a string.

Time: $\mathcal{O}(n)$

d41d8c, 16 lines

```

vi pi(const string& s) {
    vi p(sz(s));
    rep(i,1,sz(s)) {
        int g = p[i-1];
        while (g && s[i] != s[g]) g = p[g-1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}

vi match(const string& s, const string& pat) {
    vi p = pi(pat + '\0' + s), res;
    rep(i,sz(p)-sz(s),sz(p))
        if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
    return res;
}

```

Zfunc.h

Description: $z[i]$ computes the length of the longest common prefix of $s[i:]$ and s , except $z[0] = 0$. (abacaba -> 0010301) Matches $[0, z[i])$ with $[i, i + z[i])$: starts included, ends excluded.

Time: $\mathcal{O}(n)$

d41d8c, 12 lines

```

vi Z(const string& S) {
    vi z(sz(S));
    int l = -1, r = -1;
    rep(i,1,sz(S)) {
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
        while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
            z[i]++;
        if (i + z[i] > r)
            l = i, r = i + z[i];
    }
    return z;
}

```

Manacher.h

Description: For each position in a string, computes $p[0][i]$ = half length of longest even palindrome around pos i , $p[1][i]$ = longest odd (half rounded down). Even palindrome: $[i - p[0][i], i + p[0][i])$. Odd palindrome: $[i - p[1][i], i + p[1][i] + 1)$. Both include the left index and exclude the right.

Time: $\mathcal{O}(N)$

d41d8c, 13 lines

```

array<vi, 2> manacher(const string& s) {
    int n = sz(s);
    array<vi, 2> p = {vi(n+1), vi(n)};
    rep(z,0,2) for (int i=0, l=0, r=0; i < n; i++) {
        int t = r - i + 1;
        if (i < r) p[z][i] = min(t, p[z][l+t]);
        int L = i - p[z][i], R = i + p[z][i] - 1;
        while (L >= 1 && R + 1 < n && s[L-1] == s[R+1])
            p[z][i]++, L--, R++;
        if (R > r) l=L, r=R;
    }
    return p;
}

```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.

Usage: `rotate(v.begin(), v.begin()+minRotation(v), v.end());`

Time: $\mathcal{O}(N)$

d41d8c, 8 lines

```

int minRotation(string s) {
    int a=0, N=sz(s); s += s;
    rep(b,0,N) rep(k,0,N) {
        if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1); break;}
        if (s[a+k] > s[b+k]) {a = b; break;}
    }
    return a;
}

```

SuffixArray.h

Description: Builds suffix array for a string. `sa[i]` is the starting index of the suffix which is i 'th in the sorted suffix array. The returned vector is of size $n + 1$, and `sa[0] = n`. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: `lcp[i] = lcp(sa[i], sa[i-1])`, `lcp[0] = 0`. Also stores rank (inverse SA) and an RMQ table on LCP so `getLCP(i, j) / cmpSubstr` are $O(1)$. The input string must not contain any nul chars. Bytes are ordered as unsigned; `lim` must exceed all bytes. `rmq(l, r)` queries LCP indices $[l, r]$, both endpoints included. `cmpSubstr(a, lena, b, lenb, s)` compares $[a, a + lena)$ with $[b, b + lenb)$: starts included, ends excluded.

Memory: $O(n \log n + \text{lim})$

Time: $O((n + \text{lim}) \log n)$ build

d41d8c, 57 lines

```
struct SuffixArray {
    vi sa, lcp, rank, lg;
    vector<vi> st;
    SuffixArray(string s, int lim = 256) {
        s.push_back(0); int n = sz(s), k = 0, a, b;
        vi x(n), y(n), ws(max(n, lim));
        rep(i, 0, n) x[i] = (unsigned char)s[i];
        sa = lcp = y, iota(all(sa), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
            p = j, iota(all(y), n - j);
            rep(i, 0, n) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i, 0, n) ws[x[i]]++;
            rep(i, 1, lim) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[i]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            rep(i, 1, n) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
        }
        for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
            for (k && k--, j = sa[x[i] - 1];
                 s[i + k] == s[j + k]; k++);
        rank = x;
        buildRMQ();
    }
    void buildRMQ() {
        int n = sz(lcp);
        lg.assign(n + 1, 0);
        rep(i, 2, n+1) lg[i] = lg[i / 2] + 1;
        st.assign(lg[n] + 1, vi(n));
        st[0] = lcp;
        rep(k, 1, sz(st)) rep(i, 0, n - (1 << k) + 1)
            st[k][i] = min(st[k-1][i], st[k-1][i + (1<<(k-1))]);
    }
    int rmq(int l, int r) { // inclusive
        if (l > r) return INT_MAX;
        int k = lg[r - l + 1];
        return min(st[k][l], st[k][r - (1 << k) + 1]);
    }
    int getLCP(int i, int j) {
        if (i == j) return sz(sa) - 1 - i; // ignore sentinel
        int ri = rank[i], rj = rank[j];
        if (ri > rj) swap(ri, rj);
        return rmq(ri + 1, rj);
    }
}
// compare s[a..a+lena) vs s[b..b+lenb); -1 / 0 / 1
int cmpSubstr(int a, int lena, int b, int lenb,
    const string& s) {
    int common = min(getLCP(a, b), min(lena, lenb));
    if (common == min(lena, lenb)) {
        if (lena == lenb) return 0;
        return lena < lenb ? -1 : 1;
    }
}
```

```
return (unsigned char)s[a + common] <
(unsigned char)s[b + common] ? -1 : 1;
}
};
```

SuffixTree.h

Description: Ukkonen's algorithm for online suffix tree construction. Each node contains indices $[l, r)$ into the string, and a list of child nodes. Suffixes are given by traversals of this tree, joining $[l, r)$ substrings. Each edge includes index l and excludes index r . The root is 0 (has $l = -1, r = 0$), non-existent children are -1. To get a complete tree, append a dummy symbol – otherwise it may contain an incomplete path (still useful for substring matching, though).

Time: $O(26N)$

d41d8c, 50 lines

```
struct SuffixTree {
    enum { N = 200010, ALPHA = 26 }; // N ~ 2*maxlen+10
    int toi(char c) { return c - 'a'; }
    string a; // v = cur node, q = cur position
    int t[N][ALPHA], l[N], r[N], p[N], s[N], v=0, q=0, m=2;

    void ukkadd(int i, int c) { suff:
        if (r[v] <= q) {
            if (t[v][c] == -1) { t[v][c] = m; l[m] = i;
                p[m++] = v; v = s[v]; q = r[v]; goto suff; }
            v = t[v][c]; q = l[v];
        }
        if (q == -1 || c == toi(a[q])) q++; else {
            l[m+1] = i; p[m+1] = m; l[m] = l[v]; r[m] = q;
            p[m] = p[v]; t[m][c] = m+1; t[m][toi(a[q])] = v;
            l[v] = q; p[v] = m; t[p[m]][toi(a[l[m]])] = m;
            v = s[p[m]]; q = l[m];
            while (q < r[m]) { v = t[v][toi(a[q])]; q += r[v] - l[v]; }
            if (q == r[m]) s[m] = v; else s[m] = m+2;
            q = r[v] - (q - r[m]); m += 2; goto suff;
        }
    }

    SuffixTree(string a) : a(a) {
        fill(r, r+N, sz(a));
        memset(s, 0, sizeof s);
        memset(t, -1, sizeof t);
        fill(t[1], t[1]+ALPHA, 0);
        s[0] = 1; l[0] = l[1] = -1; r[0] = r[1] = p[0] = p[1] = 0;
        rep(i, 0, sz(a)) ukkadd(i, toi(a[i]));
    }

    // example: find longest common substring (uses ALPHA = 28)
    pii best;
    int lcs(int node, int i1, int i2, int olen) {
        if (l[node] <= i1 && i1 < r[node]) return 1;
        if (l[node] <= i2 && i2 < r[node]) return 2;
        int mask = 0, len = node ? olen + (r[node] - l[node]) : 0;
        rep(c, 0, ALPHA) if (t[node][c] != -1)
            mask |= lcs(t[node][c], i1, i2, len);
        if (mask == 3)
            best = max(best, {len, r[node] - l[node]});
        return mask;
    }
    static pii LCS(string s, string t) {
        SuffixTree st(s + (char)('z' + 1) + t + (char)('z' + 2));
        st.lcs(0, sz(s), sz(s) + 1 + sz(t), 0);
        return st.best;
    }
}
};
```

Hashing.h

Description: Self-explanatory methods for string hashing. `hashInterval(a, b)` hashes $[a, b)$: a included, b excluded. `getHashes(s, len)[i]` hashes $[i, i + len)$, excluding $i + len$.

d41d8c, 44 lines

```
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on evil test data (e.g. Thue-Morse, where
// ABBA... and BAAB... of length 2^10 hash the same mod 2^64).
// "typedef ull H;" instead if you think test data is random,
// or work mod 10^9+7 if the Birthday paradox is not a problem.
typedef uint64_t ull;
struct H {
    ull x; H(ull x=0) : x(x) {}
    H operator+(H o) { return x + o.x + (x + o.x < x); }
    H operator-(H o) { return *this + ~o.x; }
    H operator*(H o) { auto m = (__uint128_t)x * o.x;
        return H((ull)m) + (ull)(m >> 64); }
    ull get() const { return x + 1; }
    bool operator==(H o) const { return get() == o.get(); }
    bool operator<(H o) const { return get() < o.get(); }
};
static const H C = (ll)1e11+3; // (order ~ 3e9; random also ok)

struct HashInterval {
    vector<H> ha, pw;
    HashInterval(string& str) : ha(sz(str)+1), pw(ha) {
        pw[0] = 1;
        rep(i, 0, sz(str))
            ha[i+1] = ha[i] * C + str[i],
            pw[i+1] = pw[i] * C;
    }
    H hashInterval(int a, int b) { // hash [a, b)
        return ha[b] - ha[a] * pw[b - a];
    }
};

vector<H> getHashes(string& str, int length) {
    if (sz(str) < length) return {};
    H h = 0, pw = 1;
    rep(i, 0, length)
        h = h * C + str[i], pw = pw * C;
    vector<H> ret = {h};
    rep(i, length, sz(str)) {
        ret.push_back(h = h * C + str[i] - pw * str[i-length]);
    }
    return ret;
}
```

H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return h;}

Hashing-codeforces.h

Description: Various self-explanatory methods for string hashing. Use on Codeforces, which lacks 64-bit support and where solutions can be hacked. `hashInterval(a, b)` hashes $[a, b)$: a included, b excluded. `getHashes(s, len)[i]` hashes $[i, i + len)$, excluding $i + len$.

<=sys/time.h> d41d8c, 52 lines

```
typedef uint64_t ull;
static int C; // initialized below

// Arithmetic mod two primes and 2^32 simultaneously.
// "typedef uint64_t H;" instead if Thue-Morse does not apply.
template<int M, class B>
struct A {
    int x; B b; A(int x=0) : x(x), b(b) {}
    A(int x, B b) : x(x), b(b) {}
};
```

```

A operator+(A o){int y = x+o.x; return{y - (y>=M)*M, b+o.b};}
A operator-(A o){int y = x-o.x; return{y + (y< 0)*M, b-o.b};}
A operator*(A o) { return {(int)(1LL*x*o.x % M), b*o.b}; }
explicit operator ull() const { return x ^ (ull) b << 21; }
bool operator==(A o) const { return (ull)*this == (ull)o; }
bool operator<(A o) const { return (ull)*this < (ull)o; }
};
typedef A<1000000007, A<1000000009, unsigned>> H;

struct HashInterval {
    vector<H> ha, pw;
    HashInterval(string& str) : ha(sz(str)+1), pw(ha) {
        pw[0] = 1;
        rep(i,0,sz(str))
            ha[i+1] = ha[i] * C + str[i],
            pw[i+1] = pw[i] * C;
    }
    H hashInterval(int a, int b) { // hash [a, b]
        return ha[b] - ha[a] * pw[b - a];
    }
};

vector<H> getHashes(string& str, int length) {
    if (sz(str) < length) return {};
    H h = 0, pw = 1;
    rep(i,0,length)
        h = h * C + str[i], pw = pw * C;
    vector<H> ret = {h};
    rep(i,length,sz(str)) {
        ret.push_back(h = h * C + str[i] - pw * str[i-length]);
    }
    return ret;
}

H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return h;}

int main() {
    timeval tp;
    gettimeofday(&tp, 0);
    C = (int)tp.tv_usec; // (less than modulo)
    assert((ull)(H(1)*2+1-3) == 0);
    // ...
}

```

AhoCorasick.h

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(pat, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries.

Time: construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$.

d41d8c, 66 lines

```

struct AhoCorasick {
    enum {alpha = 26, first = 'A'}; // change this!
    struct Node {
        // (nmatches is optional)
        int back, next[alpha], start = -1, end = -1, nmatches = 0;
        Node(int v) { memset(next, v, sizeof(next)); }
    };
    vector<Node> N;

```

AhoCorasick Point lineDistance SegmentDistance

```

vi backp;
void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
        int& m = N[n].next[c - first];
        if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
        else n = m;
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
}

AhoCorasick(vector<string&> pat) : N(1, -1) {
    rep(i,0,sz(pat)) insert(pat[i], i);
    N[0].back = sz(N);
    N.emplace_back(0);

    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
        int n = q.front(), prev = N[n].back;
        rep(i,0,alpha) {
            int &ed = N[n].next[i], y = N[prev].next[i];
            if (ed == -1) ed = y;
            else {
                N[ed].back = y;
                (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
                    = N[y].end;
                N[ed].nmatches += N[y].nmatches;
                q.push(ed);
            }
        }
    }
}

vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
        n = N[n].next[c - first];
        res.push_back(N[n].end);
        // count += N[n].nmatches;
    }
    return res;
}

vector<vi> findAll(vector<string&> pat, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i,0,sz(word)) {
        int ind = r[i];
        while (ind != -1) {
            res[i - sz(pat[ind]) + 1].push_back(ind);
            ind = backp[ind];
        }
    }
    return res;
}
};

```

Geometry (9)

Geometry uses double coordinates: pp = complex<dd> with dd = double. Use real() and imag() for coordinates, and free dotp, crossp, and orient for predicates. Three-dimensional routines use Point3D.

9.1 Geometric primitives

Point.h

Description: Complex points: $x = \text{real}()$, $y = \text{imag}()$. All planar geometry uses pp = complex<double>. orient(a,b,c) is positive for a left turn. norm(p) is squared length; abs(p) is length, arg(p) is angle, p/abs(p) is a unit vector (nonzero points only). Sort with PointLess{}; complex has no ordering.

Usage: pp a(3,5), rotated = a * polar(1.0, acos(-1.0)/3);

d41d8c, 23 lines

```

typedef double dd;
typedef complex<dd> pp;

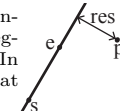
int sgn(double x) { return (x > 0) - (x < 0); }
double dotp(pp a, pp b) {
    return a.real()*b.real() + a.imag()*b.imag();
}
double crossp(pp a, pp b) {
    return a.real()*b.imag() - a.imag()*b.real();
}
double orient(pp a, pp b, pp c) {
    return crossp(b-a, c-a);
}
pp perp(pp p) {
    return {-p.imag(), p.real()};
}
double dist(pp p) { return abs(p); }
struct PointLess {
    bool operator()(pp a, pp b) const {
        return make_pair(a.real(), a.imag()) <
            make_pair(b.real(), b.imag());
    }
};

```

lineDistance.h

Description:

Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. In 3D use (b-a).cross(p-a).dist()/(b-a).dist() with Point3D; that distance is non-negative.



"Point.h"

d41d8c, 3 lines

```

double lineDist(pp a, pp b, pp p) {
    return crossp(b-a, p-a)/dist(b-a);
}

```

SegmentDistance.h

Description:

Returns the shortest distance between point p and the line segment from point s to e. Both segment endpoints s and e are included.

Usage: complex<double> a, b(2,2), p(1,1);

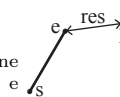
bool onSegment = segDist(a,b,p) < 1e-10;

d41d8c, 5 lines

```

double segDist(pp s, pp e, pp p) {
    if (s==e) return abs(p-s);
    auto d = norm(e-s), t = min(d,max(.0,dotp(p-s, e-s)));
    return abs((p-s)*d-(e-s)*t)/d;
}

```



}

SegmentIntersection.h

Description:

If a unique intersection point between the line segments going from s_1 to e_1 and from s_2 to e_2 exists then it is returned. Both endpoints of each segment are included. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment.

Usage: vector<pp> inter = segInter(s_1, e_1, s_2, e_2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;

"Point.h", "OnSegment.h" d41d8c, 13 lines

```
vector<pp> segInter(pp a, pp b, pp c, pp d) {
    auto oa = orient(c, d, a), ob = orient(c, d, b),
          oc = orient(a, b, c), od = orient(a, b, d);
    // Checks if intersection is single non-endpoint point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<pp, PointLess> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
    if (onSegment(a, b, d)) s.insert(d);
    return {all(s)};
}
```

lineIntersection.h

Description:

If a unique intersection point of the lines going through s_1, e_1 and s_2, e_2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned.

Usage: auto res = lineInter(s_1, e_1, s_2, e_2);
if (res.first == 1)
cout << "intersection point at " << res.second << endl;

"Point.h" d41d8c, 7 lines

```
pair<int, pp> lineInter(pp s1, pp e1, pp s2, pp e2) {
    auto d = crossp(e1 - s1, e2 - s2);
    if (d == 0) // if parallel
        return {-(orient(s1, e1, s2) == 0), pp(0, 0)};
    auto p = orient(s2, e1, e2), q = orient(s2, e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
```

sideOf.h

Description: Returns where p is as seen from s towards e . $1/0/-1 \Rightarrow$ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line.

Usage: bool left = sideOf(p_1, p_2, q)==1;

"Point.h" d41d8c, 7 lines

```
int sideOf(pp s, pp e, pp p) { return sgn(orient(s, e, p)); }
```

```
int sideOf(pp s, pp e, pp p, double eps) {
```

```
    auto a = crossp(e-s, p-s);
    double l = dist(e-s)*eps;
    return (a > l) - (a < -l);
}
```

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e . Both segment endpoints s and e are included. Use (segDist(s, e, p)<=epsilon) for an epsilon check.

"Point.h" d41d8c, 3 lines

```
bool onSegment(pp s, pp e, pp p) {
    return orient(p, s, e) == 0 && dotp(s - p, e - p) <= 0;
}
```

linearTransformation.h

Description:

Apply the linear transformation (translation, rotation and scaling) which takes line p_0-p_1 to line q_0-q_1 to point r .

"Point.h" d41d8c, 3 lines

```
pp linearTransformation(pp p0, pp p1, pp q0, pp q1, pp r) {
    return q0 + (r-p0) * (q1-q0) / (p1-p0);
}
```

LineProjectionReflection.h

Description: Projects point p onto line ab . Set refl=true to get reflection of point p across line ab instead.

"Point.h" d41d8c, 5 lines

```
pp lineProj(pp a, pp b, pp p, bool refl=false) {
    pp v = b - a;
    auto d = crossp(v, p-a) * (1+refl);
    return p - perp(v)*d/norm(v);
}
```

Angle.h

Description: A class for ordering angles (as represented by complex points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage: vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }
// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i
Never compare a zero direction.

"Point.h" d41d8c, 38 lines

```
struct Angle {
    pp p;
    int t;
    Angle(pp p, int t=0) : p(p), t(t) {}
    Angle operator-(Angle b) const { return {p-b.p, t}; }
    int half() const {
        assert(p != pp(0));
        return p.imag() < 0 || (p.imag() == 0 && p.real() < 0);
    }
    Angle t90() const {
        return {perp(p), t + (half() && p.real() >= 0)};
    }
    Angle t180() const { return {-p, t + half(); }
    Angle t360() const { return {p, t + 1}; }
};
```

```
bool operator<(Angle a, Angle b) {
    // Add norm(a.p), norm(b.p) to break ties by distance.
    if (a.t != b.t) return a.t < b.t;
    if (a.half() != b.half()) return a.half() < b.half();
    return crossp(a.p, b.p) > 0;
}
```

// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.

```
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
    if (b < a) swap(a, b);
    return (b < a.t180() ?
        make_pair(a, b) : make_pair(b, a.t360()));
}
Angle operator+(Angle a, Angle b) { // point a + vector b
    Angle r(a.p + b.p, a.t);
    if (a.t180() < r) r.t--;
    return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
    int tu = b.t - a.t; a.t = b.t;
    return {{dotp(a.p, b.p), crossp(a.p, b.p)}, tu - (b < a)};
}
```

9.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h" d41d8c, 10 lines

```
bool circleInter(pp a, pp b, double r1, double r2, pair<pp, pp*> out) {
    if (a == b) { assert(r1 != r2); return false; }
    pp vec = b - a;
    double d2 = norm(vec), sum = r1+r2, dif = r1-r2,
           p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
    if (sum*sum < d2 || dif*dif > d2) return false;
    pp mid = a + vec*p, per = perp(vec) * sqrt(fmax(0, h2) / d2);
    *out = {mid + per, mid - per};
    return true;
}
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r_2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r_2 to 0.

"Point.h" d41d8c, 12 lines

```
vector<pair<pp, pp*>> tangents(pp c1, double r1, pp c2, double r2) {
    pp d = c2 - c1;
    double dr = r1 - r2, d2 = norm(d), h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0) return {};
    vector<pair<pp, pp*>> out;
    for (double sign : {-1, 1}) {
        pp v = (d * dr + perp(d) * sqrt(h2) * sign) / d2;
        out.push_back({c1 + v * r1, c2 + v * r2});
    }
    if (h2 == 0) out.pop_back();
    return out;
}
```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points.

"Point.h" d41d8c, 8 lines

```
vector<pp> circleLine(pp c, double r, pp a, pp b) {
    pp ab = b - a, p = a + ab * dotp(c-a, ab) / norm(ab);
    double s = orient(a, b, c), h2 = r*r - s*s / norm(ab);
    if (h2 < 0) return {};
    if (h2 == 0) return {p};
    pp h = ab / abs(ab) * sqrt(h2);
```

```
    return {p - h, p + h};
}
```

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.

Time: $\mathcal{O}(n)$

"../content/geometry/Point.h" d41d8c, 22 lines

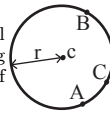
```
double angleBetween(pp p, pp q) {
    return atan2(crossp(p, q), dotp(p, q));
}

double circlePoly(pp c, double r, vector<pp> ps) {
    auto tri = [&](pp p, pp q) {
        auto r2 = r * r / 2;
        pp d = q - p;
        if (p == q) return 0.0;
        auto a = dotp(d, p)/norm(d), b = (norm(p)-r*r)/norm(d);
        auto det = a * a - b;
        if (det <= 0) return angleBetween(p, q) * r2;
        auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
        if (t < 0 || 1 <= s) return angleBetween(p, q) * r2;
        pp u = p + d * s, v = q + d * (t-1);
        return angleBetween(p,u) * r2 + crossp(u,v)/2
            + angleBetween(v,q) * r2;
    };
    auto sum = 0.0;
    rep(i,0,sz(ps))
        sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
    return sum;
}
```

circumcircle.h

Description:

The circumcircle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



"Point.h" d41d8c, 8 lines

```
double ccRadius(pp A, pp B, pp C) {
    return abs(B-A)*abs(C-B)*abs(A-C)/
        abs(crossp(B-A, C-A))/2.0;
}

pp ccCenter(pp A, pp B, pp C) {
    pp b = C-A, c = B-A;
    return A + perp(b*norm(c)-c*norm(b))/crossp(b, c)/2.0;
}
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.

Time: expected $\mathcal{O}(n)$

"circumcircle.h" d41d8c, 18 lines

```
pair<pp, double> mec(vector<pp> ps) {
    if (ps.empty()) return {pp(), 0};
    shuffle(all(ps), mt19937(time(0)));
    pp o = ps[0];
    double r = 0, EPS = 1 + 1e-8;
    rep(i,0,sz(ps)) if (abs(o - ps[i]) > r * EPS) {
        o = ps[i], r = 0;
        rep(j,0,i) if (abs(o - ps[j]) > r * EPS) {
            o = (ps[i] + ps[j]) / 2.0;
            r = abs(o - ps[i]);
            rep(k,0,j) if (abs(o - ps[k]) > r * EPS) {
```

```
                o = ccCenter(ps[i], ps[j], ps[k]);
                r = abs(o - ps[i]);
            }
        }
    }
    return {o, r};
}
```

9.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary.

Usage: vector<pp> v = {pp{4,4}, pp{1,2}, pp{2,1}};

bool in = inPolygon(v, pp{3, 3}, false);

Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h" d41d8c, 11 lines

```
bool inPolygon(vector<pp> &p, pp a, bool strict = true) {
    int cnt = 0, n = sz(p);
    rep(i,0,n) {
        pp q = p[(i + 1) % n];
        if (onSegment(p[i], q, a)) return !strict;
        //or: if (segDist(p[i], q, a) <= eps) return !strict;
        int above = (a.imag()<p[i].imag()) - (a.imag()<q.imag());
        cnt ^= above * orient(a, p[i], q) > 0;
    }
    return cnt;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Uses coordinates relative to the first vertex.

"Point.h" d41d8c, 6 lines

```
double polygonArea2(const vector<pp>& v) {
    if (v.empty()) return 0;
    double a = 0;
    rep(i,1,sz(v)-1) a += crossp(v[i]-v[0], v[i+1]-v[0]);
    return a;
}
```

PolygonCenter.h

Description: Returns the center of mass for a polygon with nonzero signed area. Uses local coordinates to avoid cancellation when far from the origin.

Time: $\mathcal{O}(n)$

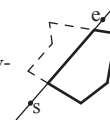
"Point.h" d41d8c, 10 lines

```
pp polygonCenter(const vector<pp>& v) {
    pp res(0, 0); double A = 0;
    rep(i,1,sz(v)-1) {
        pp p = v[i]-v[0], q = v[i+1]-v[0];
        double a = crossp(p, q);
        res += (p + q) * a;
        A += a;
    }
    return v[0] + res / A / 3.0;
}
```

PolygonCut.h

Description:

Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.



Usage: vector<pp> p = ...;

p = polygonCut(p, pp(0,0), pp(1,0));

"Point.h" d41d8c, 12 lines

```
vector<pp> polygonCut(const vector<pp>& poly, pp s, pp e) {
    vector<pp> res;
    rep(i,0,sz(poly)) {
        pp cur = poly[i], prev = i ? poly[i-1] : poly.back();
        auto a = orient(s, e, cur), b = orient(s, e, prev);
        if ((a < 0) != (b < 0))
            res.push_back(cur + (prev - cur) * (a / (a - b)));
        if (a < 0)
            res.push_back(cur);
    }
    return res;
}
```

HalfplaneIntersection.h

Description: Intersection of half-planes, each the left side of directed line $s \rightarrow e$ (including the boundary). Adds a bounding box of side $2 \cdot 10^9$. Returns vertices of the bounded convex polygon in order, or empty if the intersection is empty or has zero area. Requires $s \neq e$. HP_EPS is a distance tolerance.

Time: $\mathcal{O}(N \log N)$

"Point.h", "PolygonArea.h" d41d8c, 59 lines

const double HP_EPS = 1e-9, HP_INF = 1e9;

```
struct HP {
    pp s, e, d;
    double ang;
    HP() {}
    HP(pp a, pp b) : s(a), e(b), d((b-a)/abs(b-a)),
        ang(arg(d)) {}
    bool out(pp p) { return crossp(d, p - s) < -HP_EPS; }
    bool operator<(HP o) const { return ang < o.ang; }
};
```

```
pp hpI(HP a, HP b) {
    return a.s + a.d * (crossp(b.s-a.s, b.d) /
        crossp(a.d, b.d));
}
```

```
vector<pp> halfPlaneInter(vector<HP> h) {
    pp box[] = {pp(HP_INF, HP_INF), pp(-HP_INF, HP_INF),
        pp(-HP_INF, -HP_INF), pp(HP_INF, -HP_INF)};
    rep(i,0,4) h.push_back(HP(box[i], box[(i+1)%4]));
    sort(all(h));
    deque<HP> dq;
    for (HP L : h) {
        while (sz(dq) > 1 &&
            L.out(hpI(dq.back(), dq[sz(dq)-2])))
            dq.pop_back();
        while (sz(dq) > 1 && L.out(hpI(dq[0], dq[1])))
            dq.pop_front();
        if (!dq.empty() &&
            (L.d == dq.back().d || L.d == -dq.back().d ||
            crossp(L.d, dq.back().d) == 0)) {
            if (dotp(L.d, dq.back().d) < 0) return {};
            if (L.out(dq.back().s)) dq.pop_back();
            else continue;
        }
        dq.push_back(L);
    }
    while (sz(dq) > 2 &&
        dq[0].out(hpI(dq.back(), dq[sz(dq)-2])))
        dq.pop_back();
}
```

```

while (sz(dq) > 2 &&
      dq.back().out(hpI(dq[0], dq[1])))
  dq.pop_front();
if (sz(dq) < 3) return {};
vector<pp> res;
rep(i,0,sz(dq))
  res.push_back(hpI(dq[i], dq[(i+1)%sz(dq)]));
vector<pp> out;
for (pp p : res) {
  if (out.empty() || abs(p - out.back()) > HP_EPS)
    out.push_back(p);
}
if (sz(out) >= 2 && abs(out[0]-out.back()) <= HP_EPS)
  out.pop_back();
if (sz(out) < 3) return {};
if (polygonArea2(out) == 0) return {};
return out;
}

```

PolygonUnion.h

Description: Calculates the area of the union of n polygons (not necessarily convex). The points within each polygon must be given in CCW order. (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be needed.)

Time: $\mathcal{O}(N^2)$, where N is the total number of points

```

"Point.h", "sideOf.h" d41d8c, 34 lines
double rat(pp a, pp b) {
  return b.real() ? a.real()/b.real() : a.imag()/b.imag();
}
double polyUnion(vector<vector<pp>&& poly) {
  double ret = 0;
  rep(i,0,sz(poly)) rep(v,0,sz(poly[i])) {
    pp A = poly[i][v], B = poly[i][(v+1)%sz(poly[i])];
    vector<pair<double, int>> segs = {{0, 0}, {1, 0}};
    rep(j,0,sz(poly)) if (i != j) {
      rep(u,0,sz(poly[j])) {
        pp C = poly[j][u], D = poly[j][(u+1)%sz(poly[j])];
        int sc = sideOf(A, B, C), sd = sideOf(A, B, D);
        if (sc != sd) {
          double sa = orient(C, D, A), sb = orient(C, D, B);
          if (min(sc, sd) < 0)
            segs.emplace_back(sa / (sa - sb), sgn(sc - sd));
        } else if (!sc && !sd && j < i && sgn(dotp(B-A, D-C)) > 0) {
          segs.emplace_back(rat(C - A, B - A), 1);
          segs.emplace_back(rat(D - A, B - A), -1);
        }
      }
    }
  }
  sort(all(segs));
  for (auto& s : segs) s.first = min(max(s.first, 0.0), 1.0);
  double sum = 0;
  int cnt = segs[0].second;
  rep(j,1,sz(segs)) {
    if (!cnt) sum += segs[j].first - segs[j-1].first;
    cnt += segs[j].second;
  }
  ret += crossp(A, B) * sum;
}
return ret / 2;
}

```

ConvexHull.h

Description:

Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.

Time: $\mathcal{O}(n \log n)$

```

"Point.h" d41d8c, 12 lines
vector<pp> convexHull(vector<pp> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts), PointLess{});
  vector<pp> h(sz(pts)+1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (pp p : pts) {
      while (t >= s + 2 && orient(h[t-2], h[t-1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}

```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).

Time: $\mathcal{O}(n)$

```

"Point.h" d41d8c, 14 lines
array<pp, 2> hullDiameter(vector<pp> S) {
  assert(!S.empty());
  int n = sz(S), j = n < 2 ? 0 : 1;
  auto best = norm(S[0]-S[0]);
  array<pp, 2> res{S[0], S[0]};
  rep(i,0,j)
    for (; j = (j+1)%n) {
      if (auto d2 = norm(S[i]-S[j]); d2 > best)
        best = d2, res = {S[i], S[j]};
      if (crossp(S[(j+1)%n]-S[j], S[i+1]-S[i]) >= 0)
        break;
    }
  return res;
}

```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

Time: $\mathcal{O}(\log N)$

```

"Point.h", "sideOf.h", "OnSegment.h" d41d8c, 13 lines
bool inHull(const vector<pp>& l, pp p, bool strict = true) {
  if (l.empty()) return false;
  int a = 1, b = sz(l) - 1, r = !strict;
  if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
  if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
  if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p) <= -r)
    return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
  }
  return sgn(orient(l[a], l[b], p)) < r;
}

```

LineHullIntersection.h



Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $(-1, -1)$ if no collision; $(i, -1)$ if touching the corner i ; (i, i) if along side $(i, i+1)$; (i, j) if crossing sides $(i, i+1)$ and $(j, j+1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i, i+1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.

Time: $\mathcal{O}(\log n)$

```

"Point.h" d41d8c, 38 lines
#define cmp(i,j) sgn(crossp(perp(dir), poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i+1, i) >= 0 && cmp(i, i-1+n) < 0
int extrVertex(vector<pp>& poly, pp dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo+1, lo), ms = cmp(m+1, m);
    (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
  }
  return lo;
}
#define cmpl(i) sgn(orient(a, poly[i], b))
array<int, 2> lineHull(pp a, pp b, vector<pp>& poly) {
  int endA = extrVertex(poly, perp(a - b));
  int endB = extrVertex(poly, perp(b - a));
  if (cmpl(endA) < 0 || cmpl(endB) > 0)
    return {-1, -1};
  array<int, 2> res;
  rep(i,0,2) {
    int lo = endB, hi = endA, n = sz(poly);
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
      (cmpl(m) == cmpl(endB) ? lo : hi) = m;
    }
    res[i] = (lo + !cmpl(hi)) % n;
    swap(endA, endB);
  }
  if (res[0] == res[1]) return {res[0], -1};
  if (!cmpl(res[0]) && !cmpl(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
      case 0: return {res[0], res[0]};
      case 2: return {res[1], res[1]};
    }
  return res;
}

```

9.4 Misc. Point Set Problems

ClosestPair.h

Description: Closest pair by divide and conquer. Requires at least two finite points; squared distances must not overflow or underflow. Supports fractional coordinates.

Memory: $\mathcal{O}(n)$

Time: $\mathcal{O}(n \log n)$

```

"Point.h" d41d8c, 40 lines
pair<pp, pp> closest(vector<pp> v) {
  assert(sz(v) > 1);
  sort(all(v), PointLess{});
  vector<pp> tmp(sz(v));
  double best = INFINITY;
  pair<pp, pp> ret{v[0], v[1]};
  auto upd = [&](pp a, pp b) {

```

```

    if (double d = norm(a - b); d < best)
        best = d, ret = {a, b};
};

auto byY = [](pp a, pp b) { return a.imag() < b.imag(); };
auto rec = [&](auto&& self, int l, int r) -> void {
    if (best == 0) return;
    if (r - l <= 3) {
        rep(i, l, r) rep(j, i+1, r) upd(v[i], v[j]);
        sort(v.begin() + l, v.begin() + r, byY);
        return;
    }
    int m = (l + r) / 2;
    double x = v[m].real();
    self(self, l, m), self(self, m, r);
    if (best == 0) return;
    auto a = v.begin() + l, b = v.begin() + m;
    auto c = v.begin() + r;
    merge(a, b, b, c, tmp.begin(), byY);
    copy_n(tmp.begin(), r - l, a);
    int k = 0;
    rep(i, l, r) {
        double dx = v[i].real() - x;
        if (dx * dx < best) tmp[k++] = v[i];
    }
    rep(i, 0, k) rep(j, i+1, k) {
        double dy = tmp[j].imag() - tmp[i].imag();
        if (dy * dy >= best) break;
        upd(tmp[i], tmp[j]);
    }
};
rec(rec, 0, sz(v));
return ret;
}

```

ManhattanMST.h

Description: Given N points, returns up to 4*N edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights $w(p, q) = |p.x - q.x| + |p.y - q.y|$. Edges are MEdge{distance, src, dst}. Use a standard MST algorithm on the result to find the final MST.

Time: $\mathcal{O}(N \log N)$

```

"Point.h"
d41d8c, 25 lines

struct MEdge { double d; int a, b; };
vector<MEdge> manhattanMST(vector<pp> ps) {
    vi id(sz(ps));
    iota(all(id), 0);
    vector<MEdge> edges;
    rep(k, 0, 4) {
        sort(all(id), [&](int i, int j) {
            return (ps[i]-ps[j]).real() < (ps[j]-ps[i]).imag();});
        map<double, int> sweep;
        for (int i : id) {
            for (auto it = sweep.lower_bound(-ps[i].imag());
                 it != sweep.end(); sweep.erase(it++)) {
                int j = it->second;
                pp d = ps[i] - ps[j];
                if (d.imag() > d.real()) break;
                edges.push_back({d.imag() + d.real(), i, j});
            }
            sweep[-ps[i].imag()] = i;
        }
        for (pp& p : ps)
            p = k & 1 ? pp(-p.real(), p.imag())
                : pp(p.imag(), p.real());
    }
}

```

```

    return edges;
}

```

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

```

"Point.h"
d41d8c, 64 lines

typedef double T;
const T INF = numeric_limits<T>::max();

bool on_x(pp a, pp b) { return a.real() < b.real(); }
bool on_y(pp a, pp b) { return a.imag() < b.imag(); }

struct Node {
    pp pt; // if this is a leaf, the single point in it
    T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
    Node *first = 0, *second = 0;

    T distance(pp p) { // min squared distance to a point
        T x = (p.real() < x0 ? x0 : p.real() > x1 ? x1 : p.real());
        T y = (p.imag() < y0 ? y0 : p.imag() > y1 ? y1 : p.imag());
        return norm(pp(x, y) - p);
    }

    Node(vector<pp&& vp) : pt(vp[0]) {
        for (pp p : vp) {
            x0 = min(x0, p.real()); x1 = max(x1, p.real());
            y0 = min(y0, p.imag()); y1 = max(y1, p.imag());
        }
        if (vp.size() > 1) {
            // split on x if width >= height (not ideal...)
            sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
            // divide by taking half the array for each child (not
            // best performance with many duplicates in the middle)
            int half = sz(vp)/2;
            first = new Node({vp.begin(), vp.begin() + half});
            second = new Node({vp.begin() + half, vp.end()});
        }
    }
};

struct KDTree {
    Node* root;
    KDTree(const vector<pp&& vp) : root(new Node({all(vp)})) {}

    pair<T, pp> search(Node *node, pp p) {
        if (!node->first) {
            // uncomment if we should not find the point itself:
            // if (p == node->pt) return {INF, pp()};
            return make_pair(norm(p - node->pt), node->pt);
        }

        Node *f = node->first, *s = node->second;
        T bfist = f->distance(p), bs = s->distance(p);
        if (bfist > bs) swap(bs, bfist), swap(f, s);

        // search closest side first, other side if needed
        auto best = search(f, p);
        if (bs < best.first) {
            auto other = search(s, p);
            if (other.first < best.first) best = other;
        }
        return best;
    }

    // find nearest point to a point, and its squared distance

```

```

// Requires a nonempty point set.
pair<T, pp> nearest(pp p) {
    return search(root, p);
}

```

DelaunayTriangulation.h

Description: Computes the Delaunay triangulation of a set of points. Each circumcircle contains none of the input points. If any three points are collinear or any four are on the same circle, behavior is undefined.

Time: $\mathcal{O}(n^2)$

```

"Point.h", "3dHull.h"
d41d8c, 10 lines

template<class F>
void delaunay(vector<pp>& ps, F trifun) {
    if (sz(ps) == 3) { int d = (orient(ps[0], ps[1], ps[2]) < 0);
        trifun(0, 1, 2, d); }
    vector<P3> p3;
    for (pp p : ps) p3.emplace_back(p.real(), p.imag(), norm(p));
    if (sz(p3) > 3) for (auto t:hull3d(p3)) if ((p3[t.b]-p3[t.a]).
        cross(p3[t.c]-p3[t.a]).dot(P3(0, 0, 1)) < 0)
        trifun(t.a, t.c, t.b);
}

```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Predicates may have precision issues. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ...}, all counter-clockwise.

Time: $\mathcal{O}(n \log n)$

```

"Point.h"
d41d8c, 89 lines

typedef struct Quad* Q;
typedef long double lll;
pp arb(DBL_MAX, DBL_MAX); // not equal to any other point

struct Quad {
    Q rot, o; pp p = arb; bool mark;
    pp& F() { return r()->p; }
    Q& r() { return rot->rot; }
    Q prev() { return rot->o->rot; }
    Q next() { return r()->prev(); }
} *H;

bool circ(pp p, pp a, pp b, pp c) { // is p in the circumcircle?
    lll p2 = norm(p), A = norm(a)-p2,
        B = norm(b)-p2, C = norm(c)-p2;
    return orient(p, a, b)*C + orient(p, b, c)*A
        + orient(p, c, a)*B > 0;
}

Q makeEdge(pp orig, pp dest) {
    Q r = H ? H : new Quad(new Quad(new Quad(new Quad{0})));
    H = r->o; r->r()->r() = r;
    rep(i, 0, 4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
    r->p = orig; r->F() = dest;
    return r;
}

void splice(Q a, Q b) {
    swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}

Q connect(Q a, Q b) {
    Q q = makeEdge(a->F(), b->p);
    splice(q, a->next());
    splice(q->r(), b);
    return q;
}

```

```

}

pair<Q,Q> rec(const vector<pp>& s) {
    if (sz(s) <= 3) {
        Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
        if (sz(s) == 2) return {a, a->r() };
        splice(a->r(), b);
        auto side = orient(s[0], s[1], s[2]);
        Q c = side ? connect(b, a) : 0;
        return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
    }

#define H(e) e->F(), e->p
#define valid(e) (orient(e->F(), H(base)) > 0)
    Q A, B, ra, rb;
    int half = sz(s) / 2;
    tie(ra, A) = rec({all(s) - half});
    tie(B, rb) = rec({sz(s) - half + all(s)});
    while ((orient(B->p, H(A)) < 0 && (A = A->next())) ||
        (orient(A->p, H(B)) > 0 && (B = B->r()->o)));
    Q base = connect(B->r(), A);
    if (A->p == ra->p) ra = base->r();
    if (B->p == rb->p) rb = base;

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
        Q t = e->dir; \
        splice(e, e->prev()); \
        splice(e->r(), e->r()->prev()); \
        e->o = H; H = e; e = t; \
    }
    for (;;) {
        DEL(LC, base->r(), o); DEL(RC, base, prev());
        if (!valid(LC) && !valid(RC)) break;
        if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
            base = connect(RC, base->r());
        else
            base = connect(base->r(), LC->r());
    }
    return { ra, rb };
}

vector<pp> triangulate(vector<pp> pts) {
    sort(all(pts), PointLess{});
    assert(unique(all(pts)) == pts.end());
    if (sz(pts) < 2) return {};
    Q e = rec(pts).first;
    vector<Q> q = {e};
    int qi = 0;
    while (orient(e->o->F(), e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
    q.push_back(c->r()); c = c->next(); } while (c != e); }
    ADD; pts.clear();
    while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
    return pts;
}

```

9.5 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

d41d8c, 6 lines

```

template<class V, class L>
double signedPolyVolume(const V& p, const L& trilst) {

```

PolyhedronVolume Point3D 3dHull sphericalDistance

```

double v = 0;
for (auto i : trilst) v += p[i.a].cross(p[i.b]).dot(p[i.c]);
return v / 6;
}

```

Point3D.h

Description: Handles points in 3D space using doubles. Angle ranges: phi in $[-\pi, \pi]$, theta in $[0, \pi]$; both endpoints included in each range.

d41d8c, 36 lines

```

struct Point3D {
    double x, y, z;
    explicit Point3D(double x=0, double y=0, double z=0) :
        x(x), y(y), z(z) {}
    bool operator<(Point3D p) const {
        return tie(x, y, z) < tie(p.x, p.y, p.z); }
    bool operator==(Point3D p) const {
        return tie(x, y, z) == tie(p.x, p.y, p.z); }
    Point3D operator+(Point3D p) const {
        return Point3D(x+p.x, y+p.y, z+p.z); }
    Point3D operator-(Point3D p) const {
        return Point3D(x-p.x, y-p.y, z-p.z); }
    Point3D operator*(double d) const {
        return Point3D(x*d, y*d, z*d); }
    Point3D operator/(double d) const {
        return Point3D(x/d, y/d, z/d); }
    double dot(Point3D p) const { return x*p.x + y*p.y + z*p.z; }
    Point3D cross(Point3D p) const {
        return Point3D(y*p.z-z*p.y, z*p.x-x*p.z, x*p.y-y*p.x);
    }
    double dist2() const { return x*x + y*y + z*z; }
    double dist() const { return sqrt(dist2()); }
    //Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
    double phi() const { return atan2(y, x); }
    //Zenith angle (latitude) to the z-axis in interval [0, pi]
    double theta() const { return atan2(sqrt(x*x+y*y),z); }
    Point3D unit() const { return *this/dist(); } //makes dist()=1
    //returns unit vector normal to *this and p
    Point3D normal(Point3D p) const { return cross(p).unit(); }
    //returns point rotated 'angle' radians ccw around axis
    Point3D rotate(double angle, Point3D axis) const {
        double s = sin(angle), c = cos(angle);
        Point3D u = axis.unit();
        return u*dot(u)*(1-c) + (*this)*c - cross(u)*s;
    }
};

```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.

Time: $\mathcal{O}(n^2)$

"Point3D.h"

d41d8c, 49 lines

```

typedef Point3D P3;

struct PR {
    void ins(int x) { (a == -1 ? a : b) = x; }
    void rem(int x) { (a == x ? a : b) = -1; }
    int cnt() { return (a != -1) + (b != -1); }
    int a, b;
};

```

```

struct F { P3 q; int a, b, c; };

```

```

vector<F> hull3d(const vector<P3>& A) {

```

```

assert(sz(A) >= 4);
vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
#define E(x,y) E[f.x][f.y]
vector<F> FS;
auto mf = [&](int i, int j, int k, int l) {
    P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
    if (q.dot(A[l]) > q.dot(A[i]))
        q = q * -1;
    F f{q, i, j, k};
    E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
    FS.push_back(f);
};
rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
    mf(i, j, k, 6 - i - j - k);

rep(i,4,sz(A)) {
    rep(j,0,sz(FS)) {
        F f = FS[j];
        if (f.q.dot(A[i]) > f.q.dot(A[f.a])) {
            E(a,b).rem(f.c);
            E(a,c).rem(f.b);
            E(b,c).rem(f.a);
            swap(FS[j--], FS.back());
            FS.pop_back();
        }
    }
    int nw = sz(FS);
    rep(j,0,nw) {
        F f = FS[j];
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
        C(a, b, c); C(a, c, b); C(b, c, a);
    }
    for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
        A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
    return FS;
};

```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. Converts to Cartesian unit vectors and uses $2 \operatorname{atan2}(|b-a|, |b+a|)$, stable also near coincident and antipodal points.

d41d8c, 13 lines

```

double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
    array<double,3> a = {sin(t1)*cos(f1),
        sin(t1)*sin(f1), cos(t1)};
    array<double,3> b = {sin(t2)*cos(f2),
        sin(t2)*sin(f2), cos(t2)};
    double d = 0, s = 0;
    rep(i,0,3) {
        d += (b[i]-a[i])*(b[i]-a[i]);
        s += (b[i]+a[i])*(b[i]+a[i]);
    }
    return radius*2*atan2(sqrt(d), sqrt(s));
}

```

Various (10)

EulerTourTree.h

Description: Unrooted dynamic forest, vertices $[0, N)$. Vertex indices include 0 and exclude N. Initially isolated with value 0. Link requires different components; cut requires an existing edge (either order). Point set/get and component update/query/size. Each tour stores one token per vertex and two zero-valued tokens per edge. Only vertex tokens contribute to sums. No path queries. Edge tokens are recycled. Change V, U, id, def, idU, binop, applyUpdate, mergeUpdate. Default is component add/sum. binop must be commutative and edge tokens must stay neutral. For a rooted subtree, cut its parent edge, operate on its component, then link back.

Memory: $\mathcal{O}(N)$

Time: Expected $\mathcal{O}(\log N)$ per operation.

d41d8c, 141 lines

```
struct EulerTourTree {
    using V = ll;
    using U = ll;
    static constexpr V id = 0;
    static constexpr V def = 0;
    static constexpr U idU = 0;
    V binop(V a, V b) { return a + b; }
    V applyUpdate(U u, V v, int len) { return v + u * len; }
    U mergeUpdate(U oldU, U nw) { return oldU + nw; }
    struct Node {
        int l = 0, r = 0, p = 0, c = 1;
        int own = 0, cnt = 0; // Vertex token / vertex count.
        unsigned y = 0;
        V val = id, agg = id;
        U lazy = idU;
    };
    vector<Node> t;
    vi spare;
    map<pii, pii> edges;
    mt19937 rng{712367};
    EulerTourTree(int n) : t(n + 1) {
        t.reserve(3*n + 1); spare.reserve(2*n);
        t[0].c = 0;
        rep(i, 1, n+1) {
            t[i].y = (unsigned)rng();
            t[i].own = t[i].cnt = 1;
            t[i].val = t[i].agg = def;
        }
    }
    void apply(int x, U u) {
        if (!x) return;
        t[x].val = applyUpdate(u, t[x].val, t[x].own);
        t[x].agg = applyUpdate(u, t[x].agg, t[x].cnt);
        t[x].lazy = mergeUpdate(t[x].lazy, u);
    }
    void push(int x) {
        if (t[x].lazy == idU) return;
        apply(t[x].l, t[x].lazy);
        apply(t[x].r, t[x].lazy);
        t[x].lazy = idU;
    }
    void pull(int x) {
        int l = t[x].l, r = t[x].r;
        t[x].c = 1 + t[l].c + t[r].c;
        t[x].cnt = t[x].own + t[l].cnt + t[r].cnt;
        t[x].agg = binop(binop(t[l].agg, t[x].val),
            t[r].agg);
        if (l) t[l].p = x;
        if (r) t[r].p = x;
    }
    int root(int x) {
        while (t[x].p) x = t[x].p;
        return x;
    }
};
```

```
}
int rank(int x) {
    int k = t[t[x].l].c;
    for (int p; (p = t[x].p); x = p)
        if (t[p].r == x) k += t[t[p].l].c + 1;
    return k;
}
int merge(int a, int b) {
    if (!a || !b) {
        int x = a + b;
        if (x) t[x].p = 0;
        return x;
    }
    if (t[a].y < t[b].y) {
        push(b);
        t[b].l = merge(a, t[b].l);
        pull(b); t[b].p = 0; return b;
    }
    push(a);
    t[a].r = merge(t[a].r, b);
    pull(a); t[a].p = 0; return a;
}
pii split(int x, int k) { // First k tokens go left.
    if (!x) return {0, 0};
    push(x);
    int a, b;
    if (t[t[x].l].c >= k) {
        tie(a, b) = split(t[t[x].l], k);
        t[x].l = b; b = x;
    } else {
        tie(a, b) = split(t[t[x].r, k - t[t[x].l].c - 1];
        t[x].r = a; a = x;
    }
    pull(x);
    if (a) t[a].p = 0;
    if (b) t[b].p = 0;
    return {a, b};
}
int rotate(int x) { // Start the cyclic tour at x.
    auto [a, b] = split(root(x), rank(x));
    return merge(b, a);
}
int token() {
    int x;
    if (spare.empty()) x = sz(t), t.emplace_back();
    else x = spare.back(), spare.pop_back();
    t[x] = Node{}; t[x].y = (unsigned)rng();
    return x;
}
bool connected(int u, int v) {
    return root(u + 1) == root(v + 1);
}
void link(int u, int v) {
    assert(!connected(u, v));
    int a = rotate(u + 1), b = rotate(v + 1);
    int x = token(), y = token();
    edges[minmax(u, v)] = {x, y};
    merge(merge(a, x), merge(b, y));
}
void cut(int u, int v) {
    auto it = edges.find(minmax(u, v));
    assert(it != edges.end());
    auto [x, y] = it->second;
    int r = rotate(x);
    auto [a, b] = split(r, rank(y));
    split(a, 1); split(b, 1); // Drop x and y.
}
```

```
spare.push_back(x); spare.push_back(y);
edges.erase(it);
}
void expose(int x) { // Push ancestors before point access.
    if (t[x].p) expose(t[x].p);
    push(x);
}
void set(int u, V val) {
    int x = u + 1;
    expose(x);
    t[x].val = val;
    for (; x; x = t[x].p) pull(x);
}
V get(int u) { expose(u + 1); return t[u + 1].val; }
void componentUpdate(int u, U upd) {
    apply(root(u + 1), upd);
}
V componentQuery(int u) { return t[root(u+1)].agg; }
int componentSize(int u) {
    return t[root(u+1)].cnt;
}
};
```

BinaryTrie.h

Description: Binary trie on $[0, 2^B)$: 0 included, 2^B excluded. Set insert/erase, multiset insert<1>, XOR-min/max, count $x \oplus y < k$ (countLG<0>) or $> k$ (countLG<1>), lazy XOR-all, mex (set). Both countLG variants exclude equality to k. count(x) counts copies of x. kth(k): 0-indexed kth smallest (with repeats), requires $0 \leq k < \text{cnt}$. XOR queries take xr (default 0). cnt is the size. each(f) calls $f(x, \text{cnt})$. merge is set-union (destroys o; safe to delete); merge<1> adds counts. mex needs unique values.

Time: $\mathcal{O}(B)$ per op

d41d8c, 100 lines

```
// Only implement required functions
struct BinaryTrie {
    using T = BinaryTrie;
    static const int B = 30;
    int cnt = 0, lazy = 0;
    T *c[2] = {};
    T& ch(int b) {
        return c[b] ? c[b] : c[b] = new T();
    }
    int cc(int b) { return c[b] ? c[b]->cnt : 0; }
    void push(int i) {
        if (!lazy) return;
        if (i && (lazy >> (i - 1) & 1)) swap(c[0], c[1]);
        rep(b, 0, 2) if (c[b]) c[b]->lazy ^= lazy;
        lazy = 0;
    }
    void xorAll(int x) { lazy ^= x; }
    template<int ms = 0>
    int insert(int x, int i = B) {
        push(i);
        if (!i) return !ms && cnt ? 0 : (++cnt, 1);
        int b = x >> --i & 1;
        int add = ch(b)->insert<ms>(x, i);
        return cnt += add, add;
    }
    int erase(int x, int i = B) {
        if (!cnt) return 0;
        push(i);
        int sub = 1;
        if (i) {
            int b = x >> --i & 1;
            sub = c[b] ? c[b]->erase(x, i) : 0;
        }
    }
};
```

```

    }
    return cnt -= sub, sub;
}
int count(int x, int i = B) {
    if (!i || !cnt) return cnt;
    push(i);
    int b = x >> --i & 1;
    return c[b] ? c[b]->count(x, i) : 0;
}
int kth(int k, int i = B) {
    assert(0 <= k && k < cnt);
    if (!i) return 0;
    push(i);
    int n = cc(0), b = k >= n;
    return c[b]->kth(k - (b ? n : 0), i - 1)
        | (b << (i - 1));
}
int minxor(int xr = 0, int i = B) {
    if (!i || !cnt) return 0;
    push(i);
    int b = xr >> --i & 1;
    return cc(b) ? c[b]->minxor(xr, i) :
        c[!b]->minxor(xr, i) | 1 << i;
}
int maxxor(int xr = 0, int i = B) {
    if (!i || !cnt) return 0;
    push(i);
    int b = (xr >> --i & 1) ^ 1;
    return cc(b) ? c[b]->maxxor(xr, i) | 1 << i :
        c[!b]->maxxor(xr, i);
}
template<int sgn = 0>
int countLG(int xr = 0, int k = 0, int i = B) {
    if (!i || !cnt) return 0;
    push(i);
    int b = (xr ^ k) >> --i & 1;
    return ((k >> i & 1) ^ sgn ? cc(!b) : 0) +
        (c[b] ? c[b]->countLG<sgn>(xr, k, i) : 0);
}
int mex(int xr = 0, int i = B) { // unique values
    if (!i) return 0;
    push(i);
    int b = xr >> --i & 1;
    if (cc(b) == 1 << i)
        return (c[!b] ? c[!b]->mex(xr, i) : 0) | 1 << i;
    return c[b] ? c[b]->mex(xr, i) : 0;
}
template<class F>
void each(F f, int i = B, int x = 0) {
    if (!cnt) return;
    push(i);
    if (!i) f(x, cnt);
    else rep(b, 0, 2) if (c[b])
        c[b]->each(f, i - 1, x | (b << (i - 1)));
}
template<int ms = 0>
void merge(T& o, int i = B) { // o safe to delete
    if (!o.cnt) return;
    push(i); o.push(i);
    if (i) rep(b, 0, 2) {
        if (!c[b]) swap(c[b], o.c[b]);
        else if (o.c[b]) c[b]->merge<ms>(*o.c[b], i - 1);
    }
    cnt = i ? cc(0) + cc(1) : (ms ? cnt + o.cnt : 1);
    o.cnt = 0;
}

```

```

~BinaryTrie() { delete c[0]; delete c[1]; }
};

```

SegmentTreeBeats.h

Description: Segment tree beats. Range chmin/chmax/add and range sum/min/max. All update/query bounds $[ql, qr]$ include both endpoints; full range $[0, n - 1]$.

Time: $\mathcal{O}(\log^2 N)$ amortized

d41d8c, 172 lines

```

struct SegmentTreeBeats {
    static constexpr ll INF = LLONG_MAX / 4;
    struct Node {
        ll sum = 0, lazy = 0;
        ll mx1 = 0, mx2 = -INF, mxc = 0;
        ll mn1 = 0, mn2 = INF, mnc = 0;
    };
    vector<Node> t;
    int n;

    SegmentTreeBeats(int n) : t(4 * n + 4), n(n) {
        build(1, 0, n - 1, nullptr);
    }
    SegmentTreeBeats(const vector<ll>& a) :
        t(4 * sz(a) + 4), n(sz(a)) {
        build(1, 0, n - 1, &a);
    }

    void pull(int i) {
        Node &u = t[i], &l = t[i << 1], &r = t[i << 1 | 1];
        u.sum = l.sum + r.sum;
        if (l.mx1 == r.mx1) {
            u.mx1 = l.mx1;
            u.mx2 = max(l.mx2, r.mx2);
            u.mxc = l.mxc + r.mxc;
        } else if (l.mx1 > r.mx1) {
            u.mx1 = l.mx1;
            u.mx2 = max(l.mx2, r.mx1);
            u.mxc = l.mxc;
        } else {
            u.mx1 = r.mx1;
            u.mx2 = max(l.mx1, r.mx2);
            u.mxc = r.mxc;
        }
        if (l.mn1 == r.mn1) {
            u.mn1 = l.mn1;
            u.mn2 = min(l.mn2, r.mn2);
            u.mnc = l.mnc + r.mnc;
        } else if (l.mn1 < r.mn1) {
            u.mn1 = l.mn1;
            u.mn2 = min(l.mn2, r.mn1);
            u.mnc = l.mnc;
        } else {
            u.mn1 = r.mn1;
            u.mn2 = min(l.mn1, r.mn2);
            u.mnc = r.mnc;
        }
    }
    void applyAdd(int i, int l, int r, ll v) {
        if (!v) return;
        Node& u = t[i];
        u.sum += (r - l + 1) * v;
        u.mx1 += v;
        if (u.mx2 != -INF) u.mx2 += v;
        u.mn1 += v;
        if (u.mn2 != INF) u.mn2 += v;
    }
}

```

```

    u.lazy += v;
}
void applyChmin(int i, ll v, bool leaf) {
    Node& u = t[i];
    if (v >= u.mx1) return;
    u.sum -= (u.mx1 - v) * u.mxc;
    u.mx1 = v;
    if (leaf) u.mn1 = v;
    else if (v <= u.mn1) u.mn1 = v;
    else if (v < u.mn2) u.mn2 = v;
}
void applyChmax(int i, ll v, bool leaf) {
    Node& u = t[i];
    if (v <= u.mn1) return;
    u.sum += (v - u.mn1) * u.mnc;
    u.mn1 = v;
    if (leaf) u.mx1 = v;
    else if (v >= u.mx1) u.mx1 = v;
    else if (v > u.mx2) u.mx2 = v;
}
void push(int i, int l, int r) {
    if (l == r) return;
    int m = (l + r) >> 1;
    applyAdd(i << 1, l, m, t[i].lazy);
    applyAdd(i << 1 | 1, m + 1, r, t[i].lazy);
    t[i].lazy = 0;
    applyChmin(i << 1, t[i].mx1, l == m);
    applyChmin(i << 1 | 1, t[i].mx1, m + 1 == r);
    applyChmax(i << 1, t[i].mn1, l == m);
    applyChmax(i << 1 | 1, t[i].mn1, m + 1 == r);
}
void build(int i, int l, int r, const vector<ll>* a) {
    t[i].lazy = 0;
    if (l == r) {
        ll v = a ? (*a)[l] : 0;
        t[i].sum = t[i].mx1 = t[i].mn1 = v;
        t[i].mxc = t[i].mnc = 1;
        t[i].mx2 = -INF;
        t[i].mn2 = INF;
        return;
    }
    int m = (l + r) >> 1;
    build(i << 1, l, m, a);
    build(i << 1 | 1, m + 1, r, a);
    pull(i);
}
void add(int ql, int qr, ll v,
    int i = 1, int l = 0, int r = -1) {
    if (r == -1) r = n - 1;
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr) {
        applyAdd(i, l, r, v); return;
    }
    push(i, l, r);
    int m = (l + r) >> 1;
    add(ql, qr, v, i << 1, l, m);
    add(ql, qr, v, i << 1 | 1, m + 1, r);
    pull(i);
}
void chmin(int ql, int qr, ll v,
    int i = 1, int l = 0, int r = -1) {
    if (r == -1) r = n - 1;
    if (qr < l || r < ql || v >= t[i].mx1) return;
    if (ql <= l && r <= qr && v > t[i].mx2) {

```

```

    applyChmin(i, v, l == r); return;
}
push(i, l, r);
int m = (l + r) >> 1;
chmin(ql, qr, v, i << 1, l, m);
chmin(ql, qr, v, i << 1 | 1, m + 1, r);
pull(i);
}
void chmax(int ql, int qr, ll v,
int i = 1, int l = 0, int r = -1) {
    if (r == -1) r = n - 1;
    if (qr < l || r < ql || v <= t[i].mn1) return;
    if (ql <= l && r <= qr && v < t[i].mn2) {
        applyChmax(i, v, l == r); return;
    }
    push(i, l, r);
    int m = (l + r) >> 1;
    chmax(ql, qr, v, i << 1, l, m);
    chmax(ql, qr, v, i << 1 | 1, m + 1, r);
    pull(i);
}

ll qsum(int ql, int qr, int i = 1, int l = 0, int r = -1) {
    if (r == -1) r = n - 1;
    if (qr < l || r < ql) return 0;
    if (ql <= l && r <= qr) return t[i].sum;
    push(i, l, r);
    int m = (l + r) >> 1;
    return qsum(ql, qr, i << 1, l, m)
        + qsum(ql, qr, i << 1 | 1, m + 1, r);
}

ll qmax(int ql, int qr, int i = 1, int l = 0, int r = -1) {
    if (r == -1) r = n - 1;
    if (qr < l || r < ql) return -INF;
    if (ql <= l && r <= qr) return t[i].mx1;
    push(i, l, r);
    int m = (l + r) >> 1;
    return max(qmax(ql, qr, i << 1, l, m),
        qmax(ql, qr, i << 1 | 1, m + 1, r));
}

ll qmin(int ql, int qr, int i = 1, int l = 0, int r = -1) {
    if (r == -1) r = n - 1;
    if (qr < l || r < ql) return INF;
    if (ql <= l && r <= qr) return t[i].mn1;
    push(i, l, r);
    int m = (l + r) >> 1;
    return min(qmin(ql, qr, i << 1, l, m),
        qmin(ql, qr, i << 1 | 1, m + 1, r));
}
}
};

```

SparseLazySegmentTree.h

Description: Implicit segment tree with lazy range updates and point set. Creates children on demand. Node bounds $[l, r]$ and update/query bounds $[ql, qr]$ include both endpoints. For n elements use $[0, n - 1]$. Change $V, U, id, def, idU, binop, applyUpdate, mergeUpdate$. Default is range add / range max.

Time: $\mathcal{O}(\log R)$

d41d8c, 58 lines

```

struct Node {
    using V = int;
    using U = int;
    using T = Node;
    static constexpr V id = INT_MIN;
    static constexpr V def = 0;

```

SparseLazySegmentTree SparseSegmentTree2d

```

static constexpr U idU = 0;
V binop(V a, V b) { return max(a, b); }
V applyUpdate(U u, V v) { return v + u; }
U mergeUpdate(U oldU, U nw) { return oldU + nw; }
T *lt = 0, *rt = 0;
V val = def;
U lazy = idU;
void updateNode(int, int, U u) {
    lazy = mergeUpdate(lazy, u);
    val = applyUpdate(u, val);
}
void push(int l, int r) {
    if (!lt) {
        lt = new T();
        rt = new T();
    }
    int mid = l + (r - l) / 2;
    lt->updateNode(l, mid, lazy);
    rt->updateNode(mid + 1, r, lazy);
    lazy = idU;
}
void update(int l, int r, int ql, int qr, U u) {
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr) {
        updateNode(l, r, u);
        return;
    }
    push(l, r);
    int mid = l + (r - l) / 2;
    lt->update(l, mid, ql, qr, u);
    rt->update(mid + 1, r, ql, qr, u);
    val = binop(lt->val, rt->val);
}
void set(int l, int r, int i, V v) {
    if (i < l || r < i) return;
    if (l == r) { val = v; lazy = idU; return; }
    push(l, r);
    int mid = l + (r - l) / 2;
    if (i <= mid) lt->set(l, mid, i, v);
    else rt->set(mid + 1, r, i, v);
    val = binop(lt->val, rt->val);
}
V query(int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return id;
    if (ql <= l && r <= qr) return val;
    push(l, r);
    int mid = l + (r - l) / 2;
    return binop(lt->query(l, mid, ql, qr),
        rt->query(mid + 1, r, ql, qr));
}
Node() { delete lt; delete rt; }
};

```

SparseSegmentTree2d.h

Description: Online sparse 2D segment tree on $[0, n) \times [0, m)$. Point assignment with $set(x, y, v)$, rectangle query with $query(x1, x2, y1, y2)$ uses $[x1, x2) \times [y1, y2)$: lower bounds included, upper bounds excluded. Node bounds are half-open too. Unassigned points have value unit. Change $V, unit, f$ for any commutative monoid. Queries allocate no nodes.

Memory: $\mathcal{O}(k \log n \log m)$ for k distinct updated points. Allocated nodes are retained until destruction.

Time: $\mathcal{O}(\log n \log m)$ per operation.

d41d8c, 69 lines

```

struct Tree2D {
    using V = ll;

```

```

static constexpr V unit = LLONG_MIN;
static V f(V a, V b) { return max(a, b); }
struct Y {
    using T = Y;
    T *lt = 0, *rt = 0;
    V val = unit;
    static V get(T* p) { return p ? p->val : unit; }
    void set(int l, int r, int y, V v) {
        if (r - l == 1) { val = v; return; }
        int mid = l + (r - l) / 2;
        T* ch = y < mid ? lt : rt;
        if (!ch) ch = new T;
        if (y < mid) ch->set(l, mid, y, v);
        else ch->set(mid, r, y, v);
        val = f(get(lt), get(rt));
    }
    V query(int l, int r, int a, int b) const {
        if (b <= l || r <= a) return unit;
        if (a <= l && r <= b) return val;
        int mid = l + (r - l) / 2;
        return f(lt ? lt->query(l, mid, a, b) : unit,
            rt ? rt->query(mid, r, a, b) : unit);
    }
    ~Y() { delete lt; delete rt; }
};
struct Node {
    using T = Node;
    T *lt = 0, *rt = 0;
    Y tree;
    void set(int l, int r, int m, int x, int y, V v) {
        if (r - l > 1) {
            int mid = l + (r - l) / 2;
            T* ch = x < mid ? lt : rt;
            if (!ch) ch = new T;
            if (x < mid) ch->set(l, mid, m, x, y, v);
            else ch->set(mid, r, m, x, y, v);
            v = f(lt ? lt->tree.query(0, m, y, y+1) : unit,
                rt ? rt->tree.query(0, m, y, y+1) : unit);
        }
        tree.set(0, m, y, v);
    }
    V query(int l, int r, int m, int a, int b,
        int c, int d) const {
        if (b <= l || r <= a) return unit;
        if (a <= l && r <= b)
            return tree.query(0, m, c, d);
        int mid = l + (r - l) / 2;
        return f(lt ? lt->query(l, mid, m, a, b, c, d) : unit,
            rt ? rt->query(mid, r, m, a, b, c, d) : unit);
    }
    ~Node() { delete lt; delete rt; }
} root;
int n, m;
Tree2D(int n, int m) : n(n), m(m) {
    assert(n > 0 && m > 0);
}
Tree2D(const Tree2D&) = delete;
Tree2D& operator=(const Tree2D&) = delete;
void set(int x, int y, V v) {
    assert(0 <= x && x < n && 0 <= y && y < m);
    root.set(0, n, m, x, y, v);
}
V query(int x1, int x2, int y1, int y2) const {
    if (x1 >= x2 || y1 >= y2) return unit;
    return root.query(0, n, m, x1, x2, y1, y2);
}

```

```
};
```

PersistentSegmentTree.h

Description: Persistent implicit segment tree with lazy range updates and point set. Update/set return a new root; old roots stay valid. No destructor (nodes are shared). Node bounds $[l, r]$ and update/query bounds $[ql, qr]$ include both endpoints. For n elements use $[0, n-1]$. Change $V, U, id, def, idU, binop, applyUpdate, mergeUpdate$. Default is range add / range max.

Time: $\mathcal{O}(\log R)$

d41d8c, 62 lines

```
struct Node {
    using V = int;
    using U = int;
    using T = Node;
    static constexpr V id = INT_MIN;
    static constexpr V def = 0;
    static constexpr U idU = 0;
    V binop(V a, V b) { return max(a, b); }
    V applyUpdate(U u, V v) { return v + u; }
    U mergeUpdate(U oldU, U nw) { return oldU + nw; }
    T *lt = 0, *rt = 0;
    V val = def;
    U lazy = idU;
    void updateNode(int, int, U u) {
        lazy = mergeUpdate(lazy, u);
        val = applyUpdate(u, val);
    }
    void push(int l, int r) {
        int mid = l + (r - l) / 2;
        lt = new T(lt ? *lt : T());
        rt = new T(rt ? *rt : T());
        lt->updateNode(l, mid, lazy);
        rt->updateNode(mid + 1, r, lazy);
        lazy = idU;
    }
    T* update(int l, int r, int ql, int qr, U u) {
        if (qr < l || r < ql) return this;
        T* n = new T(*this);
        if (ql <= l && r <= qr) {
            n->updateNode(l, r, u);
            return n;
        }
        n->push(l, r);
        int mid = l + (r - l) / 2;
        n->lt = n->lt->update(l, mid, ql, qr, u);
        n->rt = n->rt->update(mid + 1, r, ql, qr, u);
        n->val = binop(n->lt->val, n->rt->val);
        return n;
    }
    T* set(int l, int r, int i, V v) {
        if (i < l || r < i) return this;
        T* n = new T(*this);
        if (l == r) { n->val = v; n->lazy = idU; return n; }
        n->push(l, r);
        int mid = l + (r - l) / 2;
        if (i <= mid) n->lt = n->lt->set(l, mid, i, v);
        else n->rt = n->rt->set(mid + 1, r, i, v);
        n->val = binop(n->lt->val, n->rt->val);
        return n;
    }
    V query(int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return id;
        if (ql <= l && r <= qr) return val;
        int mid = l + (r - l) / 2;
```

```
    T a = lt ? *lt : T();
    T b = rt ? *rt : T();
    a.updateNode(l, mid, lazy);
    b.updateNode(mid + 1, r, lazy);
    return binop(a.query(l, mid, ql, qr),
        b.query(mid + 1, r, ql, qr));
}
};
```

MinPlusConvolution.h

Description: Min-plus convolution with one convex or concave input. SMAWK handles arbitrary/convex; the border algorithm handles arbitrary/concave. Here, convex = smiley and concave = frowny. `min_smark(f, r, c)` returns a minimizing column for each row of the matrix f . Totally monotone means every submatrix's leftmost row-minimum indices are nondecreasing. Queried through f , not stored. Both a and b must be non-empty.

Time: $\mathcal{O}(N + M)$ convex, $\mathcal{O}(N \log M + M)$ concave

d41d8c, 96 lines

```
// Row-minima indices for totally monotone f(x,y).  $\mathcal{O}(N + M)$ 
template<class Fn>
vi min_smark_rec(Fn& f, const vi& row, vi col) {
    int n = sz(row);
    if (!n) return {};
    vi red;
    for (int c : col) {
        while (sz(red) &&
            f(row[sz(red)-1], c) < f(row[sz(red)-1], red.back()))
            red.pop_back();
        if (sz(red) < n) red.pb(c);
    }
    col = move(red);

    vi odd;
    for (int i = 1; i < n; i += 2) odd.pb(row[i]);
    vi oddans = min_smark_rec(f, odd, col), ans(n);
    rep(i, 0, sz(odd)) ans[2*i+1] = oddans[i];
    for (int i = 0, j = 0; i < n; i += 2) {
        ans[i] = col[j];
        int last = i+1 < n ? ans[i+1] : col.back();
        while (col[j] != last) {
            j++;
            if (f(row[i], col[j]) < f(row[i], ans[i]))
                ans[i] = col[j];
        }
    }
    return ans;
}
template<class Fn>
vi min_smark(Fn f, int r, int c) {
    vi row(r), col(c);
    iota(all(row), 0), iota(all(col), 0);
    return min_smark_rec(f, row, col);
}
```

// Min-plus $c[k] = \min_i \{a[i] + b[j]\}$, convex b . $\mathcal{O}(N+M)$

```
template<class V>
vector<V> min_plus_smark(const vector<V>& a,
    const vector<V>& b) {
    int n = sz(a), m = sz(b);
    assert(n && m);
    auto f = [&](int r, int c) -> tuple<int, V, int> {
        if (r < c) return {1, V{}, c};
        if (r-c >= m) return {1, V{}, -c};
        return {0, a[c]+b[r-c], 0};
```

```
};
vi cols = min_smark(f, n+m-1, n);
vector<V> d(n+m-1);
rep(r, 0, n+m-1) d[r] = a[cols[r]]+b[r-cols[r]];
return d;
}
```

// Min-plus conv for concave b . $\mathcal{O}(N \log M + M)$

```
template<class V>
vector<V> min_plus_concave_one(const vector<V>& a,
    const vector<V>& b) {
    int n = sz(a), m = sz(b), z = n+m-1;
    assert(n && m);
    vector<V> c(z, numeric_limits<V>::max());
    auto solve = [&](int l, int r, bool rev) {
        auto val = [&](int j, int k) {
            if (rev) j = n-1-j, k = z-1-k;
            return a[j]+b[k-j];
        };
        #define better(i,j,k) val(i,k) <= val(j,k)
        auto improve = [&](int u, int v, int l, int r) {
            while (r-l > 1) {
                int m = (l+r)/2;
                (better(u,v,m) ? r : l) = m;
            }
            return l;
        };
        vector<array<int, 2>> stk;
        for (int i = l, k = l, s = -1; k < r; i++, k++) {
            while (s >= 0 && i < n &&
                better(i, stk[s][0], stk[s][1])) {
                stk.pop_back(), s--;
            }
            if (i < n) {
                int t = s < 0 ? r-1
                    : improve(stk[s][0], i, k-1, stk[s][1]);
                if (t >= k) stk.pb({i, t}), s++;
            }
            int out = rev ? z-1-k : k;
            c[out] = min(c[out], val(stk[s][0], k));
            if (stk[s][1] == k) stk.pop_back(), s--;
        }
    };
    solve(0, m, 0), solve(0, m, 1);
    for (int k = m; k < z-m; k += m+1)
        solve(k+1, k+m+1, 0), solve(z-k-m, z-k, 1);
    #undef better
    return c;
}
```

LinkCutTree.h

Description: Unrooted link-cut tree. Online link/cut. Default path add + path sum. `query(u,v)` and `update(u,v,x)` use the path including both endpoint vertices u and v .

Usage: `LinkCut lc(n);`

`lc.link(u, v); lc.cut(u, v);`
`lc.connected(u, v); lc.query(u, v);`
`lc.update(u, v, x); lc.set(u, x);`

Time: amortized $\mathcal{O}(\log N)$

d41d8c, 137 lines

```
struct LinkCut {
    using V = ll;
    using U = ll;
    static constexpr V id = 0, def = 0;
```

```

static constexpr U idU = 0;
V binop(V a, V b) { return a + b; }
V rev(V a) { return a; } // non-commutative: reverse
V applyUpdate(U u, V v, int c) {
    return v + u * c;
}
U mergeUpdate(U a, U b) { return a + b; }
struct N {
    int c[2] = {-1, -1}, p = -1, sz = 1;
    int flip = 0; // reverse; keep
    V val = def, agg = def;
    U lz = idU;
};
vector<N> t;
LinkCut(int n) : t(n) {}
bool nroot(int x) { // else p is path-parent
    int p = t[x].p;
    return p != -1 &&
        (t[p].c[0] == x || t[p].c[1] == x);
}
V aggOf(int x) { return x < 0 ? id : t[x].agg; }
void pull(int x) { // path aggregate
    int a = t[x].c[0], b = t[x].c[1];
    t[x].sz = 1 + (a < 0 ? t[a].sz) + (b < 0 ? t[b].sz);
    t[x].agg = binop(binop(aggOf(a), t[x].val),
        aggOf(b));
}
void apply(int x, U u) { // path add
    t[x].val = applyUpdate(u, t[x].val, 1);
    t[x].agg = applyUpdate(u, t[x].agg, t[x].sz);
    t[x].lz = mergeUpdate(t[x].lz, u);
}
void pushFlip(int x) { // structural; not a value tag
    if (!t[x].flip) return;
    swap(t[x].c[0], t[x].c[1]);
    t[x].agg = rev(t[x].agg);
    int a = t[x].c[0], b = t[x].c[1];
    if (a >= 0) t[a].flip ^= 1, t[a].agg = rev(t[a].agg);
    if (b >= 0) t[b].flip ^= 1, t[b].agg = rev(t[b].agg);
    t[x].flip = 0;
}
void pushLazy(int x) { // skip if idU
    U u = t[x].lz;
    if (u == idU) return;
    int a = t[x].c[0], b = t[x].c[1];
    if (a >= 0) apply(a, u);
    if (b >= 0) apply(b, u);
    t[x].lz = idU;
}
void push(int x) {
    pushFlip(x); pushLazy(x);
}
void rot(int x) {
    int y = t[x].p, z = t[y].p;
    int k = t[y].c[1] == x;
    if (nroot(y)) t[z].c[t[z].c[1] == y] = x;
    int w = t[x].c[k^1];
    t[y].c[k] = w;
    if (w >= 0) t[w].p = y;
    t[x].c[k^1] = y;
    t[y].p = x; t[x].p = z;
    pull(y);
}
void splay(int x) {
    vi stk = {x};
    for (int y = x; nroot(y); )

```

```

        stk.push_back(y = t[y].p);
    while (!stk.empty())
        push(stk.back()), stk.pop_back();
    while (nroot(x)) {
        int y = t[x].p, z = t[y].p;
        if (nroot(y))
            rot((t[y].c[0] == x) == (t[z].c[0] == y) ? y : x);
        rot(x);
    }
    pull(x);
}
void access(int x) { // preferred path through x
    int last = -1;
    for (int y = x; y >= 0; y = t[y].p) {
        splay(y); t[y].c[1] = last;
        pull(y); last = y;
    }
    splay(x);
}
void makeRoot(int x) { // evert; why this is unrooted
    access(x); t[x].flip ^= 1;
}
int findRoot(int x) { // current evert; use connected
    access(x);
    for (;;) {
        push(x);
        if (t[x].c[0] < 0) break;
        x = t[x].c[0];
    }
    splay(x);
    return x;
}
void link(int u, int v) { // add edge (u, v)
    assert(!connected(u, v));
    makeRoot(u); t[u].p = v;
}
void cut(int u, int v) { // remove edge (u, v)
    makeRoot(u); access(v);
    assert(t[v].c[0] == u && t[u].c[0] < 0 &&
        t[u].c[1] < 0);
    t[v].c[0] = t[u].p = -1;
    pull(v);
}
bool connected(int u, int v) {
    return findRoot(u) == findRoot(v);
}
// makeRoot(u); access(v); => v's splay is path u-v
V query(int u, int v) {
    makeRoot(u); access(v);
    return t[v].agg;
}
void update(int u, int v, U x) {
    makeRoot(u); access(v); apply(v, x);
}
void set(int u, V x) {
    access(u); t[u].val = x; t[u].lz = idU;
    pull(u);
}
V get(int u) {
    access(u); return t[u].val;
}
int lca(int r, int u, int v) { // wrt root r
    makeRoot(r); access(u); access(v);
    splay(u);
    return t[u].p < 0 ? u : t[u].p;
}

```

};

10.1 Intervals

IntervalContainer.h

Description: Add and remove intervals from a set of disjoint intervals. Adding merges overlapping or touching intervals. Stored intervals and add/remove bounds are $[L, R)$: L included, R excluded. Empty intervals have $L = R$.

Time: $\mathcal{O}(\log N)$

d41d8c, 23 lines

```

set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
    if (L == R) return is.end();
    auto it = is.lower_bound({L, R}), before = it;
    while (it != is.end() && it->first <= R) {
        R = max(R, it->second);
        before = it = is.erase(it);
    }
    if (it != is.begin() && (--it)->second >= L) {
        L = min(L, it->first);
        R = max(R, it->second);
        is.erase(it);
    }
    return is.insert(before, {L, R});
}

```

```

void removeInterval(set<pii>& is, int L, int R) {
    if (L == R) return;
    auto it = addInterval(is, L, R);
    auto r2 = it->second;
    if (it->first == L) is.erase(it);
    else (int&)it->second = L;
    if (R != r2) is.emplace(R, r2);
}

```

IntervalCover.h

Description: Compute indices of smallest set of intervals covering another interval. Both target G and candidate intervals I use $[l, r)$: l included, r excluded. To support $[l, r]$ (both included), change (A) to add $||$ $R.empty()$. Returns empty set on failure (or if G is empty).

Time: $\mathcal{O}(N \log N)$

d41d8c, 19 lines

```

template<class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I) {
    vi S(sz(I)), R;
    iota(all(S), 0);
    sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
    T cur = G.first;
    int at = 0;
    while (cur < G.second) { // (A)
        pair<T, int> mx = make_pair(cur, -1);
        while (at < sz(I) && I[S[at]].first <= cur) {
            mx = max(mx, make_pair(I[S[at]].second, S[at]));
            at++;
        }
        if (mx.second == -1) return {};
        cur = mx.first;
        R.push_back(mx.second);
    }
    return R;
}

```

ConstantIntervals.h

Description: Split a monotone function on $[from, to]$ into a minimal set of intervals of constant value. from included, to excluded. Each callback $g(lo, hi, val)$ describes $[lo, hi)$: lo included, hi excluded.
Usage: `constantIntervals(0, sz(v), [&](int x){return v[x];}, [&](int lo, int hi, T val){...});`
Time: $\mathcal{O}(k \log \frac{n}{k})$

d41d8c, 19 lines

```
template<class F, class G, class T>
void rec(int from, int to, F& f, G& g, int& i, T& p, T q) {
    if (p == q) return;
    if (from == to) {
        g(i, to, p);
        i = to; p = q;
    } else {
        int mid = (from + to) >> 1;
        rec(from, mid, f, g, i, p, f(mid));
        rec(mid+1, to, f, g, i, p, q);
    }
}
template<class F, class G>
void constantIntervals(int from, int to, F f, G g) {
    if (to <= from) return;
    int i = from; auto p = f(i), q = f(to-1);
    rec(from, to-1, f, g, i, p, q);
    g(i, to, q);
}
```

10.2 Misc. algorithms

BigInt.h

Description: Unsigned integer with $N > 0$ base- 10^9 limbs, most significant first (up to $9N$ digits). Arithmetic and parsing wrap modulo 10^{9N} . String input must be nonempty decimal digits; output has no leading zeros. Multiply/divide by an unsigned 32-bit integer; division requires a nonzero divisor. `div(b)` divides in place and returns the remainder.
Usage: `BigInt<4> x("12345678901234567890"); x.toString();`
Time: $\mathcal{O}(N)$ arithmetic, $\mathcal{O}(N + D)$ string conversion for D decimal digits.

d41d8c, 70 lines

```
using ull = unsigned long long;
template<int N> struct BigInt : array<unsigned, N> {
    static_assert(N > 0);
    static constexpr unsigned base = 1000000000;
    BigInt(ull x = 0) : array<unsigned, N>{} {
        for (int i = N; i-- && x; x /= base)
            (*this)[i] = unsigned(x % base);
    }
    explicit BigInt(string_view s) : BigInt() {
        assert(!s.empty());
        for (char c : s) assert('0' <= c && c <= '9');
        int end = sz(s);
        for (int i = N; i-- && end > 0; i) {
            int start = max(0, end-9);
            unsigned x = 0;
            rep(j, start, end) x = x*10 + unsigned(s[j]-'0');
            (*this)[i] = x, end = start;
        }
    }
    string toString() const {
        int i = 0;
        while (i < N-1 && !(*this)[i]) i++;
        string s = to_string((*this)[i]);
        while (++i < N) {
            string t = to_string((*this)[i]);
```

```
        s += string(9-sz(t), '0') + t;
    }
    return s;
}
BigInt& operator+=(const BigInt& b) {
    unsigned carry = 0;
    for (int i = N; i--;) {
        unsigned t = (*this)[i] + b[i] + carry;
        (*this)[i] = t % base, carry = t / base;
    }
    return *this;
}
BigInt operator-() const {
    BigInt r = *this;
    for (unsigned& x : r) x = base-1-x;
    return r += 1;
}
BigInt& operator--(const BigInt& b) { return *this += -b; }
friend BigInt operator+(BigInt a, const BigInt& b) { return a += b; }
friend BigInt operator-(BigInt a, const BigInt& b) { return a -= b; }
BigInt& operator*=(unsigned b) {
    ull carry = 0;
    for (int i = N; i--;) {
        ull t = ull((*this)[i]) * b + carry;
        (*this)[i] = unsigned(t % base), carry = t / base;
    }
    return *this;
}
unsigned div(unsigned b) {
    assert(b);
    ull rem = 0;
    rep(i, 0, N) {
        ull t = rem * base + (*this)[i];
        (*this)[i] = unsigned(t / b), rem = t % b;
    }
    return unsigned(rem);
}
BigInt& operator/=(unsigned b) { div(b); return *this; }
friend BigInt operator*(BigInt a, unsigned b) { return a *= b; }
friend BigInt operator*(unsigned b, BigInt a) { return a *= b; }
friend BigInt operator/(BigInt a, unsigned b) { return a /= b; }
friend unsigned operator%(BigInt a, unsigned b) {
    return a.div(b);
}
};
```

TernarySearch.h

Description: Find the smallest i in $[a, b]$ that maximizes $f(i)$, assuming that $f(a) < \dots < f(i) \geq \dots \geq f(b)$. Both a and b are included; for an array use 0 and $n-1$. To reverse which of the sides allows non-strict inequalities, change the $<$ marked with (A) to $<=$, and reverse the loop at (B). To minimize f , change it to $>$, also at (B).
Usage: `int ind = ternSearch(0, n-1, [&](int i){return a[i];});`
Time: $\mathcal{O}(\log(b-a))$

d41d8c, 11 lines

```
template<class F>
int ternSearch(int a, int b, F f) {
    assert(a <= b);
    while (b - a >= 5) {
        int mid = (a + b) / 2;
        if (f(mid) < f(mid+1)) a = mid; // (A)
        else b = mid+1;
    }
    rep(i, a+1, b+1) if (f(a) < f(i)) a = i; // (B)
    return a;
}
```

}

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t , computes the maximum $S \leq t$ such that S is the sum of some subset of the weights.
Time: $\mathcal{O}(N \max(w_i))$

d41d8c, 16 lines

```
int knapsack(vi w, int t) {
    int a = 0, b = 0, x;
    while (b < sz(w) && a + w[b] <= t) a += w[b++];
    if (b == sz(w)) return a;
    int m = *max_element(all(w));
    vi u, v(2*m, -1);
    v[a+m-t] = b;
    rep(i, b, sz(w)) {
        u = v;
        rep(x, 0, m) v[x+w[i]] = max(v[x+w[i]], u[x]);
        for (x = 2*m; --x > m;) rep(j, max(0, u[x]), v[x])
            v[x-w[j]] = max(v[x-w[j]], j);
    }
    for (a = t; v[a+m-t] < 0; a--);
    return a;
}
```

10.3 Dynamic programming

KnuthDP.h

Description: Interval DP $dp(i, j) = \min_{i \leq k < j} (dp(i, k) + dp(k+1, j)) + C(i, j)$. dp and C use $[i, j]$, both endpoints included. Split candidates use $[i, j)$: i included, j excluded. Any fixed offsets from k work. Let $opt(i, j)$ be an optimal k . Need $opt(i, j-1) \leq opt(i, j) \leq opt(i+1, j)$. Enough if $C(b, c) \leq C(a, d)$ and quadrangle $C(a, c) + C(b, d) \leq C(a, d) + C(b, c)$ for $a \leq b \leq c \leq d$ ("wider gets worse faster"). Useful C : $C(i, j) + C(i+1, j+1) \leq C(i, j+1) + C(i+1, j)$; $C(i, j) = \sum_{k=i}^j w_k$ ($w \geq 0$); $C(i, j) = g(x_j - x_i)$, x incr, g convex nondec.; $C(i, j) = \min(A_i, B_j)$ if A decreasing, B increasing; $C(i, j) = \max(0, x_j - x_i - \Delta)$ (x incr, $\Delta > 0$). Also consider LineContainer, Li Chao, monotone queues.
Time: $\mathcal{O}(N^2)$

d41d8c, 22 lines

```
template<class F>
ll knuthDP(int N, F C) {
    vector<vector<ll>> dp(N, vector<ll>(N));
    vector<vi> opt(N, vi(N));
    rep(i, 0, N) opt[i][i] = i; // set dp[i][i] if needed
    for (int i = N-2; i >= 0; i--) {
        rep(j, i+1, N) {
            ll mn = LLONG_MAX;
            ll cost = C(i, j);
            int hi = min(j-1, opt[i+1][j]);
            rep(k, opt[i][j-1], hi+1) {
                ll val = dp[i][k] + dp[k+1][j] + cost;
                if (mn >= val) {
                    opt[i][j] = k;
                    mn = val;
                }
            }
            dp[i][j] = mn;
        }
    }
    return dp[0][N-1];
}
```

DivideAndConquerDP.h

Description: Minimum cost to partition $[0, N]$ into exactly K nonempty segments, where $0 \leq K \leq N$. $C(i, j)$ returns the cost of $[i, j]$, i included and j excluded; called only with $0 \leq i < j \leq N$. $dp(g, j) = \min_{g-1 \leq k < j} (dp(g-1, k) + C(k, j))$. Requires the smallest optimal split k to be nondecreasing in j within each layer. Sufficient: $C(a, c) + C(b, d) \leq C(a, d) + C(b, c)$ for $a \leq b < c \leq d$. All finite costs and sums must fit in `ll` and be less than `LLONG_MAX`, which denotes infinity. Returns infinity if $K = 0 < N$; returns 0 if $K = N = 0$.

Usage: `ll ans = partitionDP(N, K, cost);`

Memory: $\mathcal{O}(N)$

Time: $\mathcal{O}(KN \log N)$ with $\mathcal{O}(1)$ cost queries.

d41d8c, 25 lines

```
template<class F>
ll partitionDP(int N, int K, F C) {
    assert(0 <= K && K <= N);
    vector<ll> prev(N + 1, LLONG_MAX), cur(N + 1);
    prev[0] = 0;
    auto rec = [&](auto&& self, int L, int R,
                  int lo, int hi) -> void {
        if (L >= R) return;
        int mid = (L + R) / 2, opt = lo;
        ll best = LLONG_MAX;
        rep(k, lo, min(mid, hi)) if (prev[k] != LLONG_MAX) {
            ll val = prev[k] + C(k, mid);
            if (val < best) best = val, opt = k;
        }
        cur[mid] = best;
        self(self, L, mid, lo, opt + 1);
        self(self, mid + 1, R, opt, hi);
    };
    rep(g, 1, K+1) {
        fill(all(cur), LLONG_MAX);
        rec(rec, g, N + 1, g - 1, N);
        prev.swap(cur);
    }
    return prev[N];
}
```

10.4 Debugging tricks

- `signal(SIGSEGV, [](int) { _Exit(0); });` converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). `_GLIBCXX_DEBUG` failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- `feenableexcept(29);` kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

MemoryUsage.h

Description: Maximum RSS over the lifetime of the program so far, not current RSS. Kilobytes on Linux, bytes on macOS.

d41d8c, 7 lines

```
#include <sys/resource.h>

ll getMemoryUsage() {
    struct rusage usage;
    getrusage(RUSAGE_SELF, &usage);
    return usage.ru_maxrss;
}
```

10.5 Optimization tricks

`__builtin_ia32_ldmxcsr(40896);` disables denormals (which make floats 20x slower near their minimum value).

10.5.1 Bit hacks

- `x & -x` is the least bit in `x`.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x&-x, r = x+c; (((r^x) >> 2)/c) | r` is the next number after `x` with the same number of bits set.
- `rep(b, 0, K) rep(i, 0, (1 << K))`
if `(i & 1 << b) D[i] += D[i^(1 << b)]`; computes all sums of subsets.
- `__builtin_ctz(x)` trailing zeros (int).
- `__builtin_clzll(x)` leading zeros (ll).
- `__builtin_popcountll(x)` number of ones.

10.5.2 Pragas

Pragas.h

Description: GCC pragmas. `Ofast` implies `-ffast-math` (can change floats). `avx2` can crash on old machines. `bmi, bmi2`: `tzcnt/pdep/pext.lzcnt, popcnt`: hardware `clz/popcount`. `trapv` aborts on signed overflow (slow; debug). Do not combine `trapv` with `Ofast` if you need traps.

d41d8c, 3 lines

```
#pragma GCC optimize("Ofast,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
// #pragma GCC optimize("trapv")
```

FastMod.h

Description: Compute `a % b` about 5 times faster than usual, where `b` is constant but not known at compile time. Returns a value congruent to `a (mod b)` in the range $[0, 2b)$. 0 included, $2b$ excluded.

d41d8c, 8 lines

```
typedef unsigned long long ull;
struct FastMod {
    ull b, m;
    FastMod(ull b) : b(b), m((-1ULL / b) {}
    ull reduce(ull a) { // a % b + (0 or b)
        return a - (ull)((__uint128_t(m) * a) >> 64) * b;
    }
};
```

FastInput.h

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.

Usage: `./a.out < input.txt`

Time: About 5x as fast as `cin/scanf`.

d41d8c, 17 lines

```
inline char gc() { // like getchar()
    static char buf[1 << 16];
    static size_t bc, be;
    if (bc >= be) {
        buf[0] = 0, bc = 0;
        be = fread(buf, 1, sizeof(buf), stdin);
    }
```

```
}
    return buf[bc++]; // returns 0 on EOF
}

int readInt() {
    int a, c;
    while ((a = gc()) < 40);
    if (a == '-') return -readInt();
    while ((c = gc()) >= 48) a = a * 10 + c - 480;
    return a - 48;
}
```

BumpAllocator.h

Description: When you need to dynamically allocate many objects and don't care about freeing them. `new X` otherwise has an overhead of something like `0.05us + 16 bytes` per allocation.

d41d8c, 8 lines

```
// Either globally or in a single class:
static char buf[450 << 20];
void* operator new(size_t s) {
    static size_t i = sizeof buf;
    assert(s < i);
    return (void*)&buf[i -= s];
}
void operator delete(void*) {}
```

SmallPtr.h

Description: A 32-bit pointer that points into BumpAllocator memory.

"BumpAllocator.h" d41d8c, 10 lines

```
template<class T> struct ptr {
    unsigned ind;
    ptr(T* p = 0) : ind(p ? unsigned((char*)p - buf) : 0) {
        assert(ind < sizeof buf);
    }
    T& operator*() const { return *(T*)(buf + ind); }
    T* operator->() const { return &*this; }
    T& operator[](int a) const { return (&this)[a]; }
    explicit operator bool() const { return ind; }
};
```

BumpAllocatorSTL.h

Description: BumpAllocator for STL containers.

Usage: `vector<vector<int, small<int>>> ed(N);`

d41d8c, 14 lines

```
char buf[450 << 20] alignas(16);
size_t buf_ind = sizeof buf;

template<class T> struct small {
    typedef T value_type;
    small() {}
    template<class U> small(const U&) {}
    T* allocate(size_t n) {
        buf_ind -= n * sizeof(T);
        buf_ind &= 0 - alignof(T);
        return (T*)(buf + buf_ind);
    }
    void deallocate(T*, size_t) {}
};
```

Techniques (A)

techniques.txt

160 lines

Recursion
Divide and conquer
 Finding interesting points in N log N
Algorithm analysis
 Master theorem
 Amortized time complexity
Greedy algorithm
 Scheduling
 Max contiguous subvector sum
 Invariants
 Huffman encoding
Graph theory
 Dynamic graphs (extra book-keeping)
 Breadth first search
 Depth first search
 * Normal trees / DFS trees
 Dijkstra's algorithm
 MST: Prim's algorithm
 Bellman-Ford
 Konig's theorem and vertex cover
 Min-cost max flow
 Lovasz toggle
 Matrix tree theorem
 Maximal matching, general graphs
 Hopcroft-Karp
 Hall's marriage theorem
 Graphical sequences
 Floyd-Warshall
 Euler cycles
 Flow networks
 * Augmenting paths
 * Edmonds-Karp
 Bipartite matching
 Min. path cover
 Dilworth's theorem / chain covering
 Topological sorting
 Strongly connected components
 2-SAT
 Cut vertices, cut-edges and biconnected components
 Edge coloring
 * Trees
 Vertex coloring
 * Bipartite graphs (=> trees)
 * 3^n (special case of set cover)
 Diameter and centroid
 K'th shortest path
 Shortest cycle
Dynamic programming
 Knapsack
 Coin change
 Longest common subsequence
 Longest increasing subsequence
 Number of paths in a dag
 Shortest path in a dag
 Dynprog over intervals
 Dynprog over subsets
 Dynprog over probabilities
 Dynprog over trees
 3^n set cover
 Divide and conquer
 Knuth optimization

Convex hull optimizations
RMQ (sparse table a.k.a 2^k-jumps)
Bitonic cycle
Log partitioning (loop over most restricted)
Combinatorics
 Computation of binomial coefficients
 Pigeon-hole principle
 Inclusion/exclusion
 Catalan number
 Pick's theorem
Number theory
 Integer parts
 Divisibility
 Euclidean algorithm
 Modular arithmetic
 * Modular multiplication
 * Modular inverses
 * Modular exponentiation by squaring
 Chinese remainder theorem
 Fermat's little theorem
 Euler's theorem
 Phi function
 Frobenius number
 Quadratic reciprocity
 Pollard-Rho
 Miller-Rabin
 Hensel lifting
 Vieta root jumping
Game theory
 Combinatorial games
 Game trees
 Mini-max
 Nim
 Games on graphs
 Games on graphs with loops
 Grundy numbers
 Bipartite games without repetition
 General games without repetition
 Alpha-beta pruning
Probability theory
Optimization
 Binary search
 Ternary search
 Unimodality and convex functions
 Binary search on derivative
Numerical methods
 Numeric integration
 Newton's method
 Root-finding with binary/ternary search
 Golden section search
Matrices
 Gaussian elimination
 Exponentiation by squaring
Sorting
 Radix sort
Geometry
 Coordinates and vectors
 * Cross product
 * Scalar product
 Convex hull
 Polygon cut
 Closest pair
 Coordinate-compression
 Quadtrees
 KD-trees
 All segment-segment intersection

Sweeping
 Discretization (convert to events and sweep)
 Angle sweeping
 Line sweeping
 Discrete second derivatives
Strings
 Longest common substring
 Palindrome subsequences
 Knuth-Morris-Pratt
 Tries
 Rolling polynomial hashes
 Suffix array
 Suffix tree
 Aho-Corasick
 Manacher's algorithm
 Letter position lists
Combinatorial search
 Meet in the middle
 Brute-force with pruning
 Best-first (A*)
 Bidirectional search
 Iterative deepening DFS / A*
Data structures
 LCA (2^k-jumps in trees in general)
 Pull/push-technique on trees
 Heavy-light decomposition
 Centroid decomposition
 Lazy propagation
 Self-balancing trees
 Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)
 Monotone queues / monotone stacks / sliding queues
 Sliding queue using 2 stacks
 Persistent segment tree

Alphabetical index

.bashrc, 1
.vimrc, 1
12-fold way, 16
2sat.h, 20
3D, 31
3dHull.h, 31

AhoCorasick.h, 26
Angle.h, 27

Bézout’s identity, 14
Bell numbers, 16
BerlekampMassey.h, 9
Bernoulli numbers, 16
BiconnectedComponents.h, 20
BigInt.h, 37
BinaryLifting.h, 22
BinaryTrie.h, 32
Binomial distribution, 3
Binomials, 16
Bit hacks, 38
Blossom.h, 20
BumpAllocator.h, 38
BumpAllocatorSTL.h, 38
Burnside’s lemma, 16

CartesianTree.h, 7
Catalan numbers, 16
Centroid.h, 22
CircleIntersection.h, 27
CircleLine.h, 27
CirclePolygonIntersection.h, 28
Circles, 27
CircleTangents.h, 27
circumcircle.h, 28
ClosestPair.h, 29
Coloring, 21
ConstantIntervals.h, 37
ContinuedFractions.h, 14
Continuous distributions, 3
ConvexHull.h, 29
Convolutions, 12
CRT.h, 14
Cycles, 16

Debugging tricks, 38
DelaunayTriangulation.h, 30
Demands and lower bounds, 19
Derangements, 16
Derivatives/Integrals, 3
Determinant.h, 10
DFS algorithms, 20
DFSMatching.h, 19
Dinic variants and capacity edits, 18
Dinic.h, 18

Dinic2.h, 18
DirectedMST.h, 23
Discrete distributions, 3
DivideAndConquerDP.h, 38
Divisibility, 14
DominatorTree.h, 21
Dynamic programming, 37

EdgeColoring.h, 21
EdmondsKarp.h, 17
Equations, 2
Erdős–Gallai theorem, 24
Estimates, 15
euclid.h, 14
Eulerian numbers, 16
EulerTourTree.h, 32
EulerWalk.h, 21
Exponential distribution, 3

Factor.h, 14
Factorial, 15
FastDelaunay.h, 30
FastEratosthenes.h, 14
FastFourierTransform.h, 12
FastFourierTransformMod.h, 12
FastInput.h, 38
FastKnapsack.h, 37
FastMod.h, 38
FastSubsetTransform.h, 13
FenwickTree.h, 7
FenwickTree2d.h, 7
First success distribution, 3
FloorBlocks.h, 15
FracBinarySearch.h, 15
Fractions, 14
Fundamentals, 17

General purpose numbers, 16
Geometric primitives, 26
Geometry, 2
GlobalMinCut.h, 18
GoldenSectionSearch.h, 9
GomoryHu.h, 18

HalfplaneIntersection.h, 28
hash.sh, 1
Hashing-codeforces.h, 25
Hashing.h, 25
HashMap.h, 4
Heuristics, 21
HillClimbing.h, 9
HLD.h, 23
HopcroftKarp.h, 19
HullDiameter.h, 29

I/O, 1
Input.h, 1
InsidePolygon.h, 28
IntDeterminant.h, 10
Integrate.h, 10
IntegrateAdaptive.h, 10
IntervalContainer.h, 36
IntervalCover.h, 36
Intervals, 36
IntPerm.h, 15

Johnson’s algorithm, 17

kdTree.h, 30
KMP.h, 24
KnuthDP.h, 37
Kőnig, path cover, Dilworth, 19

Labeled unrooted trees, 16
Lagrange.h, 9
LazySegmentTree.h, 5
LCA.h, 22
LiChao.h, 6
LinearRecurrence.h, 9
LinearSieve.h, 14
linearTransformation.h, 27
LineContainer.h, 6
lineDistance.h, 26
LineHullIntersection.h, 29
lineIntersection.h, 27
LineProjectionReflection.h, 27
LinkCutTree.h, 35
Lucas’ Theorem, 16

Manacher.h, 24
ManhattanMST.h, 30
Markov chains, 4
Matching, 19
Math, 24
Mathematical constants, 2
Matrices, 10
Matrix.h, 6
MatrixInverse-mod.h, 12
MatrixInverse.h, 11
MaximalCliques.h, 21
MaximumClique.h, 22
MaximumIndependentSet.h, 22
MemoryUsage.h, 38
MillerRabin.h, 14
MinCostMaxFlow.h, 17
MinCut.h, 18
MinimumEnclosingCircle.h, 28
MinimumVertexCover.h, 19
MinPlusConvolution.h, 35
MinRotation.h, 24

Monash

Misc. algorithms, 37
 Misc. Point Set Problems, 29
 Mobius Function, 15
 Mobius.h, 15
 ModInverse.h, 13
 ModLog.h, 13
 ModMulLL.h, 13
 ModPow.h, 13
 ModSqrt.h, 13
 ModSum.h, 13
 Modular arithmetic, 13
 ModularArithmetic.h, 13
 MonotonicCHT.h, 6
 MonotonicMap.h, 8
 MoQueries.h, 8
 multinomial.h, 16

 Network flow, 17
 Normal distribution, 3
 Number of Spanning Trees, 24
 NumberTheoreticTransform.h, 12

 OfflineDynamicConnectivity.h, 5
 OnSegment.h, 27
 Optimization, 9
 Optimization tricks, 38
 OrderStatisticTree.h, 4
 Output.h, 2

 Partition function, 16
 Partitions and subsets, 16
 Permutations, 15
 PersistentSegmentTree.h, 35
 phiFunction.h, 14
 Point.h, 26
 Point3D.h, 31
 PointInsideHull.h, 29
 Poisson distribution, 3
 PolygonArea.h, 28

PolygonCenter.h, 28
 PolygonCut.h, 28
 Polygons, 28
 PolygonUnion.h, 29
 PolyhedronVolume.h, 31
 PolyInterpolate.h, 9
 Polynomial.h, 8
 Polynomials and recurrences, 8
 PolyRoots.h, 8
 Pragmas, 38
 Pragmas.h, 38
 PrefixSum.h, 6
 Primality, 14
 Primes, 15
 PrimitiveRoot.h, 15
 Probability theory, 3
 PushRelabel.h, 17
 Pythagorean Triples, 15

 Quadrilaterals, 2
 QuadRoots.h, 8

 Random.h, 1
 Recurrences, 2
 Reroot.h, 23
 RMQ.h, 7
 Rope.h, 4
 RREF.h, 11

 SCC.h, 20
 SegmentDistance.h, 26
 SegmentIntersection.h, 27
 SegmentTree.h, 4
 SegmentTreeBeats.h, 33
 Series, 3
 sideOf.h, 27
 Simplex.h, 10
 SmallPtr.h, 38
 SolveLinear.h, 28

SolveLinear2.h, 11
 SolveLinearBinary.h, 11
 SparseLazySegmentTree.h, 34
 SparseSegmentTree2d.h, 34
 Spherical coordinates, 3
 sphericalDistance.h, 31
 StaticRangeQuery.h, 8
 SteinerTree.h, 24
 Stirling numbers of the first kind, 16
 Stirling numbers of the second kind, 16
 SubMatrix.h, 5
 SuffixArray.h, 24
 SuffixTree.h, 25
 Sums, 3

 techniques.txt, 39
 template.cpp, 1
 TernarySearch.h, 37
 TopoSort.h, 17
 Treap.h, 7
 Trees, 22
 Triangles, 2
 Tridiagonal.h, 12
 Trigonometry, 2
 troubleshoot.txt, 1

 Uniform distribution, 3
 UnionFindRollback.h, 5

 VirtualTree.h, 22

 WaveletTree.h, 4
 WeightedMatching.h, 19

 XORBasis.h, 11

 Zfunc.h, 24